DEVELOPMENT OF FIRE VS FIRE - AN AUTOMATED MODULAR BROWSER EXTENSION FOR IDENTIFYING AND FILTERING OF AI GENERATED, AND OTHER CONTENT IN USER INTERFACES.

5/5/26

L Armstrong
SUPERVISED BY DR. PAUL BAKAKI | EDGE HILL UNIVERSITY

CIS3425 CW2 PART 2 – PROJECT REPORT AND ARTEFACT
THIS REPORT IS SUBMITTED IN PARTIAL FULFILMENT OF THE REQUIREMENTS FOR THE BSC HONOURS COMPUTING DEGREE AT EDGE HILL UNIVERSITY.

Abstract

AI generated content has become commonplace, public trust has evaporated due to the increasingly impossible task of distinguishing between misinformation and slop, from reality. Western citizens and the elderly are among the worst populations for AI literacy. This project addresses the growing crisis by developing Fire vs Fire, a fully automatic browser extension designed to identify and filter AI generated content in real time for free.

Unlike existing tools that rely on manual user input and paid subscription models, Fire vs Fire uses locally run machine learning classification. This approach ensures data privacy, automatically scans all content without manual direction, eliminates rate limits and caps, increases computation speed by orders of magnitude, allows immense user control, and the end user gets it for free.

The prototype demonstrates the benefits of the new first of a kind method with a high rate of accuracy, the tool features a open and modular Architecture and intuitive GUI, allowing for fully personalized content filtering and highly customizable handling of suspected content.

Acknowledgements

The art for this project was created by the certified human, Beti Gray.

This project was supervised by Dr. Paul Bakaki of the Edge Hill University computer science department.

Contents

Table of contents

Abstract 1

Acknowledgements 1

Contents 1

Introduction 5

- Background information 5
- Why is this project necessary 6
- SMART Objectives 7
- Glossary 7

Background and Literature Review 9
Corporate Watermarking 9
Competitive Analysis 9
Detection Strategies 11
The Research Gap 14
Problem Analysis and System Requirements 14
Problem Analysis 14
Functional Requirements 15
Non-Functional Requirements 16
Design justification 16
Justification of AI classification against traditional algorithms 16
Justification of separate backend server and alternative options. 17
Potential alternatives to web browser based classification 19
Development strategy justification 19
System Architecture 19
Modularity 19
Data flow 20
Implementation 22
Browser extension – content handling 22
Browser extension – UI 28
Visual Flagging System 33
Browser extension challenges and solutions 35
Local server messaging 36
Server side classifier management 39
Base classifier 42
Classification Orchestration 43
Data Fetching and Preprocessing 45
AI Image ConvNeXt-Large artifact Classifier 47
ResNet-50 Spider Classifier 52
Hybrid DistilBERT + Features text model 56
Software Testing and Known Problems 61
Conclusions 69
Critical Evaluation 70



Browser extension evaluation 70
Image classifier evaluation 76
Text classifier evaluation 81
Future Work 85
Video and audio 85
Additional features 85
Additional platforms 86
Legal, Social, Economic, and Ethical Considerations 87
Legal Considerations 87
Social Considerations 87
Economic Considerations 88
Ethical Considerations 88
References 88
Appendices 94

Figure 1 logarithmic scale for leading AI project PetaFLOPs over time (Our World in Data, 2026) 5
Figure 2 image showing Fourier transform differences between AI generated and authentic images (seismocat, 2025) 12
Figure 3 Spectrogram comparison of original and disguised voice samples (Dilbar et al., 2025) 13
Figure 4 extreme checkerboarding from older image generation models, modern implementations retain this defect to a lesser extent (Odena, Dumoulin and Olah, 2016) 13
Figure 5 Proposed architecture comparison of local server-based compute (left) and integrated web compute (right) 17
Figure 6 High level system Architecture diagram of Fire vs Fire 20
Figure 7 Fire vs Fire logos, red and green 29
Figure 8 Design of ConvNext AI classifier 48
Figure 9 design of ResNet 50 spider AI classifier 54
Figure 10 ResNet50 classifier sucsessfull page classificatoin 56
Figure 11 V2 AI classification of AI models vs AVG text length 57
Figure 12 Hybrid AI text classifier design 59
Figure 13 proof of native ARM CPU inference 61
Figure 14 full screen options page instructions 62
Figure 15 classifier creation and removal 63
Figure 16 status display system 64
Figure 17 google search classification complete 64
Figure 18 google image search classifications and flagging 65
Figure 19 Wikipedia page classifications and flagging 66

Figure 20 dual category classification example 66
Figure 21 multi classifier category example 67
Figure 22 reddit stories and GPT classification and flagging 68
Figure 23 right click manual flagging system 69
Figure 24 GUI example 71
Figure 25 classification status display 72
Figure 26 developer controls GUI 73
Figure 27 flagging example 74
Figure 28 dual flagging example 75
Figure 29 content detection and flagging of JavaScript provided content 76
Figure 30 ConvNext classification test results graph 77
Figure 31 ConvNext classification test results table 77
Figure 32 partitioned images examples from the dataset 79
Figure 33 text classification results 82
Figure 34 Generate 100 paragraphs, text classification results (150 char minimum sample) 82
Figure 35 text classification test length variation 83
Figure 36 text classification test length variation graphs 83

Introduction

Background information

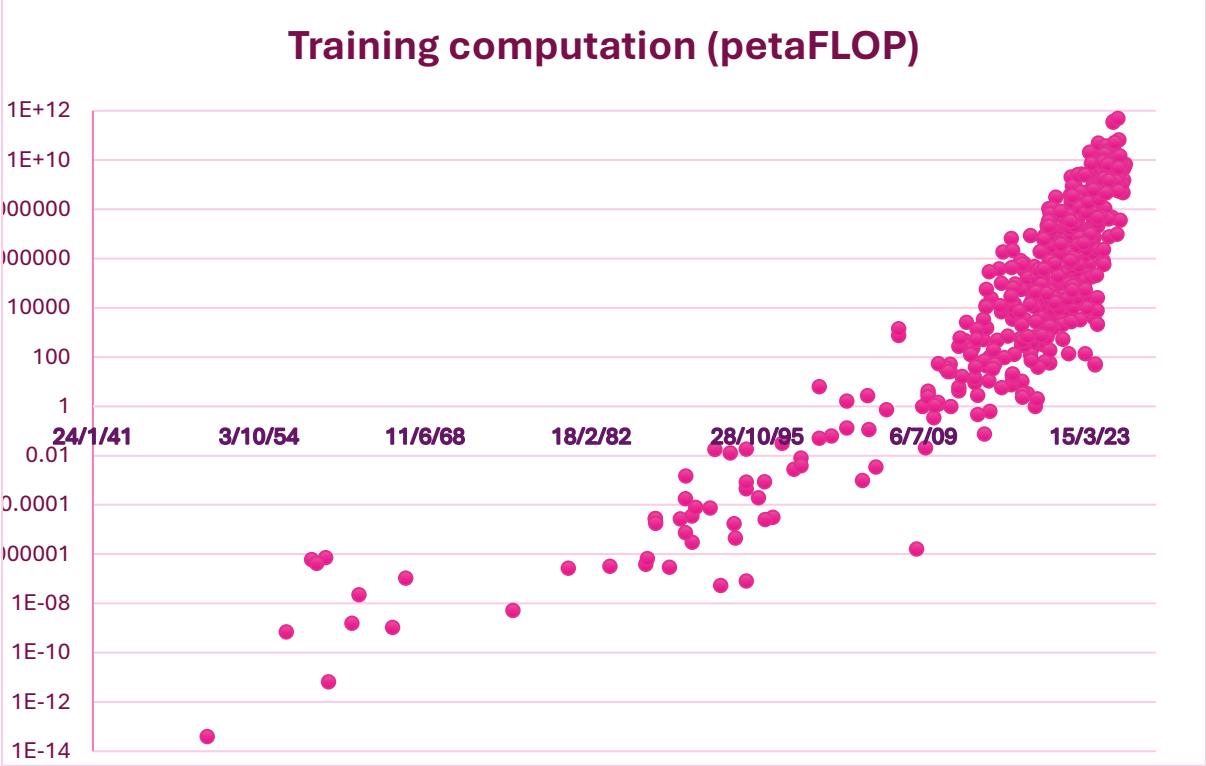


Figure 1 logarithmic scale for leading AI project PetaFLOPs over time (Our World in Data, 2026)

AI (artificial intelligance) is not a modern invention, the first use of AI was a simple toy mouse in 1950 called Thesius (Shannon, 1950). Thesius used trial and error to navigate mazes in a 5x5 grid, his brain was located beneath the table and consisted of a whopping 5600 flops worth of electomagnetic telephone relays. In the 76 years since thesius, the average compute power used to train the next bleeding edge AI project has doubled roughfly every 5.2 months (Our World in Data, 2026), resulting in the latest grock 4 model using 500 yottabytes (24 zeroes) to train and the market cap of the Nvidia corporation, who manufacrure the compute, being higher than the GDP of all exept the 3 largest economies in the world. (Macrotrends, 2026) (IMF, 2024). AI has inumerable posibilities and is aplicable to almost every aspect of modern scosiety, however marketing hype and a missunderstanding of the scale of what the word “AI” encompases has driven over 99% of the worlds effort to be directed towards generative AI, this rapid proliferation has outpaced the public's ability to distinguish between authentic and synthetic media, furthermore AI generated content is quick and easy to mass produce with little to no skill, despite the populatoin knowing that AI generated content is not authentic, it remains as one of the most effective tools for scams and propogranda (Explosive Media and Iran, 2026), (ICAEW Insights, 2025), (cyberonix, 2025).

Why is this project necessary

Survey data indicates that a majority of the UK population views the integration of AI generated content negatively, frequently associating it with words such as "impersonal," "lazy," and "untrustworthy" (Bynder, 2024). This skepticism is worsened by a significant lack of education, 73% of individuals in the UK report having received no AI-related training (Allen, 2025), leaving them vulnerable to digital deceptions.The consequences have manifested in severe real-world incidents. For example, a UK family was hospitalized after consuming toxic mushrooms identified using an AI-generated foraging guide they bought from amazon, beleavig it to be authentic (ecoloma, 2024). Insurance companies such as admiral report a 71% rise in fraud from 2024 to 2025, partialy blaming the increased use of AI to manipulate or fabricate evidance (Thomas, 2026). numerous financial frauds have been successfully executed using deepfake audio and video of corporate executives, loosing multiple millions of pounds (ICAEW Insights, 2025), (cyberonix, 2025).

The ability of large entities to manipulate people, be it for buying a product, supporting a war, or beleaving a lie, has always relied on orginisations carefully producing idieas and various forms of media to feed the population (one to manny), however with the invention of inteligent macheanes and the mass collection of personal data, it has become possible for an orginisation to dictate an end goal (buy x product), and let AI convince individuals of the idiea, in an inteligent curated one to one timeline of exposure through multiple modalities (Noels , Rogiers and Buyl , 2026). Since 2024, advertisers on Meta platforms can use their Advantage+ Creative program to generate variations of ADs with diferent backgrounds, colours, music or text based on Meta's personal profile of each individual user's habbits, lifestyle, and current situaltion (Meta, 2022). Google uses a similar system called Performance Max across its platforms such as google search, youtube and gmail (Google, 2025). For 8 hours in may 2025, xAI's grock AI chatbot pivoted every prompt it was given to push the narative of white genocide in south africa, xAI claimed a rogue employee made the change to the backend of grock (Gold and Reilly , 2025), however critics noted that the incident demonstrated how easily a platform owner's specific political interests can override a model's intended neutrality, had a more subtle aproach been used, an AI ecosystem may be capable of silently manipulate populations with extreame effectiveness (O'Brien and Associated Press, 2025).

Regulatory authorities are unequipt to deal with the pressures to balance public saftey and inovation, current protective measures are insufficient. While some orginisations have begun to develop watermarking standards like Google's SynthID or Meta's AudioSeal, they are designed for computerised detection rather than human recognition, and social media platforms, who benefit from increased user interaction, have been slow to implement any protective mesures. in this envoroment where governments and corpirations move to slwoly, there is a critical need for an autonemous, scaleable solution that empowers users to verify content independently. By providing a tool that operates in real-time on consumer hardware, this project aims to mitigate the risks of misinformation and educate users on how to detect AI generated content.

Purpose and Objectives

This project aims to develop a functional browser extension capable of automatically detecting and flagging AI-generated content in real-time without the need for user input. Modular machine learning clasification models will be utilized to acheave the highest level of accuracy possible, models will compute localy on reasonable consumer level hardwhare such as personal computers and laptops, the solution eliminates the need for cloud-based rate limits and fees, opening the door for a true content filter that checks all user facing content and protects user privacy by avoiding external data transmission.

SMART Objectives

To achieve the project aim, the following objectives were established during the planning phase of the project:

1. Research & Literature Review: Conduct an in-depth investigation into peer-reviewed literature and industry reports regarding AI detection methods, watermarking standards, and existing browser-based implementations by 14th of November 2025.
2. Prototype Extension Design: Design and implement a working prototype browser extension by 28th of November capable of parsing webpage content and communicating with a local backend for real-time classification.
3. Machine Learning Development: Develop prototype machine learning models for classifying text, images, video, and audio by 6th of February, targeting a processing speed of at least five units per second with a minimum detection accuracy of at least 75%.
4. Systematic Testing: Perform comprehensive testing across a minimum of 20 diverse websites by 27th of March to measure detection accuracy, identify false positives, and evaluate hardware processing speeds.
5. Documentation & Evaluation: Produce a final research and development report of approximately 8,000 words by 24th of April, critically evaluating the implementation against the original specification and reflecting on the project lifecycle.

Glossary

AI – Artificial intelligence, a type of computerised logic based on biological brains, can be used for complex tasks where traditional programming is not adaptable enough or too complex.

ML – Machine learning, A subset of AI focused on the development of algorithms that can identify patterns or make decisions based on data without being explicitly programmed for every scenario.

Model – A model is a specific instance of an AI that has been trained and can now be implemented to work on whatever task it was designed for, or trained further

Training – The process of refining the random behaviours of AI models towards a desired goal, training is a long and computationally intensive process



Generative AI – A category of artificial intelligence systems capable of creating new, original content by learning and replicating the underlying patterns of massive datasets. In the context of this project, Generative AI represents the source of synthetic content that this project seeks to combat by targeting the unique artifacts left behind by generative processes.

Authentic – for this project Authentic media refers to media that was not created or edited using generative AI

Synthetic content – for this project, synthetic content refers to media that was created or edited using generative AI

Local Processing – The execution of computational tasks directly on the user's hardware.

Cloud Processing – A computing paradigm where data is transmitted over the internet to be processed on remote servers.

Compute – has dual meanings used in this paper, the action of computing means using a computer's processing components to run algorithms, inference is a intensive form of computing. To use compute, or have used compute refers to the amount of computing power used for a specific task, measured in FLOPs (Floating Point Operations), you can increase the amount of compute used for a task by using more powerful processors and by using your processors for a longer period of time.

Inference – The operational phase of a machine learning model where it applies its learned patterns to new, unseen data to generate a prediction or classification in real-time.

Classifier – For this project, a Classifier refers to a specific, algorithm responsible for categorizing a piece of digital media, such as Synthetic or Human-generated. A classifier performs classification within a category. Classifiers developed during this project are pre-trained AI models, however they are implemented in a way that also supports traditional algorithmic methods.

Category – A Category represents a type of content that a user wishes to filter, for example, AI generated images. Categories contain one or many classifiers.

Modality – Modality refers to the medium of a specific piece of content, the 4 modalities discussed in this paper are image, text, video and audio

Backbone – the backbone of an AI refers to the first part of an AI model, it is responsible for looking at the content and extracting information and insights from it

Head – the head of an AI refers to the last part of an AI model, it is responsible for looking at extracted information and determining the final output

Background and Literature Review

Corporate Watermarking

While governments have not yet introduced significant regulation for AI generate content, Industry leaders have moved toward embedding invisible signatures within synthetic content for computerised detection, notable examples include Google's SynthID, which utilizes a process called tournament sampling (Omidi, Dong and Wang , 2026), a process used to bias token selection for text generation, and embeds mathematical patterns directly into the images and video. Similarly, AudioSeal by Meta (Meta Superintelligence, 2025) uses audio watermarking for robust computerised detection, even after signal transformations like compression or noise addition. Some mainstream AI video/image generators also watermark content with visible watermarks for human detection; however these can be easily removed by simply cropping or editing content (Imam, 2026) if the producer does not wish for their intended audience to know what they are consuming.

These standards are excellent implementations of watermarking at the point of content generation however these watermarks are designed to be invisible to humans, for Google's SynthID the cryptographic keys required for verification are held exclusively google, independent parties such as journalists, researchers, and the general public can only use Google's proprietary tools which are limited to 10 per day and operate exclusively through the Gemini AI as an interface (Google, 2019). Furthermore, SynthID only checks for Google's own signature. While Meta's AudioSeal is only compatible with their audio generation, it is implemented in less experimental deployments, all content uploaded to meta platforms are automatically checked for AudioSeal watermarking and the project is open source on GitHub for others to use. The issue is that major platforms such as Spotify, YouTube, X (formally twitter), Amazon, and many more (Amazon, 2026) (Youtube, 2019) (Spotify, 2025) (X, 2024) are prepared to enforce other policies such as bans on profanity or explicit content with automatic machine learning solutions, they are reluctant to implement invisible watermarking standards to their platforms, instead opting for voluntary declaration of synthetic content, most without penalties for failure to declare. The only major platform to attempt true policing of AI generated content is TikTok (Storrito, 2026) with the use of the C2PA (Coalition for Content Provenance and Authenticity) standard to recognize metadata in uploaded files, which can be stripped from files by uploaders. Even if major platforms were to adopt watermarking standards, bad actors would simply use alternative or older models for content generation, thus the need for a true, universal solution, without the requirement of collaboration from AI generations corporations whose income is dependant on the content they produce fitting into existing spaces as easily as possible (Liu et al., 2026), social media platforms who benefit from increased user interaction (EDMO, 2026), or voluntary declaration by bad actors such as scammers or propagandisers.

Competitive Analysis

The need for third party tools such as browser extensions has been recognised by others already, external tools such as Hive or Winston AI offer broader detection, but introduce

significant barriers to entry for the average user and are fundamentally flawed in their design. These tools are often manual, requiring the user to proactively upload an image select it on a webpage for classification, and are frequently locked behind paid subscription models, strict rate limits or both. Since 73% of the UK public has received no AI training (Allen, 2025), a tool that requires manual suspicion and a financial commitment is unlikely to reach the most vulnerable populations or have a significant impact on the media landscape.

The largest competitor in this space is Hive AI, who maintain a wide range of AI services including generative AI and AI detection of AI, they offer AI classification of images and text through their API and through a chrome extension, however due to the cloud based nature of the system, they are required to charge 5$ per 1000 per image requests and $0.5 per 1000 text sections, limited to 1024 characters in length (Hive AI, 2024). Furthermore, the service is reactive, requiring a user to select an image and send it to hive's servers for classification, instead of the proactive approach of Fire vs Fire with unlimited uses and comparatively negligible cost for a user's own power.

An alternative to hive is Winston AI, which positions itself as a higher accuracy solution, designed for publishing and marketing professionals. However, the limited free trial is restricted to a one-time 7-day window with a maximum usage of 2k credits. Using their cheapest "Essential" tier, users are charged a minimum of £8 per month when paying annually, to access 80,000 credits, where one credit is consumed for every single word of text scanned, and 300 credits per image classification, Similar to Hive their browser extension and website are reactive measures, a user must manually highlight text or select an image for classification (Winston AI, 2026).

Similarly, GPTZero is a reactive cloud bsed text classification tool, while they offer a limited free tier of 10,000 words per month in their website, their "Essential" premium plan costs £13 per month when paying annually, and is capped at 300,000 words per month, where credits do not carry over, their chrome extension is a paid feature, only the website can be accessed with the free version, requiring users to copy paste suspected text (GPTZero, 2023).

These 3 prominent systems are examples of a cloud based compute method similar to that of this project, compared to Fire vs Fire these services are extremely expensive to use to the point of the general public not using them, and their main demographic being businesses and educational institutions. While the operational costs for these services are not public, we know that some of the subscription revenue covers the cost of electricity used for compute, however services like Hive AI and GPTZero must also cover the cost of building datacentres, research and development costs, maintenance, marketing, and maintain a profit margin, none of which are concerns to users of this project, who are using the compute of their own devices they own already, and using classification models provided for free with this project or that they developed on themselves.

While it is impossible to estimate the running cots of fire vs fire to the user accurately due to vast verity in speed and efficiency of processors used by consumers, we can make a rough estimate using the power draw and computation times observed during testing. Using a flagship workstation laptop equipped with modern snapdragon x elite processor, Fire vs Fire

was demonstrated to consistently finish full page classification in under 3 seconds. Observed power draw of whole system (including laptop screen, ram, multiple open apps, etc) sat at ~15w while idle when opening a high density google image search with classification enabled, power draw spikes to ~65w, using 2026 UK peak energy rates from (Morley, 2025), a sustained 50W power delta operated continuously for one year would theoretically cost at most £60, however this estimate will never be reached, the usage of Fire vs Fire is infrequent, short bursts, likely never exceeding £5 annually. compared to the cheapest option from the 3 systems discussed above, Winston AI essentials £96 annual plan allowing for 266 images to be classifications, or approximately half of the images loaded for a single google image search without scrolling down. the estimated running cost to the user of Fire vs Fire for the same number of classifications is negligible ($£1.9*10^{-6}$).

Non ML methods of creating a AI filtering browser extensions exist but are limited in functionality, many examples exist of browser extensions similar to (hadesTTW AI Blocker, 2025), which removes the AI overview element from google search, and hides AI features in google workspace apps, however the claim on the extension page that this method "Blocks all AI content on the internet" is false. A method with higher potential is represented by implementations similar to (SkipSlop by Tanathy, 2026), this extension uses crowdsourcing detection to block AI content while browsing, users can report content as AI, following users who visit the same page afterwards will not see reported content, if this browser extension or one like it assembled a large enough user base then it could serve as a genuine alternative to local classification, however such an extension is useless without a large user base and none exist that are at all active, perhaps if a major web browser launched a similar service natively as part of their app it would gain enough traction to become viable.

Detection Strategies

The detection of synthetic content relies on identifying generative artifacts created by the mathematical processes of generative models, these are often invisible, statistical or structural patterns that we can use for AI classification (Yu and Xu, 2024). Detection of non AI categories is less important to this project, users are encouraged to research and develop their own models, it Is beyond the scope of this project to develop other classifiers.

There exist many methods for AI generation, within each modality we must take advantage of the most common traces left behind for accurate classification. While the machine learning classifiers used in this project are considered black boxes, meaning that it is impossible for a human developer to understand why the internal weights acquired through training are superior to any other set of randomly generated configurations, it is possible to influence the models towards higher accuracy by manipulating input before it reaches the classifier, tweaking training processes and providing a large and diverse dataset (Meng and Xing, 2024).

For Visual Media (Image/Video), Most premier image generators can be categorised as GANs such as BigGAN and CycleGAN, or the more modern Diffusion models such as Stable Diffusion, DALL-E 3 and Midjourney. Both of these architecture use layers to increase image resolution which often leaves checkerboard or grid patterns artifacts in the high-frequency components of the image (Erukude, Marella and Veluru, 2025). We can use this defect to our

advantage by feeding image classifiers the standards red, blue, green values required for coloured images, alongside a unique 4th layer generated using a 2D Discrete Fourier Transform (DFT), this layer has higher (brighter) values in areas with high frequency variability such as along the previously invisible grid lines that can be left from up sampling during image generation. While the classifier does not see the image in the same way as humans do, and may have trained itself to consider the possibility of grid lines in the image without manual processing beforehand, by providing the Fourier Transform layer we can increase the likelihood of the AI learning that these grid lines are significant.

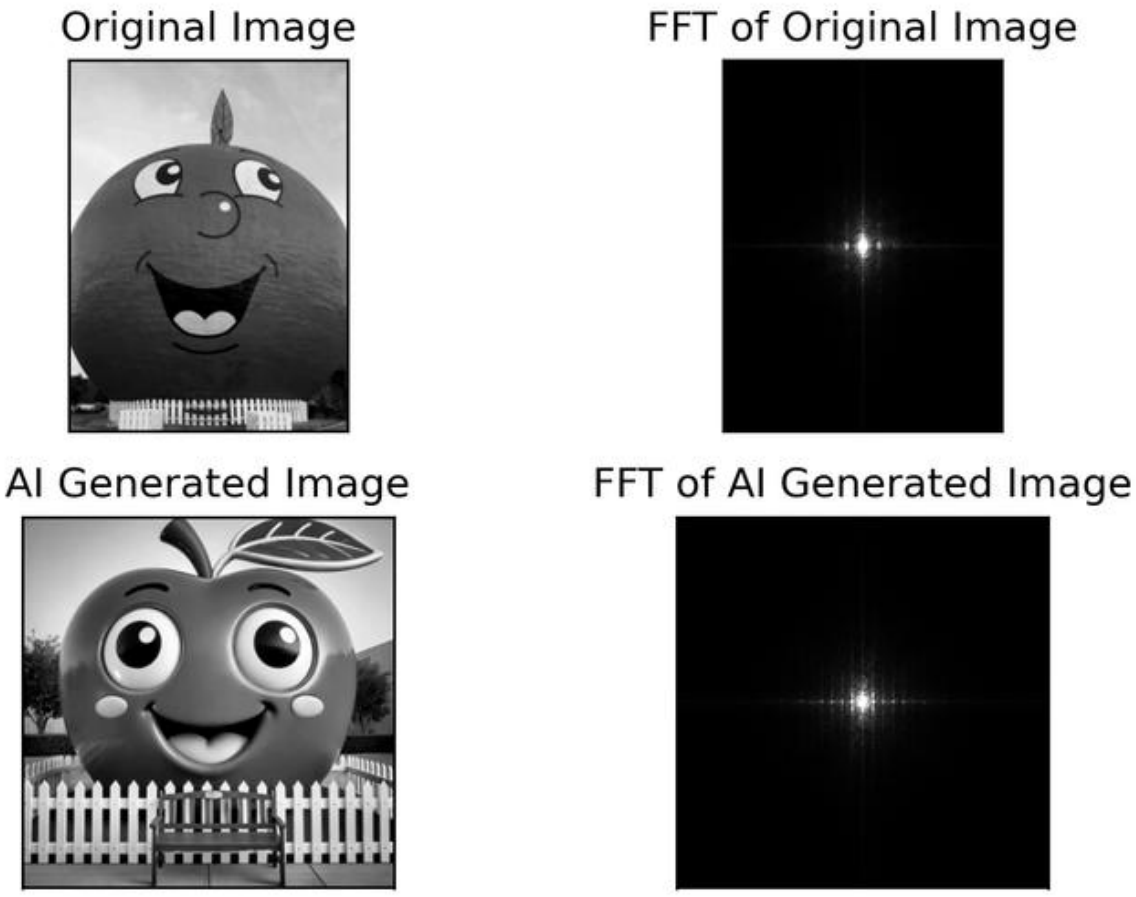


Figure 2 image showing Fourier transform differences between AI generated and authentic images (seismocat, 2025)

Other artifacts left over in visual media include temporal jitter or flickering in generated video, where the background or lighting changes slightly between frames, while a generic image classifier could be used for video classification, we should take advantage of the additional information available and used 3D models (include time) to enable classifiers to comprehend differences between frames of video (Hemant Appa Tirmare, 2026) (Zhang et al., 2021). And Perspective and Attribute Errors, AI often fails at complex semantic relationships such as impossible vanishing points (Borji , 2024) and AI models still struggle with anomalous limb counts or mismatched earrings (Ma et al., 2023), physical features such as these are valuable for ML classifiers because they may be easier to pick up on due to their backbones likely being trained for object detection more than complex pattern recognition.

Synthetic voices often exhibit vocoder artifacts, unnatural energy distributions in the higher frequency ranges. These appear as sharp, vertical phase discontinuities when viewed as a spectrogram (csun22, 2025), all information that can be passed into AI models, it is possible to use an ensemble of multiple classifiers, including image classifiers that look at a 2D array of spectrogram data alongside native audio models for audio classification (Bouaisha and Elsheh, 2025). Human speech contains tiny, microvariations in pitch and timing that current AI text to speech models often cannot replicate, resulting in a signal that is statistically predictable and overly perfect (Lancucki, 2021).

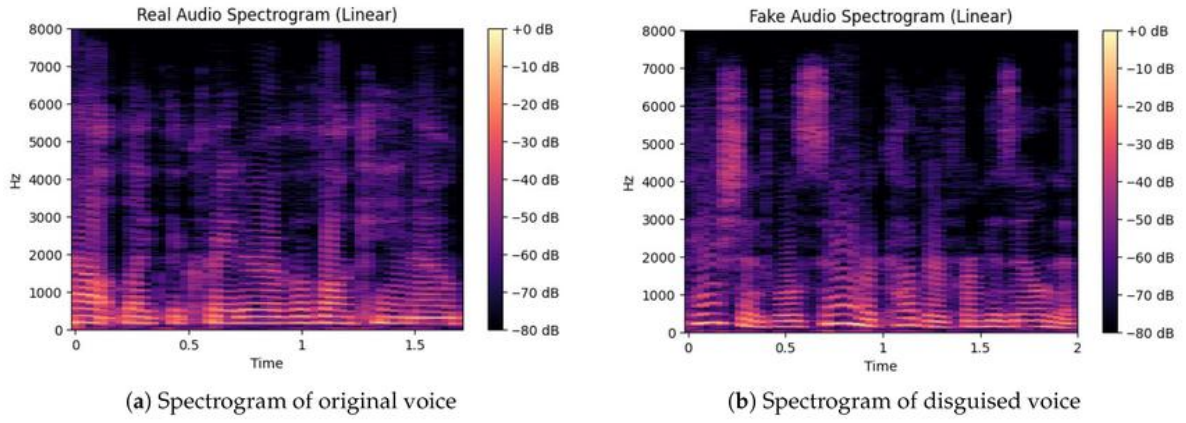


Figure 3 Spectrogram comparison of original and disguised voice samples (Dilbar et al., 2025)

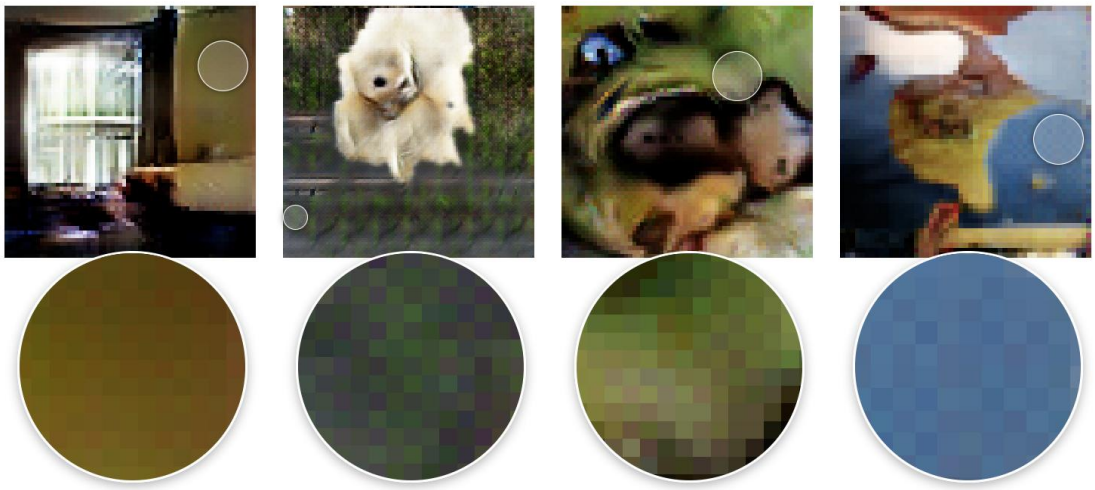


Figure 4 extreme checkerboarding from older image generation models, modern implementations retain this defect to a lesser extent (Odena, Dumoulin and Olah, 2016)

The paper (Mindner, Schlippe and Schaaff, 2023) is an in depth discussion of 47 features that are useful for AI text classification, they determined that the 5 mode decisive features are:

Perplexity: This is the most significant feature, and it measures how unexpected specific words are. Because AI models are essentially glorified autocorrect, AI-generated text is more predictable compared to authentic writing.

Standard Deviation of sentence length: a measure of the variation in sentence lengths within a text segment. Human writers naturally create a mix of their sentence structure with short, punchy sentences and long, complex ones. AI text often to produces sentences of a more uniform length.

Average Sentence Length: The researchers found that average length of sentences written by humans was larger than those written by AI in certain contexts.

Comma Frequency: The distribution of punctuation, particularly commas, was a highly ranked feature. AI models are more likely to adhere to strict grammatical rules compared to patterns found in human-authored text.

Stop Word Frequency: AI often uses a different ratio of stop words (function words) and nouns compared to humans.

The reaserchers in this paper claim to outperform existing text classification methods such as GPTZero by significant margins using only traditional algorithmic steps, GPTZero have since updated their models to include features such as the ones discussed in this paper. This project could also use these features for text classification, potentially developing an extremely small model with only a few numerical inputs from extracted features from a given text section, the classifier model would never see the text itself.

The Research Gap

The literature indicates a significant gap in the market. While high-accuracy models exist in the lab (Mindner et al.) and proprietary watermarks exist in corporate development branches (SynthID), there is a lock of willingness to use these, current societal structures incentivise corporations to invest in AI, instead of against it, there are no equivalent systems for Fire vs Fire on any browser extension store, for seemingly ne reason.

Problem Analysis and System Requirements

Problem Analysis

Synthetic content is increasingly indistinguishable from authentic media. A significant "detection gap" exists, s established in the introduction. While training compute for frontier models doubles approximately every 5.2 months, the number of new models is increasing, and more efficient usage of training compute. The public's ability to identify AI-generated content remains low, research revealed that existing solutions are often reactive, requiring users to possess the skills to first become suspicious of content, before manually initiating a check with one of the existing browser extensions to affirm their suspicions, this approach is a result of the architecture of existing models, while it is a positive first step, it achieves nothing in the way of detecting AI generated content that the user missed, or for those with less digital



literacy who would forget to use the tool, they are the most likely to be deceived by AI generated content or scams. Furthermore, many current tools rely on cloud-based processing, which introduces privacy concerns and more importantly, financial barriers through subscription models which significantly reduces the number of users willing to protect themselves or their loved ones against a threat that they are aware of.

The fundamental problem addressed by this system is the need for a proactive, transparent, and accessible method for content verification. The system must operate automatically to protect users who may not realize they are being exposed to synthetic media, such as older generations or those with limited digital literacy. By moving the classification process to the user's local hardware, the tool aims to eliminate the need for prior knowledge and significantly reduce the barrier for entry by providing the tool for free.

Functional Requirements

Functional requirements define the core actions the software must perform to achieve its goal of real-time, on-device AI detection and filtering.

- Automatic content collection – The browser extension must automatically scan the Document Object Model (DOM) of all websites to collect all content encountered by the user while browsing.
- Content collection filtering – To increase performance and reliability, the system shall only collect classifiable content, for example text blocks exceeding 250 characters, images of sufficient size, encouraging UI elements and short phrases not to waste unnecessary compute cycles on likely unreliable classification.
- Local backend communication – The system must interface with a local backend to send collected content for classification
- Modality-Specific Routing – Content must be split into distinct modalities and routed through the appropriate classification pipeline or pipelines
- Local content classification – Content must be classified using machine learning models running locally, each individual item must return a scale of certainty for AI generation 0% - 100%
- Ensemble Configuration – Advanced users must be able to configure and independently produce additional classifiers and categories allowing multiple models to operate concurrently with custom weights, detection thresholds, and methods.
- Webpage Alteration – Based on the classification response, the extension must modify the webpage's CSS or HTML to provide unmistakeable visual feedback to the user of suspected AI content.
- Customizable Formatting Modes – Users must be able to customize visual feedback for suspected AI content Off (no change), Hover (filter applied until the mouse hovers over content), and Always (filter consistently applied).

- Setup wizard – The local classification system must be installed using a setup wizard for simple, painless and foolproof instillation.

Non-Functional Requirements

Non-functional requirements describe the quality attributes and performance constraints of the system.

- Performance and Throughput – The base classification models must achieve a minimum processing speed of 5 units per second for real-time operation. In practical implementation, the image model should aim for a higher throughput, such as speeds greater than 100 images per second.
- Accuracy – The system shall aim for a minimum classification accuracy of 75% to maintain user trust.
- Privacy by Design – All classification must occur on the user's device, no data is to be transmitted to any external service.
- Hardware Compatibility – The system must be capable of running on varied consumer hardware, utilizing the users GPU or NPU when available, falling back to CPU when none are available.
- Platform Availability - The initial prototype must support the Firefox browser using Manifest V2 API, with a planned design that allows for future porting to Chromium-based browsers such as chrome, edge and others using Manifest V3. The initial prototype must first support windows operating systems, with the option to

---

Design justification

Justification of AI classification against traditional algorithms

The project prioritizes AI classification over traditional rule-based algorithms for several key reasons, while the downsides posed by an AI strategy are the high compute cost and unpredictability of AI models, hallucinations by AI are more prevalent in high complexity models such as LLMs or other generative use cases, AI classifiers output on a scale of certainty, the user can define the threshold for action and is clearly informed of the confidence for each classification. Traditional rule based algorithms for AI classification rely on extracting forensic data from images such as a performing a 2D Fourier transform, using Benford's Law, or looking for traces left by camera sensors when capturing a real image with PRNU (Photo Response Non-Uniformity), These strategies are effective at exacerbating defects in synthetic content, however they still produce nuanced data that traditional algorithms do not possess the adaptability to exploit, while some methods for non AI classification of simpler data such as text exist (Mindner, Schlippe and Schaaff, 2023), the focus among forensic researchers for more complex mediums, has been in extracting forensic mentioned above features from

synthetic media, and feeding that output to AI models for final judgement (Croitoru, Ionescu and Hondru, 2020) (Yadav and Kumar, 2025) (Raza et al., 2026).

Traditional algorithmic options use strategies such as checking specific metadata tags or using inherently rigid edge detection rules. They rely on deep research to discover features that differentiate authentic and synthetic media, however as generative models evolve and research to hide forensic features expands (Sugawara, Shiota and Kiya, 2019), established rules become obsolete faster than they can be discovered. ML models discover patterns and structures implicitly from vast datasets, the benefit is that they are uniquely capable of identifying complex and high dimensional correlations that humans cannot explicitly define (LeCun, Bengio and Hinton, 2015).

Justification of separate backend server and alternative options.

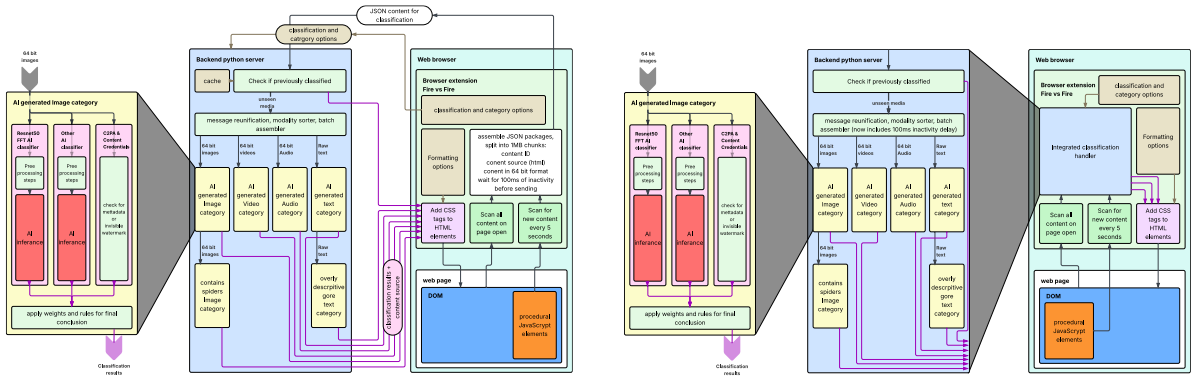


Figure 5 Proposed architecture comparison of local server-based compute (left) and integrated web compute (right)

The primary design advancement of Fire vs Fire is the local processing of content, for this system there are two design routes to considered, whether to conduct the machine learning inference directly within the browser using recently supported TensorFlow.js or alternatively to create a more complex Local Python Backend, with higher performance and potential for features via a bridge between the browser extension and the server. A perfect system would offer the option for both the simple we based inference and the increase performance and features of a local server.

While a pure JavaScript solution integrated entirely into the web browser using TensorFlow.js would simplify the installation process, it was rejected due to limitations that web browsers impose on systems to isolate scripts from the underlying hardware for security (Maczan, 2026). This abstraction layer introduces several hardware bottlenecks that are critical for live computation of intensive tasks such as AI inference. Browsers access the GPU through the wrappers WebGL or WebGPU, in contrast, a local Python environment can interface directly with hardware, while increased capabilities of browser and web applications using local hardware are positive, direct control shows an estimated gain of between 500% and 1000% (Maczan , 2024) (Zeigermann, 2022). The other hardware consideration is memory usage, the largest performance bottleneck by far for this system is loading of AI models into ram, this issue was alleviated using lazy loading, where models stay in memory after usage in preparation for subsequent classifications, browsers perform aggressive memory management to reduce system load (Chrome, 2019), this would likely lead to large models

frequently being cleaned up or failing to load. Browser extensions are aware of these limitations, and provide native messaging options as a solution for interfacing with local services such as our backend server (Mozilla, 2023), and allow extensions that require external components to function on their extension stores, as long as the extension explicitly informs the user that it requires external software to function, the extensions uses the Native Messaging API, and that the extension does not silently install software on the users computer, instead requiring the user to download software manually using an external repository such as GitHub.

The local server solves issues found with browser computation but also provides unique benefits that also factor into the decision for choosing a separate local server. Choosing to use a local server increases development complexity compared to a complete integrated web solution when developing a prototype with for one browser, however after prototyping is complete and the focus moves towards porting Fire vs Fire to other browsers, only having to adapt half the system may save development time in the long run. By separating the computation from the browser, we provide a future option for the computation to serve more than just a single linked browser, a user could have multiple separate browsers using the same server, this is also a viable solution to keep user settings consistent between browsers for Fire vs Fire, when a traditional centralised server would be needed for this feature. While the benefits of the Fire vs Fire local server architecture have been argued for extensively in this paper, many of the benefits are actually not dependent on a local server, the real source of benefit comes from the system not being dependant on a corporate server, paired with the fact that this system aims to democratize methods for combating AI generated content, but will inevitably not be capable of reaching those who have old and underpowered systems or inherently less powerful mobile devices, extending the reach of one backend server beyond the device that houses it are not planned for development yet, however the opportunity proposes many new levels of reach:

- LAN server, one powerful device in a home or office environment serves all devices on the network, opening possibilities for lower power systems and mobile devices

- Password protected WAN server, individuals would be able to create a password for their server at home and use its power while outside the house. Technically minded individuals, who are more likely to care about this issue and more likely to have a device powerful enough to perform as a backend server could share their password with loved ones who would otherwise be unable to run it.

- Crowdsourced computation WAN, give individuals the option to volunteer their powerful devices to compute for strangers who are unable to run models themselves, significant security considerations would be required. Crowdsourced compute is a visionary goal, however success has been found by some when proposed as an moral or ethical issue (Alexander, 2026), during covid, the largest computational network ever assembled at the time was formed of crowdsourced volunteers to help fight the virus, the network was more powerful than the top 500 traditional supercomputers combined (Folding@Home, 2026).

Using a PC based server for computation from mobile devices, even those that are powerful enough to run models locally, will benefit those mobile device users through saving them from unnecessarily wasting the battery of these devices, which may be a concern even when the monetary cost of running Fire vs Fire is negligeable.

Potential alternatives to web browser based classification

During the concept phase of this project, screen capture was considered as an alternative to the eventual web browser implementation. The proposed app would be entirely standalone, independent from any web browser or other “frame”, it would take periodic screenshots as input for classification. The benefit of this would be complete universal availability in all apps and services, however the system would require a separate computer vision model to first identify content before classification, likely increasing performance burden significantly. Classification would not be able to utilize context such as metadata or consistent tracking of previously identified content, and some quality would likely be lost due to screen capture compared to downloaded images. It would require significant compute power to make any interface not feel sluggish or choppy and would likely never perfectly track a flagged image with applied formatting such as a blur or border during scrolling. This concept was rejected in Favor of the web browser method, due to its increased reliability, polish and performance burden.

Firefox was chosen as the initial platform because any solutions possible on an older manifest V2 platform are more likely to still work when porting to newer manifest V3 browsers like chrome and edge.

Development strategy justification

The development of this project was coordinated using the Agile Machine Learning Lifecycle. More linear and restrictive methodologies like waterfall are often ill suited for large, solo, research and development projects where development often reveals new opportunities for unconsidered aspects of the design, an ability to rapidly prototype and peruse potential improvements has lead this project to deprioritise planned features of the design in return for new, unplanned features. After the deadline for this rough prototype project, the subsequent polished product will be built from scratch with knowledge gained during this time, it benefits the most from the current phase only if the freedom of a prototyping is fully taken advantage of and result in a better end product.

at the end of the prototyping phase, the Fire vs Fire project file is extremely messy, the researcher estimates that approximately half of all files in the project folder are no longer attached to the runtime or provide any function. This prototype has performed its

System Architecture

Modularity



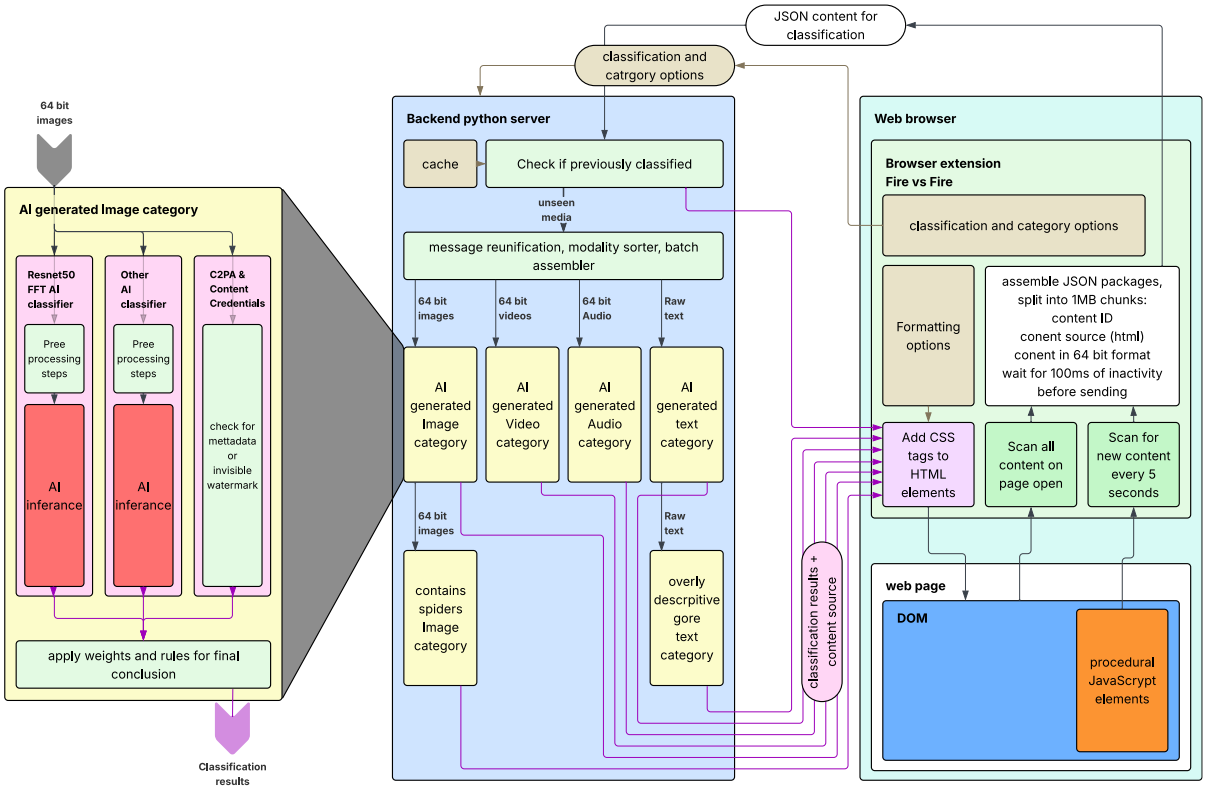


Figure 6 High level system Architecture diagram of Fire vs Fire

This diagram visualises the high level system architecture of a possible Fire vs Fire configuration, the highly modular design allows for immense user customization. Within the GUI interface the user can create, delete and edit new filters for different types of content they wish to flag, these are called Categories, within each category there are one or many classification pipelines called classifiers, each classifier is a self contained python script with common methods that are called by the parent category, the user can use the GUI inside the web browser to change which classifiers each category uses, and also to discover new classifiers that are placed inside the same folder as the main local server.

When a new user installs the extension and server, they will find 4 stock AI content categories (one for each modality) along with accompanying classifiers inside, in the example configuration seen in figure 5 above, the user has added a category to hide images that contain spiders, and also a category to hide text that contains descriptive gore, inside the expanded AI generated image category we can see that the user has decided to improve accuracy by adding an additional AI classifier with unique pre-processing and a algorithmic classifier that checks for metadata and open source watermarking standards, users develop make their own classifiers using the publicly available classifier template. When two or more categories of the same modality flag the same content inside the web browser, the formatting effects are combined automatically.

Data flow

Data flow begins when a new webpage is loaded, the browser extension scans the DOM (Document Object Model) for all media simultaneously a periodic scan for the current window

only begins scanning for elements added through Java script, these elements are common on infinitely scrolling websites and for deployments where an algorithm is used to decide what content is show to a user such as on social media platforms. All discovered content is collected and formatted into a JSON file containing content ID, its modality, the source URL, and for the final section, images, video and audio are encoded into text using the 64 bit library, text is inserted as plain text. 64 bit encoding is used because it prevents the need for the backend server to download content from the web a second time, which is a slower solution and would double data usage in metered scenarios, the source URL is used for identification of previously classified content and also as a fallback if the 64 bit solutions fails, it should be noted that this fallback is a potential security issue for code injection into the python server, for example the code <img src="http://192.168.1.1/admin"> in a website that a user visits would return the configuration dashboard of the user's router, as classifiers gain access to this information, a custom classifier that is badly or maliciously configured could be vulnerable to similar attacks.

To avoid constant spinning up and down of classifiers, the browser sends data to the local server in bulk, achieved by waiting until no new content has been found for 100ms before sending. As mandated by browser providers, data is sent between the local server and the browser using native messaging APIs that are unique to each browser. When the local server receives a batch from the web browser, it first checks against a cache of recently classified URLs and text, if this content has been recently classified then the previous classification result is returned immediately, if not then content is separated into modalities and fed through respective categories, each category is run in parallel, including multiple categories of the same modality, inside each category, all classifiers run in parallel.

Stock classifiers all perform preprocessing on content before sequentially classifying each item from the batch, further parallelisation was found to decrease performance, batch size can be partially controlled by increasing or decreasing the inactivity window observed by the browser extension before it sends data, 100ms was found to be a sweet spot although this is likely hardware and network dependant, this delay is configurable in the browser extension's options GUI. Once all classifiers in a category finish classifying, the category applies its weights and rules to combine classification results from all classifiers, the user can configure different classifiers to be considered with higher weight than others, or introduce complex rules like "if A > 0.5: x = 1, else x = B * 0.3 + C * 0.7". Categories do not wait to send classification results back to the browser extension, even for categories still processing content from the same batch, this increases responsiveness by preventing simple classifications such as text from being bottlenecked by unrelated computing.

The browser extension maintains a list of CSS styling tags for each category, these are editable in the options GUI, when the browser extension receives the response containing classification results, it matches the URL source of a content classification with the element in the page and adds the CSS tag to content over the threshold for flagging, at this point, if an element already has a Frie vs Fire assigned tag, a new combined tag is created on demand using the average of common attributes. The user can change attributes of tags and the page

updates live, however significant changes such as editing the threshold for content flagging requires re-classification.

Implementation

Browser extension – content handling

The first step of the browser extension is content detection, we use a multi layered detection system to identify and extract content from webpages in real time, handling both static DOM content and dynamically loaded JavaScript.

Initial page scan

Upon page load, the content script content.js performs an initial extraction of all classifiable content. The system is currently limited to modalities however others are planned. To detect simple Dom images, the extension queries all <img> elements using document.querySelectorAll('img') and applies filtering logic to eliminate false positives. Images are validated against multiple criteria including:

- Images below 64×64 pixels are rejected to exclude UI icons and tracking pixels, this is configureable via minImageDimension
- Only images with naturalWidth > 0 and naturalHeight > 0 are processed, ensuring the browser has fully decoded the image data
- Furthermore, Images that encode to less than 45KB in base64 format are temporarily skipped, this is because images outside the viewport of websites are lazy loaded at extremely low quality that is not viable for classification, this also increases performance on websites that employ this strategy because only images the user can actually see are classified. These elements are removed from processedImages to allow re-processing when the full-resolution image loads
- Optionally, the user can enable alt text filtering, when enabled, images without alt text are skipped to increase performance and avoid flagging GUI elements/logos, a whitelist for exempt URLs is used for this setting for platforms that don't have alt tests but still require classification, for example YouTube thumbnails (i.ytimg.com/vi/) and Google images (encrypted-tbn*.gstatic.com)



Long-form text is extracted from common text HTML elements including <p>, <li>, <pre>, <blockquote>, <td>, <th>, and <h1> to <h6> The extraction process includes:

- retrieving innerText or textContent from each listed element type on the page
- Normalizing whitespace by replacing all sequences of whitespace characters such as spaces, tabs, newlines, etc, with a single space using .replace(/\s+/g, ' ').trim()
- Filters sections below the minimum character threshold with a default of 150 characters, controllable in the GUI
- Deduplicates identical text blocks to prevent redundant classification
- Avoids processing parent nodes by excluding container elements that may contain other text elements

Dynamic Content Detection

Many websites such as social media platforms use infinite scrolling, lazy loading, and JavaScript-driven content injection. The extension implements three observers to handle these patterns:

The mutation observer is the main system for detecting new content, other observers are necessary only for edge cases, the mutation observer monitors the DOM for structural changes for items changing size or new items being added, this is a viable solution for instant detection when algorithmic content is served to the user and should work for all common frameworks including React, Vue, Angular and vanilla JavaScript. When new nodes are added, the observer:

- Processes new <img> tags
- Processes new text sections
- Automatically re-processes images when their src attribute changes, this finds previously skipped images that were lazy loaded

Similarly the resize observer, captures a separate lazy loaded images style used by some websites that load placeholder images as 1×1 transparent pixels when they are off the screen or not loaded yes, and then they are later replaced with actual content. The resize observer detects these dimension changes and triggers classification.

Intersection Observer is a legacy system that is mostly obsolete, it detects when images enter the viewport with a 200px margin, configurable in the code with rootMargin. This observer's functionality has been superseded by tracking attribute changes in the mutation observer, however it serves as a backup for edge cases.



Finaly, a periodic Scan is used every 3 seconds and can be configured in the code with setInterval. This catches edge cases missed by previous observers, such as images loaded via non-standard JavaScript frameworks. This scan also performs garbage collection, removing disconnected elements from tracking sets.

Data Encoding

To minimize data usage, reduce overhead and avoid re-downloading images, the extension attempts to extract image's data directly from the DOM using HTML5 Canvas, this solution prevents having to use any additional data compared to normal web browsing when image data is already on the user's device in the browser's allocated RAM, the simpler alternative would be to re download images based on their URL in the local server, which would double data usage, double RAM usage, which is needed for AI classification and increase delay before finished classification and flagging. Special care must be taken when handling and encoding image data for this system since the AI classification models are designed to identify fragile artifacts that are created by generative AI processes, for this process images are saved as lossless PNGs, encoding to JPEGs can obscure AI artifacts and create new compression artifacts, even when the original image from the browser's memory was already a JPEG.

Javascript

```
function imageToBase64(imgElement) {
  const canvas = document.createElement('canvas');
  canvas.width = imgElement.naturalWidth;
  canvas.height = imgElement.naturalHeight;
  const ctx = canvas.getContext('2d');
  ctx.drawImage(imgElement, 0, 0);
  return canvas.toDataURL('image/png');
}
```

If canvas extraction fails due to security issues such as CORS (Cross Origin Resource Sharing) restrictions, the extension falls back to sending the image URL, allowing the backend to download the image, however this poses security threats as discussed in system architecture > data flow

Text content is transmitted as UTF-8 strings with classification does not require extensive information or metadata, classifiers extract their own features from raw text.

Batching

The extension communicates with the local server using Firefox's native messaging API, this is a persistent bidirectional channel that requires registry changes in windows, these are handled by the install script. Classification requests are batched to reduce overhead and because over paralysation for the AI models exponentially increases inference time beyond a point. The buildClassificationBatches() function implements intelligent splitting:

- Enforces a 4GB maximum size limit per batch to stay within native messaging constraints, anticipates if by adding an image to this batch it will exceed 4GB

- Assigns a persistent identifier to each item (img-0, img-1, text-0) that survive page mutations.

- Maintains a Map of itemId vs DOMElement to correlate classification results with their corresponding page elements

Batches exceeding the size limit are split into multiple chunks, sent in parallel via Promise.all(), and reassembled by the local server using jobId and chunkIndex metadata.

Native Messaging

The background script background.js manages a persistent port connection to the native host:

Javascript

```
const HOST_NAME = 'com.aidetector.classifier';
nativePort = ext.runtime.connectNative(HOST_NAME);
```

This connection is registered via a JSON manifest file register-firefox-host.reg that maps the host name to the Python executable path. The manifest specifies that the only extension allowed to communicate is one using the unique Fire vs Fire extension ID. It points to the python script wrapper run_host.bat. it uses stdio for JSON message exchange

Message Protocol



The extension implements a versioned JSON protocol for classification requests, the request schemea was switched to a V2 at the same time as the local server (native host) was also re written to its own V2, V2 supports batched requests and other features that were not part of the V1 standard, V2 label is necessary because the local server is backwards compatible with old V1 requests:

JSON

Classification request schema example

```
{
  "version": 2,
  "type": "classifyJobChunk",
  "jobId": "job-1234567890-abc123",
  "chunkIndex": 0,
  "totalChunks": 3,
  "timestamp": 1234567890000,
  "payload": {
    "items": [
      {
        "id": "img-0",
        "modality": "image",
        "url": "...",
        "width": 1920,
        "height": 1080
      },
      {
        "id": "text-0",
        "modality": "text",
        "text": "Lorem ipsum...",
        "length": 500
      }
    ],
    "model": {
      "ensemblesByModality": {
        "image": [{"id": "ai_images", "classifiers":
["res_net50_fft"], "weights": null}],
        "text": [{"id": "ai_text", "classifiers": ["text"],
"weights": null}]
      },
      "lazyLoad": false,
      "streamResults": true
    }
  }
}
```

JSON

Classification results schema example

```
{
  "type": "classifyResultChunk",
  "jobId": "job-1234567890-abc123",
  "results": [
    {
      "id": "img-0",
      "modality": "image",
      "ensembleId": "ai_images",
      "score": 0.87,
      "label": "ai",
      "displayLabel": "AI generated"
    }
  ]
}
```

Streaming Results and Progressive Rendering

To achieve the goal of providing the fastest feedback possible to the user, the local server returns classification results as they complete rather than waiting for the entire batch. The background script forwards these chunks to the content script via ext.tabs.sendMessage(), allowing the UI to update incrementally.

The content script creates and maintains a Map for itemId vs ResultData to accumulate results from multiple ensembles. When a new chunk arrives, applyResultChunk() updates the map and immediately applies visual effects to the corresponding DOM elements.

Error Handling

The native messaging port can disconnect because of a veriaty of reasons including: backend crashes, extension reloads, or system resource constraints. The background script handles disconnection and reconnects gracefully:



Javascript

```
nativePort.onDisconnect.addListener(() => {
  nativePort = null;
  // Reject all pending requests
  for (const [reqId, { reject, timeoutId }] of
pendingNativeRequests) {
    clearTimeout(timeoutId);
    reject(new Error('Native host disconnected'));
  }
  pendingJobs.clear();
  activeClassifyCount = 0;
});
```

Subsequent classification requests automatically trigger reconnection using ensureNativePort(), which calls ext.runtime.connectNative() if there is no active port.

Multi classifier category management

While the server performs the computationally expensive inference, the browser stores all results and handles local management of classifier scores, the browser extension manages category configurations and weighted calculations providing a flexible system for creating custom detection categories combining the strengths elements of multiple models and allowing for greater classification accuracy.

Browser extension – UI

The options page (options.html and options.js) provides a comprehensive interface for configuring detection behaviour, visual styling, and performance parameters, its features include:

Activity Indicator



The background script tracks outstanding classification jobs using activeClassifyCount. While there are active jobs, the extension icon and logo within the options GUI changes from the default red (showing idle) to green (showing classification in progress) using ext.browserAction.setIcon(), and a badge displays the number of concurrent jobs. This provides quick and easy feedback on classification activity for the user weather or not they have the popup open.

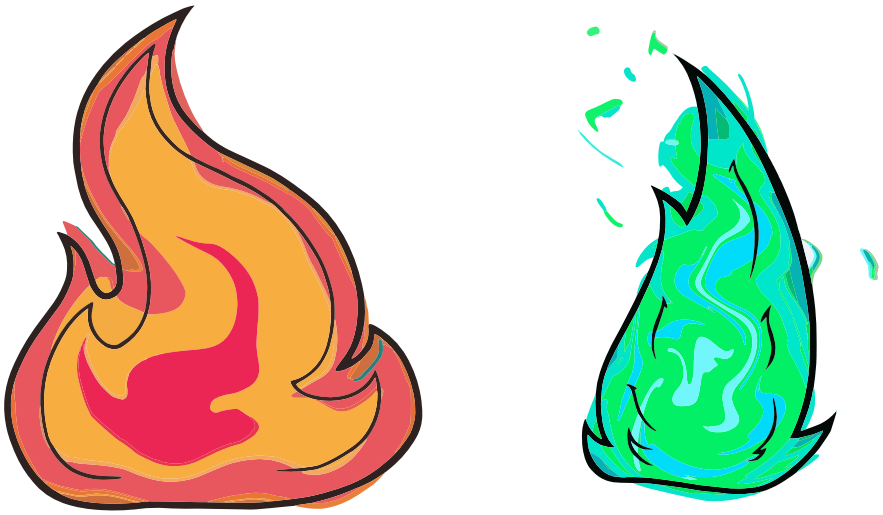


Figure 7 Fire vs Fire logos, red and green

Category Configuration (Ensembles)

The extension uses a category system that allows users to create, edit and remove multiple independent detection types and control their classification methods. The name “Category” is a recent development, categories were formerly referred to as ensembles, the codebase still uses this vocabulary. Each category has various elements that can be controlled in the GUI:

- Modality, Image or text
- Toggle for turning on/off without deleting configuration
- The confidence score threshold required to flag content
- Classifier selection, select one or more classifier models from the list
- Visual style options for flagged content

Additionally, multi classifier categories have:

- Category name, where the category name was previously the name of a sole classifier in the category, with multiple classifiers the user can edit the name of the category, this name is also used as the label for flagged content
- Classifier weights, the user can use the GUI to set the weight of each classifier independently, to emphasize more accurate classifiers, weights automatically balance in the UI to visually indicate to the user the influence of each classifier

Visual Style Configuration

Visual effects for categories have three modes, off, hover, and always:

- Off, this effect is not applied
- Hover, this effects is applied by default and removed on hover allowing the user to clearly see the content
- Always, this effects permanently applied, unless the category tag is removed by the user in the right click context menu.

The mode is stored in element.dataset.aiImageStyle as JSON, allowing the hover handler to restore the correct state when the cursor leaves. The hover handler uses the global pointermove listener and elementsFromPoint() to detect hover through overlapping elements such as transparent overlays or text layers that block standard the generic :hover CSS functionality.

Javascript

```
document.addEventListener('pointermove', (evt) => {
  const hits = document.elementsFromPoint(evt.clientX, evt.clientY);
  const candidate = hits.find(el => el.classList.contains('ai-
detected'));

  if (candidate !== currentHoverEl) {
    if (currentHoverEl) hideAi(currentHoverEl);  // Restore blur
    currentHoverEl = candidate;
    if (currentHoverEl) showAi(currentHoverEl);  // Remove blur
  }
}, true);
```

Images have 3 flagging options available, In the event of multiple categories flagging the same image, these settings are merged:

- A gaussian blur with a customizable radius of 0-20px, applied via the CSS filter: blur() property
- A coloured outline that scales automatically to a thickness of 2% of image's smaller dimension clamped to 2-10px, with a configurable multiplier of 0-5×, the border is achieved using a CSS outline property with a negative outline offset to make the outline occupy space inside the image instead of expanding beyond it's borders and displacing the other elements due to the additional image size.
- A confidence percentage badge positioned in top-right corner with the category label, stacking vertically for multiple categories flagging the same image

Text segments have 4 flagging options available:

- A gaussian blur with a customizable radius 0-10px using the blur() property
- The underline text decoration with configurable colour
- The strikethrough effect, decoration with configurable colour, underline and strikethrough cannot be separate colours, the underline overrides strikethrough in the event of a conflict
- Highlight using a background colour overlay of the whole element

Colores are configured using 3 sliders (RGB, 0-255 per channel) and preview, furthermore each slier shows a gradient of what colour would be produced if that slider were moved to that position, this solution is necessary because the Firefox colour picker is not compatible with popups, colours are converted to hex strings eg #FF0064 for storage and use in Firefox's elements.

Performance and Debugging Settings

At the bottom of the options GUI there are developer and performance options including:

- The Classification delay, a debounce timer (0-500ms) to batch fast sequential DOM mutations
- Image capture quality slider, control the compression of images sent to the local server, (20%-100%) at 100% images are encoded as a lossless PNG, otherwise they are a JPEG of that set quality.
- Toggle to lazy load models, lazy loaded AI models are unloaded from VRAM after each batch, reduces memory usage but significantly increases latency



- Verbose logs, Enable more detailed console output for debugging, new logs include batch splitting, GPU info, timing metrics, etc.
- A global reclassify button

Storage and Synchronization

Settings are persisted to browser.storage.sync or browser.storage.local as fallback, allowing configuration to sync across devices despite no centralised server infrastructure for Fire vs Fire. Every input to the options page triggers storage.set().

The content script listens for storage.onChanged events and implements applicable reload logic from 3 severities to avoid reclassifying or reformatting unnecessarily:

- Classification settings (classifier selection, enabled state): Trigger full process rescan
- Visual settings (blur, colours, thresholds, category weights): Reapply styles to existing results without re-classification
- Performance settings (debounce, logging): Update memory variables only

Classifier Discovery

The options page requests available classifiers with the LIST_MODELS message, to which the local server responds to with:

JSON – Available models response

```
{
  "classifiers": [
    {"id": "res_net50_fft", "name": "ResNet50 FFT", "modalities":
["image"]},
    {"id": "text", "name": "Text Classifier", "modalities":
["text"]},
    {"id": "convnext_large_artifact", "name": "ConvNeXt Artifact",
"modalities": ["image"]}
  ]
}
```

The GUI adapts to the fit each classifier, all classifiers are visible in all category types, but incompatible classifiers with the selected modality cannot be selected

Visual Flagging System

The content script applies CSS visual effects to flagged content. Before applying styles, the script checks if element.isConnected to skip elements removed from the DOM

Image Flagging

Flagged images receive the .ai-detected class and multiple CSS properties, the .ai-detected class is universal and applied to even content of different categories:

Blurred images also receive an animation to subtly oscillate brightness to draw attention without being distracting with ai-pulse:

Javascript

```
element.style.setProperty('filter', `blur(${blurPx}px) saturate(1)
brightness(0.85)`, 'important');
element.style.setProperty('animation', 'ai-pulse 2s ease-in-out
infinite', 'important');
```

Adaptive border:

Javascript

```
const borderWidth = Math.max(2, Math.min(10, Math.round(minSide *
0.02))) * multiplier;
element.style.setProperty('outline', `${borderWidth}px solid
${borderColor}`, 'important');
element.style.setProperty('outline-offset', `-${borderWidth}px`,
'important');
```

A dynamically-created element is injected into the nearest positioned ancestor (figure, link, div) for a Confidence Badge, Existing badges are removed before creating new ones to prevent duplicate overlays when categories are reconfigured:

Javascript



```
badge.textContent = `AI generated 87%`;
badge.style.background = `rgba(255, 0, 100, 0.9)`;
badge.style.position = 'absolute';
badge.style.top = '6px';
badge.style.right = '6px';
```

Badge labels are determined by:

Single classifier category: Uses the label returned by the classifier

Multi classifier category (a true ensemble): Uses the category name, customizable in the GUI or defaults to Category N if name is blank

Text Flagging

Flagged text receives the .ai-detected-text class with configurable decorations similar to flagged images:

Text elements support multiple simultaneous effects (blur, strikethrough, highlight), allowing users to create highly visible warnings.

Javascript – apply all styling

```
element.style.setProperty('text-decoration', 'line-through
underline', 'important');
element.style.setProperty('text-decoration-color', '#ff0064',
'important');
element.style.setProperty('background-color', '#fff3a1',
'important');
```

Context Menu Toggle System

A context menu system for toggling classification tags is included in this extension. The user can use this system to manually classify content, or remove classification tags from already classified content, this allows the user to easily correct bad classifications or hide unwanted content. When the user right-clicks on an image or text element, the content script captures the target element and sends metadata to the background script so the prompt in the context menu accurately describes the action the user can take:

Javascript



```
document.addEventListener('contextmenu', (evt) => {
  lastContextMenuTarget = evt.target;

  if (evt.target.tagName.toLowerCase() === 'img') {
    const hasVisibleAiStyling = evt.target.classList.contains('ai-
detected');
    let label = 'tag';

    if (hasVisibleAiStyling) {
      // Extract label from badge (e.g., "AI generated 87%" -> "AI
generated")
      const badgeIds = getImageBadgeIds(evt.target);
      const badge = document.getElementById(badgeIds[0]);
      const match = badge.textContent.match(/^(.+?)\s+\d+%$/);
      label = match ? match[1] : badge.textContent.split(' ')[0];
    }

    ext.runtime.sendMessage({
      type: 'UPDATE_CONTEXT_MENU',
      hasAiTags: hasVisibleAiStyling,
      label: label,
      modality: 'image'
    });
  }
}, true);
```

When the user selects the context menu item, the background script sends a TOGGLE_BLUR message back to the content script with the target element's src URL (for images) or selection text. The content script then interacts with the itemResults map to remove all classification results from the content, then, if the content currently meets the threshold for any category, effects are immediately removed, if the content is not currently flagged by any classifiers, the content is given the manual blur category tag, a static category with grey colours and a strong blur.

Browser extension challenges and solutions

CORS and Canvas Tainting

Cross-origin images cannot be drawn to canvas without CORS headers, causing toDataURL() to throw a security exception. The extension handles this gracefully by catching the exception and falling back to URL-based fetching in the backend.



Infinite Scroll and Memory Management

Websites like Twitter and Reddit continuously inject new content while removing off-screen elements. The extension implements garbage collection in the periodic scan, removing disconnected elements from processedImages and processedTextElements sets to prevent memory leaks.

Message Size Limits

Native messaging enforces a 4gB message size limit. The extension implements automatic batch splitting to handle high throughput edge cases such as full system reclassify being triggered with many tabs open or particularaly intensive websites, sending chunks in parallel and reassembling them in the backend using jobId and chunkIndex metadata.

Alt text filtering

This extension uses alt text filtering to avoid unnecessary classification of UI elements, reducing performance and impacting the users ability to navigate, a whitelist exception is maintained for this filter for edge cases such as YouTube thumbnails from alt-text filtering, this solution increases performance and the user can alter the list themselves in the file system with GUI planned, but this solution is not elegant or adaptable, which are fundamental values of Fire vs Fire's design, this is the only case of such an inelegant solution in the project.

Local server messaging

Native messaging protocol

The local classification server: echo_host_V2.py, is referred to in the codebase my various names that evolved as scope and capabilities expanded. Names used to refer to the local server are: echo host, native host, native server, local server. The local server communicates with the browser extension via Firefox's Native Messaging API, which uses a binary protocol over stdin/stdout. The native messaging enforces a strict binary protocol with these requirements:

- A 4-byte unsigned integer length prefix
- Is UTF-8 encoded JSON message

Python

```
def read_message():
    raw_length = sys.stdin.buffer.read(4)
```



```
    message_length = struct.unpack('<I', raw_length)[0]
    message = sys.stdin.buffer.read(message_length).decode('utf-8')
    return json.loads(message)

def send_message(message):
    encoded = json.dumps(message, separators=(',',
':')).encode('utf-8')
    sys.stdout.buffer.write(struct.pack('<I', len(encoded)))
    sys.stdout.buffer.write(encoded)
    sys.stdout.buffer.flush()
```

The protocol imposes allows for 4GB messages from the browser extension, however it imposes a 1MB hard limit for incoming messages to Firefox.

Message types

The server handles four message types:

A simple ping check for connection testing

JSON

```
{"type": "ping"}
→ {"ok": true, "time": 1234567890000}
```

classifyJobChunk, the chunked classification request (primary workflow)

JSON

```
{
  "type": "classifyJobChunk",
  "jobId": "job-1234567890-abc123",
  "chunkIndex": 0,
  "totalChunks": 3,
  "payload": {
    "items": [...],
    "model": {...}
  }
}
```

listModels, to query available classifiers for options GUI

JSON



```
{"type": "listModels"}
→ {"ok": true, "classifiers": [...], "model_files": [...]}
```

And Classify, the native host still supports the legacy single request classification system (deprecated)

Logging

Since stdout is reserved for native messaging, all debug output is written to stderr or a log file native_host_v2.log. The log() function appends timestamped messages to the file without blocking the main thread:

Python

```
LOG_FILE = Path(__file__).parent.parent / "native_host_v2.log"
def log(msg):
    with open(LOG_FILE, "a") as f:
        f.write(f"[{time.strftime('%H:%M:%S')}] {msg}\n")
```

An example console log is:

```
[Native Host V2] Processing 47 image(s) with 1 model(s) | DirectML: AMD Radeon RX 7900 XTX
```

Job system

The JobAssembler class reconstructs multi-chunk classification requests sent by the extension. Each job is tracked by a unique jobId. The assembler maintains a dictionary of in progress jobs:

Python

```
self.jobs[job_id] = {
    'created_at': time.time(),
    'total_chunks': total_chunks,
    'received': {},  # chunk_index -> items
    'bytes': 0,
```



```
    'model': None

}
```

Chunks can arrive out-of-order due to parallel Promise.all() execution in the extension. The assembler reconstructs the original order by sorting chunks by index to ensure that items maintain their original itemId mappings for result correlation.

When a chunk arrives, the assembler follows this logic:

- Validates job existence and creates new job entry if first chunk
- Ignores chunks with already-received indices
- Rejects jobs exceeding memory limits
- Accumulates chunk items in received dictionary keyed by index
- When len(received) >= total_chunks, it assembles the final job

Garbage collection

Jobs are removed after 2 minutes to prevent memory leaks from abandoned or otherwise erroneous requests, cleanup runs before each new chunk is added, ensuring stale jobs don't accumulate:

Python

```
def _cleanup(self):
    now = time.time()
    to_delete = [job_id for job_id, job in self.jobs.items()
                 if now - job['created_at'] > self.ttl_seconds]
    for job_id in to_delete:
        self.jobs.pop(job_id, None)
```

Server side classifier management

Discovery

The ModelRegistry class provides dynamic classifier discovery, lazy loading, and lifecycle management. On initialization, the registry scans the classifiers/ directory for Python files, and imports any classes inheriting from BaseClassifier, this streamlines the classifier instillation



process for user adding their own, making it as simple as dropping the new classifier into the folder:

Python

```
def discover_classifiers(self):
    classifiers_dir = Path(__file__).parent / 'classifiers'
    for py_file in classifiers_dir.glob('*.py'):
        if py_file.name.startswith('_'):
            continue

        spec = importlib.util.spec_from_file_location(
            f"classifiers.{module_name}", py_file
        )
        module = importlib.util.module_from_spec(spec)
        sys.modules[spec.name] = module
        spec.loader.exec_module(module)

        for name, obj in inspect.getmembers(module,
inspect.isclass):
            if issubclass(obj, BaseClassifier) and obj is not
BaseClassifier:
                classifier_id = self._generate_id(name)
                self._classifiers[classifier_id] = obj
```

ID & name generation

The registry uses a two-tier naming system:

Unique Classifier ID used internally, are automatically generated from class names using CamelCase to snake case conversion:

- ResNet50FFTClassifier > resnet50_fft
- ConvNeXtLargeArtifactClassifier > convnext_large_artifact
- TextClassifier > text

User facing display labels, are defined by each classifier's get_model_name() method, allowing desired detail and format, The list_available_details() method queries each classifier to retreve its display name:



Python

```
for classifier_id, classifier_class in self._classifiers.items():
    instance = classifier_class(model_path=None)
    name = instance.get_model_name()
    details.append({'id': classifier_id, 'name': name, ...})
```

Python - from text_classifier.py

```
def get_model_name(self) -> str:
    return "TF-IDF model A"
```

Lazy Loading and Caching

Classifiers are instantiated on-demand and cached to avoid repeated initialization overhead, this solution includes the model path, allowing multiple instances of the same classifier with different weights, or for easy hot swapping of models during development.

Python

```
def get_classifier(self, classifier_id: str, model_path:
Optional[Path] = None,
                   lazy_load: bool = True) ->
Optional[BaseClassifier]:
    cache_key = f"{classifier_id}:{model_path}"
    if lazy_load and cache_key in self._loaded_instances:
        return self._loaded_instances[cache_key]

    classifier_class = self._classifiers[classifier_id]
    instance = classifier_class(model_path=model_path)

    if lazy_load:
        self._loaded_instances[cache_key] = instance

    return instance
```

Modality Metadata



The registry discovers classifier capabilities via the list_available_details() method, which queries each classifier's supported modalities, this metadata is sent to the extension's options GUI, which uses it to prevent incompatible classifiers:

Python

```
def list_available_details(self) -> List[Dict[str, Any]]:
    details = []
    for classifier_id, classifier_class in
self._classifiers.items():
        instance = classifier_class(model_path=None)
        modalities =
sorted(list(instance.get_supported_modalities()))
        name = instance.get_model_name()
        details.append({
            'id': classifier_id,
            'name': name,
            'modalities': modalities
        })
    return details
```

Base classifier

All classifiers must use the BaseClassifier class base_classifier.py, which defines a standardized interface for classifiers, this template is a critical feature of Fire vs Fire, it not only allows for, but encourages an unparalleled level of control by users, this feature is why Fire vs Fire can not be limited by its first priority as an AI detector, and is instead a mouldable everything detector.

Required Methods

All classifiers must implement these methods, comments inside the classifier document requirements and instructions no how to create a classifier using this template:

- get_supported_modalities(), Returns a set of supported input types (image, text, audio, video) multiple modalities in one classifier is supported, this may be useful if a classifier makes use of a method for classification that makes use of greater context, for example a classifier that is more likely to label videos as AI if it also believes that the text on this page is AI generated, discusses AI topics, or contains giveaways like



text from known AI content creator's YouTube video descriptions, which are often copied between all videos for a given channel.
- load_model(), Loads model weights from disk and initializes inference engine. Returns success: bool, error: Optional[str]
- preprocess_batch(), Converts raw inputs (PIL Images, text strings) into model-ready tensors. Returns batch_tensor, valid_indices where valid_indices tracks which inputs pre-processed successfully
- classify_batch(), Runs inference on pre-processed batch. Returns List[float] with scores from 0 to 1 where 1 is the content intended for flagging, 0.5 is uncertain, 0 is not intended for flagging, and -1 for errors
- get_device_info(), Returns hardware information {device: cuda, name: RTX 5090, backend: CUDA}
- get_model_name(), Returns human-readable model name for options GUI selection.

Optional Methods

- get_label(), Returns the text for the badge label, Defaults to AI generated if no label is provided
- unload_model(), unloads the model from memory.
- process_batch(), This is a optional method that handles model loading, pre-processing, classification, and error propagation

Utility methods

- _log_to_stderr(), used for logging
- _try_cuda_then_directml(): Automatic GPU detection with fallback chain CUDA (Nvidia GPUs) > DirectML (AMD/Intell GPUs) > CPU. This has not been tested on a Nvidia GPU
- _ensure_imports_safely(): Lazy import helper for avoiding startup delays.

Classification Orchestration

The classify_items() function orchestrates the entire classification pipeline, handling modality routing, batch processing, and result streaming.

Modality separation



Items are first separated into their modalities:

Python

```
image_items = [it for it in items if it.get('modality') == 'image'
and it.get('url')]
text_items = [it for it in items if it.get('modality') == 'text' and
it.get('text')]
other_items = [it for it in items if it not in image_items and it
not in text_items]
```

Per category classification

The local server is not provided with category configurations, it simply runs classifiers as instructed, instead, all classification results are sent to the browser as they complete, category weight management is handled inside the browser so changing category weights does not require re classification, this is contrary to the oversight seen in the design architecture plan. Much server side weight and category management code exists in the codebase however it was never implemented, upon returning to the issue this alternative solution was followed.

Batch processing

Large jobs are split into mini batches to prevent RAM/VRAM exhaustion, ram management is critical for Fire vs Fire as consumer hardware may struggle to maintain the necessary RAM for multiple simultaneous AI classifiers being loaded, particularly on windows, web browsers are already prone to using large amounts of RAM.

Python

```
for batch_start in range(0, total_images, mini_batch_size):
    batch_end = min(batch_start + mini_batch_size, total_images)
    batch_slice = image_results[batch_start:batch_end]
    batch_ids = [item_id for item_id, _, _, _ in batch_slice]
    batch_images = [img for _, img, _, _ in batch_slice]
    batch_errors = [err for _, _, err, _ in batch_slice]

    t0 = time.time()
    batch_results, device_info =
ensemble.classify_batch(batch_images, modality='image')
    classify_ms = int((time.time() - t0) * 10)
```

Result Streaming

When streamResults is enabled (this is the default behaviour), the server sends results as they are completed using classifyResultChunk messages, alternatively streamResults can be disabled and revert to the legacy behaviour of result accumulation and then sent all at once when classify_items() finishes.

Python

```
def emit_chunk(ensemble_id: str, modality: str, chunk_results:
List[Dict], chunk_errors: List[Dict]):
    if send_chunk and stream_results:
        send_chunk({
            'type': 'classifyResultChunk',
            'jobId': job_id,
            'ensembleId': ensemble_id,
            'modality': modality,
            'results': chunk_results,
            'errors': chunk_errors
        })
```

Data Fetching and Preprocessing

The classification pipeline begins with fetching and decoding content from the extension, currently three methods are supported.

For images we use the fetch_images_from_urls() function handles two image sources,, the function returns 4-tuples: (item_id, PIL.Image or None, error_reason or None, fetch_duration_ms), allowing the system to track fetch failures and timing.

Base64 images

this is the preferred method, Images extracted from the DOM via canvas are sent as ... strings. The server decodes them directly:

Python

```
if url.startswith('data:'):
    header, data_part = url.split(',', 1)
    if ';base64' in header:
        raw = base64.b64decode(data_part)
    else:
        raw = urllib.parse.unquote_to_bytes(data_part)
    img = Image.open(BytesIO(raw)).convert('RGB')
```

URL images

This is the legacy method before it was optimised, when this method is used, the server fetches the image from the web, it is one of the most necessary fallbacks in the whole system.

Python

```
elif url.startswith(('http://', 'https://')):
    req = Request(url, headers={
        'User-Agent': 'Mozilla/5.0 (native-host)',
        'Accept': 'image/*,*/*;q=0.8'
    })
    with urlopen(req, timeout=15) as resp:
        data = resp.read()
    img = Image.open(BytesIO(data)).convert('RGB')
```

Text handling

Text is already extracted by the content script, fetch_text_from_items() simply unpacks it from the JSON payload:

Python

```
def fetch_text_from_items(text_items: List[Dict[str, Any]]) ->
List[tuple]:
    results = []
    for item in text_items:
        item_id = item.get('id')
        text = item.get('text', '')
        results.append((item_id, text, None, 0))
    return results
```



AI Image ConvNeXt-Large artifact Classifier

Evolution from Previous Architectures

The ConvNeXt-Large artifact classifier represents the third distinct iteration to of candidates to become the primary AI image detection model for Fire vs Fire, incorporating lessons learned from previous ResNet-18 and ResNet-50 architectures. This model addresses the fundamental challenge of labelling AI generated from authentic web images by analysing both semantic content and generative artifacts.

The development process followed an iterative refinement strategy of rapid prototyping across three major generations, defined by major redesigns including backbone and head changes:

ResNet-18 (V1). The initial prototype served as a useful tool for, successfully identifying framing patterns and general shapes characteristic of early generative models (Midjourney V4, DALL-E 2). However, it struggled with diverse content types and achieved only moderate accuracy on real-world web images.

Modified ResNet-50 FFT (V2). The second iteration explored methods for more reliable AI classification, the more powerful ResNet50 backbone was used, and it was modified to support a fourth channel containing fourier transform magnitude data in addition to standard RGB channels. This modification provides the model with a specific insight into high-frequency artifacts such as the grid pattern that is common in GAN generated images. The architecture modification validated the hypothesis that additional frequency-domain information provides discriminative signal beyond spatial RGB data.

ConvNeXt-Large artifact – 17 channel (V3): The final architecture expands upon the success of V2 with 17 total input channels, the default 3 RGB and now 14 aditional artifact channels, designed to extract various traces of gen AI methods, channels are processed through separate specialized branches before combining for final verdict. This design isolates semantic understanding of physical factors such as vanishing points from complex pattern and noise based artifact detection, allowing each component to specialize and achieve impressive results.

Model Architecture

The ConvNeXt-Large artifact classifier employs a dual-branch architecture that processes semantic and artifact information independently before fusion:

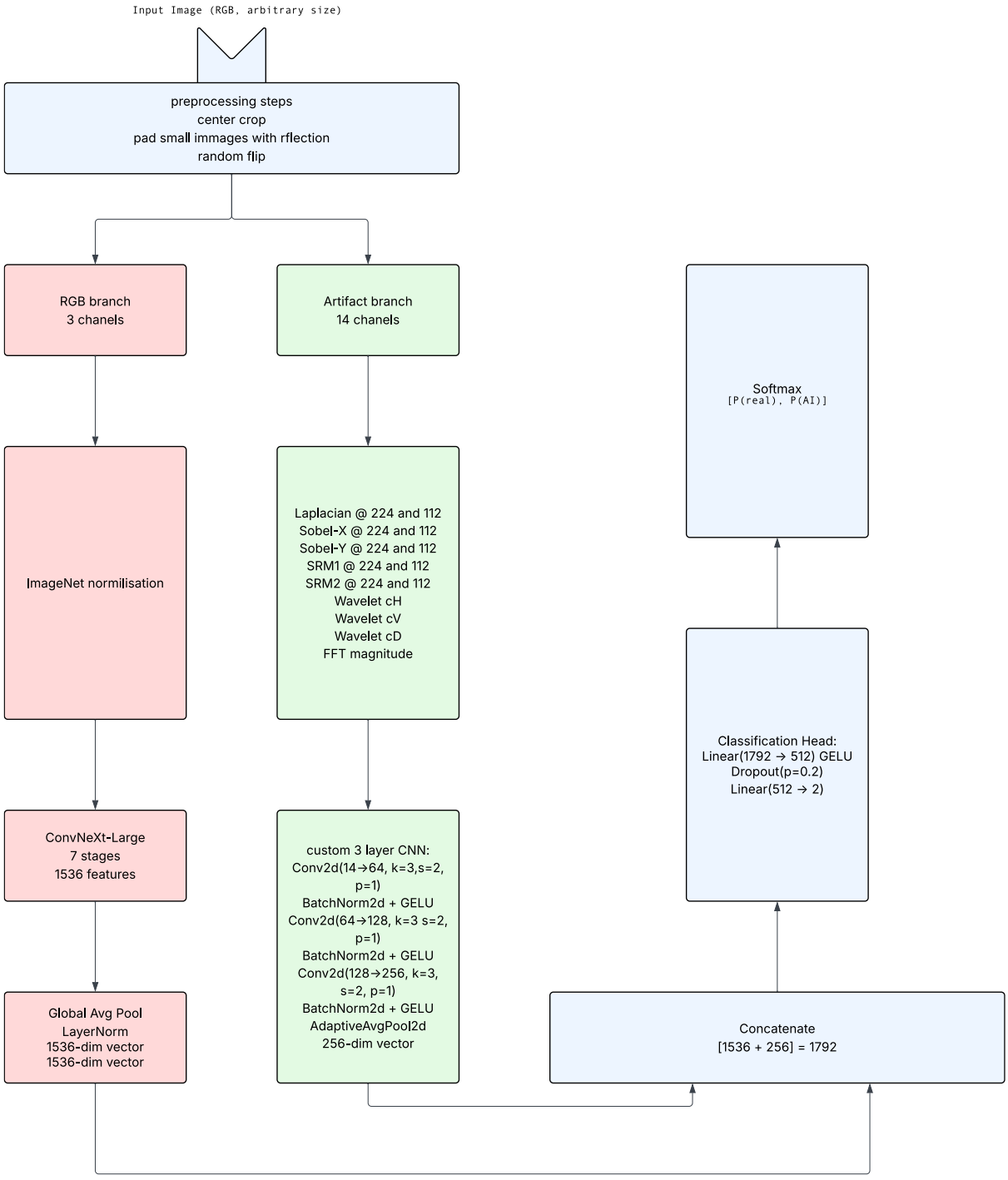


Figure 8 Design of ConvNext AI classifier

The semantic RGB branch component uses the pretrained ConvNeXt Large backbone, this is a modern CNN that achieves competitive performance with vision transformers while maintaining computational efficiency for inference on consumer hardware. The model started with ImageNet pretrained weights, providing proven feature extraction for natural image tendencies of all AI models. During training, the backbone had extremely low learning rates to prevent forgetting of these pretrained features, while the head and artifact branch were more dynamic.

The artifact branch component processes 15 channels through a bespoke lightweight 3 layer CNN. This branch is intentionally shallow to avoid overfitting to dataset specific noise patterns while still capturing complex artifact signatures. Three types of mathematical conversion were performed on images, they are:

- Residual Filters, these filters capture special information by looking at Neighbourhoods of pixels to extract various patterns and gradients, 5 Residual Filters are used for this classifier, each is calculated at a resolution of 224x224, and also at 112x112 then up sampled back to 224x224 to match other artifact channels allowing the capture of different scale's of artifacts:
    - Laplacian, useful for detecting second derivative discontinuities commonly found in GAN generated images as a result of extreme up sampling
    - Sobel-X & Sobel-Y, Captures directional edge artifacts from anisotropic generation
    - SRM1 & SRM2, Steganalysis Rich Model kernels designed for detecting statistical anomalies in image noise
- Wavelet Decomposition, filters capture both special and frequency information, the Daubechies-1 Haar wavelet transform the grayscale image into:
    - Horizontal detail, which exacerbates vertical artifacts (cH)
    - Vertical detail, which exacerbates horizontal artifacts (cV)
    - Diagonal detail which detects checkerboard patterns from transposed convolutions (cD)
- Finaly, FFT Magnitude, a low resolution Fourier transform of down sampled grayscale image, normalized to 0,1 similar to the implementation in the ResNet50 V2 classifier. This captures periodic grid artifacts invisible in the spatial domino but visible in the frequency domain.

All artifact channels are normalized per-image using subtract mean divided by standard deviation to ensure consistent scale across diverse content.

Dataset Construction and Bias Mitigation

Early training was conducted on a combination of various publicly available datasets that were shuffled and clamped to similar volumes in the combined dataset, this was done because many AI vs authentic image datasets have a specific theme e.g. AI art, so multiple were combined in an attempt to gather a more diverse image set in order to prepare the classifier for the wide variety of image formats and styles on the internet, however the model showed severe overfitting dataset biases, particularly image resolution. The initial dataset contained disproportionately small AI-generated images 512×512, likely from Midjourney V4 compared to large authentic photographs 2000×3000+ from professional cameras, causing the model to learn to only use image scale for classification, normalizing image size is common practise for AI image classification, this is usually not an issue because like other ai models, the ResNet50 FFT and ConvNext artifact models are not aware of the original size of the image when it reaches them, however the models generated for this project are specifically designed to detect invisible artifacts left by generative processes, including upscaling and downscaling, our models were successfully detecting these patterns, but the dataset bias is not reproduced in real world scenarios.

To eliminate this bias, a new dataset was constructed from scratch using a custom web scraping methodology:

Two LLMs (Claude sonnet 4.5 and Gemeni 3 thinking) jointly generated 400 diverse search queries spanning topics including landscapes, portraits, logos, art, diagrams, animals in multiple styles. This diversity is an attempt to gather data from as diverse of sources as possible terns rather than memorizing specific subject matter.

Using the Brave search API, for each search query, the web scraper would first search the query with a hard upload date window of January 1st 2010 to December 3st, 2019 which pre dates most public generative AI tools, then with a list of 4 rotating prefixes (Midjourney, DALL-E 3, Stable Diffusion, AI generated), would search again, with the date window of January 1st, 2023 to December 31st, 2025.

Each search returned exactly 100 images, all images that passed the following criteria were downloaded and added to the dataset

- Not previously downloaded using SHA-256 hash deduplication
- Minimum dimensions of 400×400 pixels
- Minimum file size of 10KB
- Successfully opens using the python PIL library, this is to check for corrupted images



Approximately 50% of returned URLs passed all validation checks,

The script was run until it ran out of the monthly free brave search AIP credits, by complete chance, the script almost perfectly overlapped with the turn of the month at 1AM gmt, which refiled the monthly API credits and produced a dataset of almost double the size, yielding a final dataset of 59,872 images (31,539 real, 28,333 AI)

The dataset was uploaded to and is now available publicly and open source on HuggingFace as LuckyCow/Ai_vs_real_web_images.

Preprocessing

A critical insight from V2 development was that common resizing customs distort the fragile resampling artifacts that the classifier is built to detect. The later V3 preprocessing pipelines eliminates this bias:

- Instead of downscaling, we perform a random crop to 224×224.
- Pad the image with reflection if smaller than 224×224 to make it fill the required size
- Random horizontal flip (50% probability)

This ensures both AI and real images pass through identical and minimal resampling operations. During validation and inference, the random flip is disabled and the crop is always in the centre, where FFT artifacts are strongest, and to make this classifier deterministic.

Training Configuration

The model was trained on an AMD Radeon RX 7900 XTX with 24GB VRAM using DirectML backend for Windows GPU acceleration. While it is widely accepted that AMD GPUs are less performative for AI workloads, this GPU was sufficient, however software support for all AI development tools is clearly built for CUDA first, with other architectures second, significant time and effort was dedicated attempting to use ROCm, the native AMD drivers for AI training, however ROCm is only supported on linux, windows users must use a VM, which introduces significant complications and refused to function during this project, to avoid CPU training, models were trained using DirectML, an open source alternative that supports windows, however ~20 hours of cumulative inference were lost due to buggy or poor features of this



driver over the course of this project, and successful tasks would have been simpler to setup and otherwise completed faster on a NVidia GPU or also likely if ROCm was used.

The following training parameters were used for training the ConvNext large Artifact model

Optimizer: SimpleAdamW, a community implementation to avoid DirectML incompatibilities with PyTorch's native AdamW

Learning rate: 1×10^{-5}

: 1×10^{-2} (L2 regularization on trainable parameters)

Batch size: 16 (limited by 24 GB VRAM for 224×224 images with 17-channel input when activations also scale with batch size)

Loss function, Cross-entropy loss

Epochs, unclear, likely ~15-20, various early checkpoint models that had success identifying biased up/downscaling artifacts were used to continue from

Validation split, 10% holdout (stratified by class)

Training Strategy, The model was initially trained for several epochs on the original biased dataset to establish baseline feature extraction. When the new web-scraped dataset became available, training resumed from the existing checkpoint rather than restarting from scratch. This transfer learning approach allowed the model to take advantage of learned artifact feature extraction. Training continued until validation accuracy began to plateau, at which point the process was manually terminated,.

ResNet-50 Spider Classifier

The ResNet-50 spider classifier demonstrates a second detection category within the Fire vs Fire system and was developed for testing purposes and to demonstrate the ease of developing additional classifiers using the base classifier. This model was developed with an unmodified ResNet50 backbone and without significant research, a basic architecture and training script were developed in less than an hour, followed by less than an hour of training using a public dataset from hugging face intended for classification of various spider species (Deng, 2026), this dataset was flattened to a single class and trained against a equal number of randomly selected images from the authentic half of the dataset generated for this project. This exercise proves that individuals can create similar personalized detection categories (e.g., blood, deep oceans, hostile sentiment/profanity) without requiring extensive ML expertise, while producing an effective and efficient classifier that interfaces with Fire vs Fire. The planned future feature for simplifying this process with premade architectures and a GUI, paired with the generic web content database would require non-technical individuals only to source images for the other half of the dataset in order to create their own AI models.

This model classifies significantly faster than the larger ConvNext model because it has a significantly lower parameter count (25M vs 197M), simpler architecture and significantly less preprocessing.

Architecture and Design Rationale

Unlike the ConvNeXt Large artifact classifier, which required extensive R&D to create the custom multi channel input and specialized preprocessing, the spider classifier uses a standard transfer learning approach optimized for speed and simplicity:

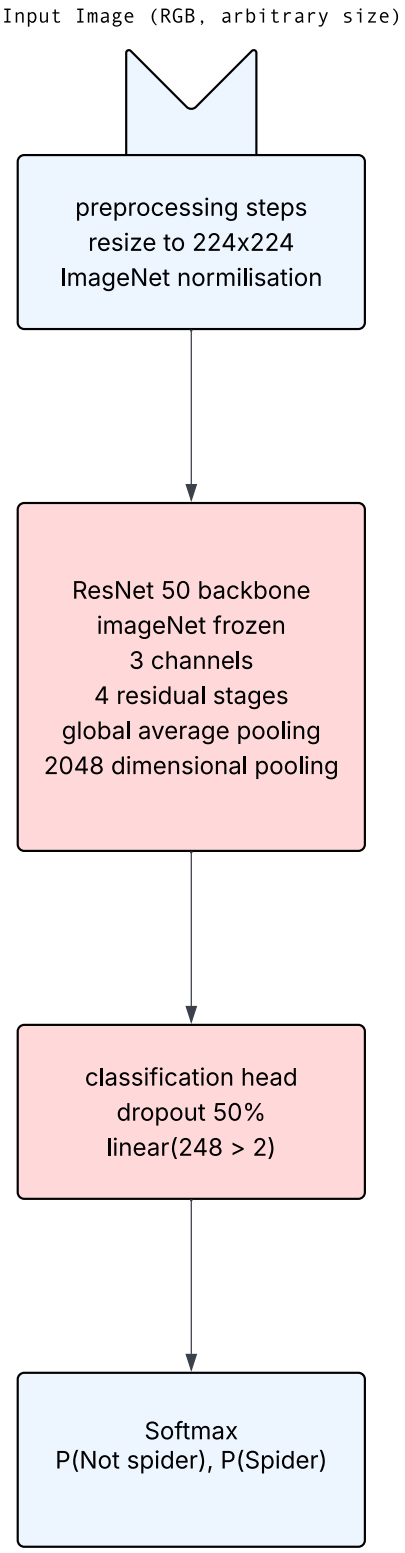


Figure 9 design of ResNet 50 spider AI classifier

In order to simulate the design process of a developer with less expertise in the field:

Backbone Selection: ResNet-50 was because it was the 1st recommendation of an LLM (Gemeni 3 thinking)

According to the instructions of the LLM, the backbone is frozen during training, with only the binary classification head receiving gradient updates. This reduces trainable parameters to ~4,000, enabling training on modest datasets without overfitting.

The only peacocking was Images being resized to 224×224 using bilinear interpolation and normalized using ImageNet statistics. A random horizontal flip was used in training.

Training was performed on the same AMD RX 7900 XTX desktop used for the ConvNeXt-Large model, using identical DirectML backend configuration. With the following parameters.

Optimizer: SimpleAdamW (DirectML-compatible)

Learning rate: 1×10^{-4} (10× higher than ConvNeXt due to smaller trainable parameter count)

Weight decay: 0.0 (no regularization needed for 4K parameters)

Batch size: 32 (2× larger than ConvNeXt due to simpler architecture)

Loss function: Cross-entropy loss

Epochs: 12 (early stopping when validation accuracy plateaued)

Validation split: 10% holdout (stratified by class)

Performance

The spider classifier performs ~5x faster than the ConvNext AI model, although it takes longer for the initial load to memory for an unknown reason, this Is not expected bue acceptable. It detects almost all spiders correctly, even when tested in a google image search whitch is a similar environment to where the training data of the opposite class was soured, when the spider class training material was mostly full body in view and scientific images for identification, During testing, there was never a single case of a false positive.

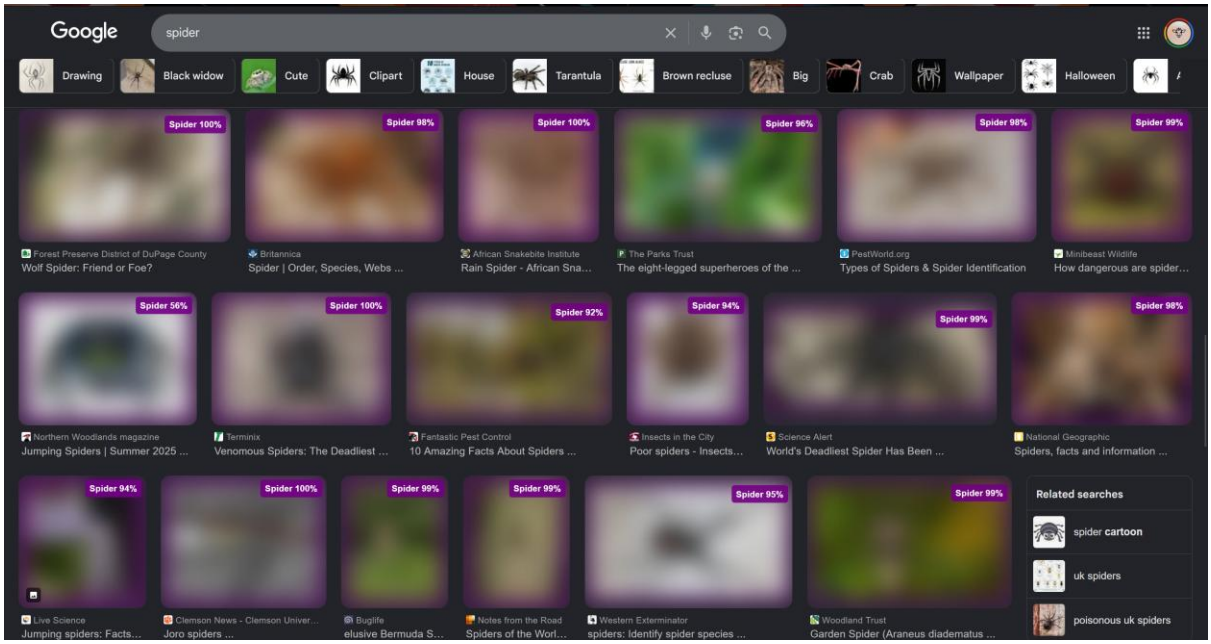


Figure 10 ResNet50 classifier sucsessfull page classificatoin

Hybrid DistilBERT + Features text model

Similar to the image models, multiple generations have ben developed for the text classification models

Evolution from Previous Architectures

TF-IDF + Logistic Regression (V1) The initial approach used traditional TF-IDF vectorization with logistic regression. This model performed poorly, failing to capture the nuanced differences between human and AI text beyond simple word frequency patterns.

Feature Based Classifier (V2) The second iteration extracted 20+ handcrafted linguistic features including perplexity burstiness paragraph length punctuation density, etc. and trained an AI classifier. However, this model suffered from critical flaws:

Unnormalized features, Features with different scales dominated training, many features were dependent on counts such as number of stop words, instead of % stop words.

Overfitting to paragraph length: The model learned to classify primarily based on text length rather than linguistic quality

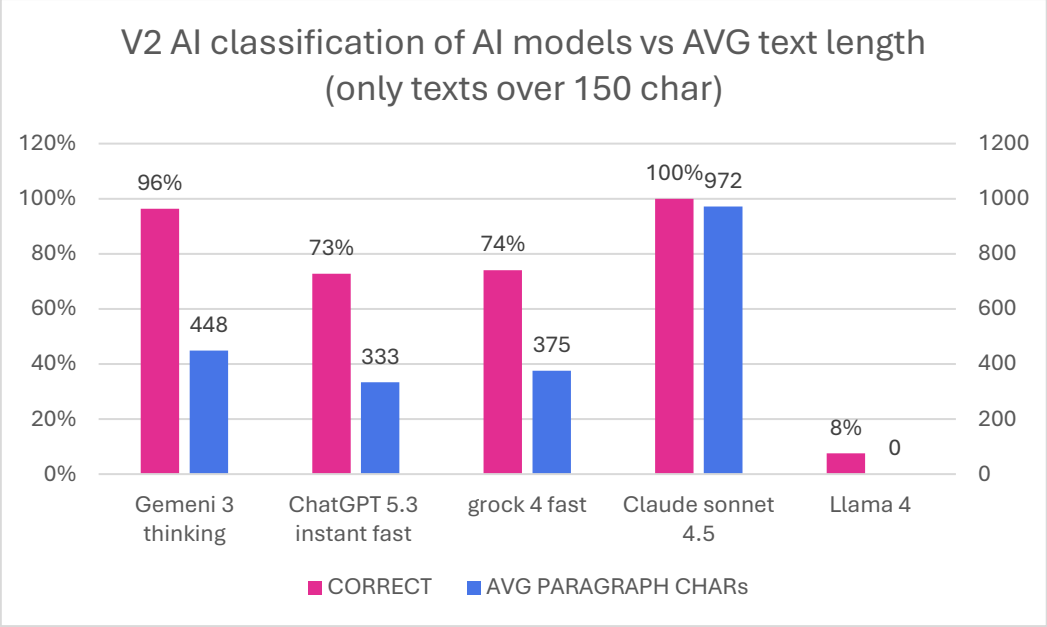


Figure 11 V2 AI classification of AI models vs AVG text length

Hybrid DistilBERT + Features (V3), this is a more powerful dual branched approach, converging in a single head, one branch discoveres deep semantic understanding while the other learns statistical patterns that distinguish human from AI writing, designed after the sucsess of the similarly designed ConvNext artifact image classifier. The current model addresses previous failures by:

- Normalizing all handcrafted features to comparable scales
- Combining DistilBERT embeddings with normalized features
- Balancing deep semantic understanding with explicit statistical signals

The features extracted from text for the current V3 model are based on the paper The paper (Mindner, Schlippe and Schaaff, 2023) and the discoveries made by analysing the weights of V2 to determine which features were causing it to overfit:

- Perplexity, Calculated using the GPT-2 small model which is available to be run locally
- Burstiness, a measure of the variance in sentence lengths
- Paragraph Average length, count (normalized)
- Lexical diversity Type-token ratio, unique word percentage
- Punctuation patterns like density, variety, ratios
- Syntactic complexity, average sentence length, word length
- Readability scores using the Flesch-Kincaid grade

All features are standardized to ensure that features like perplexity (range: 0-100+) and punctuation density (range: 0-1) contribute equally to classification decisions.

Perplexity measures how "surprised" a language model is by the text. Lower perplexity is more predictable and a sign that the text was whiten by AI, while higher perplexity suggests unexpected language choices. To calculate perplexity, a local GPT-2 small model is loaded, a rolling window is used to show the GPT model the previous section of text, then it generates what words would come next, the generated words are compared to the written words, the window is 1024 tokens wide, this is approximately 800 words wide, meaning the model should be able to touch the entirety of any web text segment that it is provided with from Fire vs Fire, which is designed to provide text segments in paragraph size snippets. The perplexity calculation is designed to calculate up to 256 tokens ahead.

Architecture

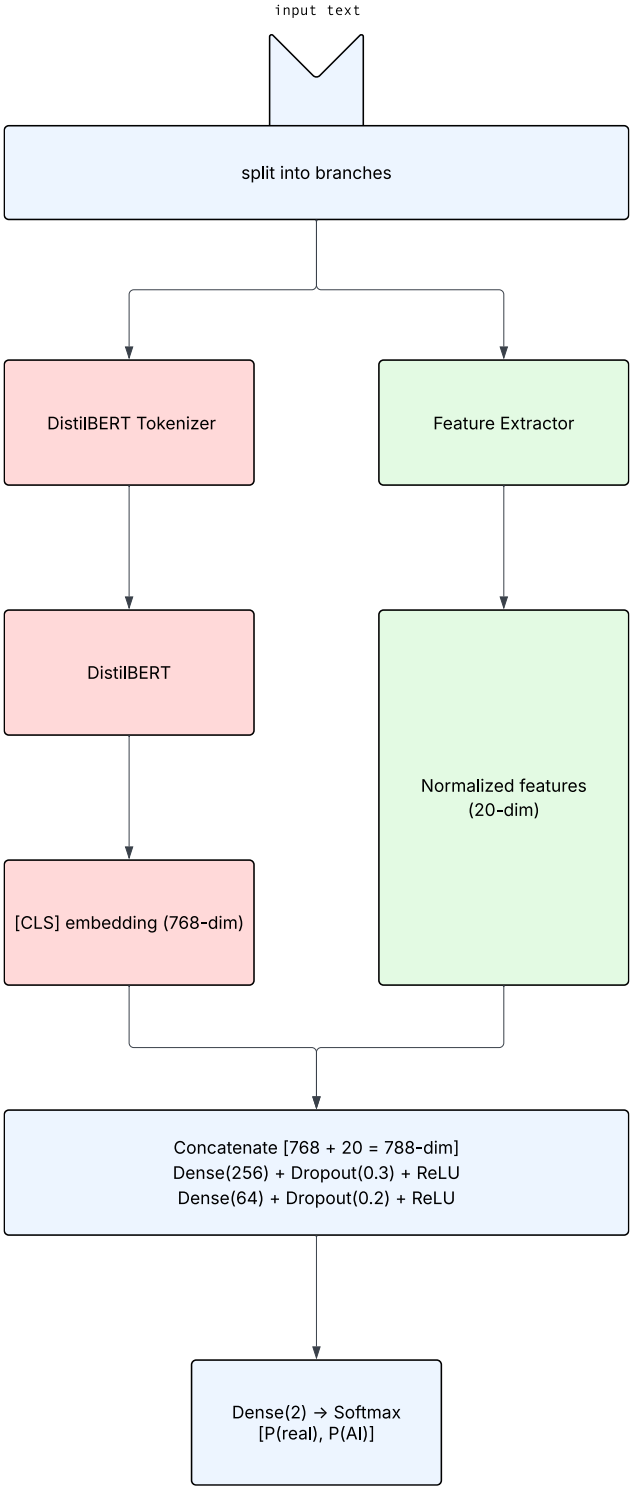


Figure 12 Hybrid AI text classifier design

Dataset construction

The r=training dataset was soured from the two hugging face repositories, this dataset provides diverse domains and writing styles from: social media, news, technical writing,

conversational Q&A, legal sources and more, and opposing text from 6 LLMs, ensuring the classifier generalizes across different text types and generation methods.

Total: 200,000 samples

- Human, 115885 (57.9%)
- AI, 84115 (42.1%)

Source 1, (artem9k, 2023), provides a Mix of Reddit WritingPrompts, OpenAI Webtext and NY Times articles with AI text from GPT-2, GPT-3, GPT-4, Gemma, Mistral, this dataset is 1.3M rows long, however training required perplexity calculations which take a long time to create, this dataset was shuffled and then ~120k rows from this were used, this dataset on its own was causing overfitting with sentence complexity, to balance this ~85k rows were added to the training pool from the second source: (DV, 2024) with the following dataset proportions:

- reddit_eli5: 51,336 human + 16,660 ChatGPT
- finance: 3,933 human + 4,503 ChatGPT
- open_qa: 1,187 human + 3,546 ChatGPT
- medicine: 1,248 human + 1,334 ChatGPT
- wiki_csai: 842 human + 842 ChatGPT

this additional dataset is majority human written and focuses on technical/scientific writing styles, with this dataset the model lo longer over relies on sentence complexity, and adapts much better to different writing styles.

Later, the datasets (mbzuai-nlp, 2025) and (Dugan, 2026) were also added, bringing the total training dataset up to 329,926 rows including:

- ~33,000 rows wikihow
- ~27,000 rows wikipedia
- ~25,000 rows peerread
- ~5,860 rows reddit
- ~6,000 rows other

In an attempt to further alleviate the models bias towards technical writing.

Training Configuration

The same machine was used to train text models with an 7900 XTX GPU using DirectML, the training process involved first calculating perplexity for 200k rows, the perplexity window for the Hybrid DistilBERT + Features text classifier uses a significantly larger context window compared to the V2 feature based classifier, calculation of perplexity scores took about 12 hours compared to 2 hours for calculation of perplexity for the V2 model, this should significantly increase the reliability of the most impactful signal.



Model Training:

- Optimizer: SimpleAdamW
- Learning rate: 2e-5 for feature/merge layers, 2e-6 for DistilBERT backbone
- Batch size: 32
- Loss function: BCEWithLogitsLoss
- Weight decay: 0.01
- Dropout: 20%

Software Testing and Demonstration

Testing environment

All testing was conducted on a galaxy book 4 Edge laptop with a 1st generation Qualcomm snapdragon x elite 084 ARM processor, this processor architecture is not common coming consumer hardware or well supported by windows, the process runs as intended on both ARM and x86 processors, for laptop testing, Fire vs Fire is running natively for in ARM, tested by running a large classification and observing the task manager, both the python.exe using 60% of the CPU and the Firefox runtimes are labelled as 64 bit (arm).

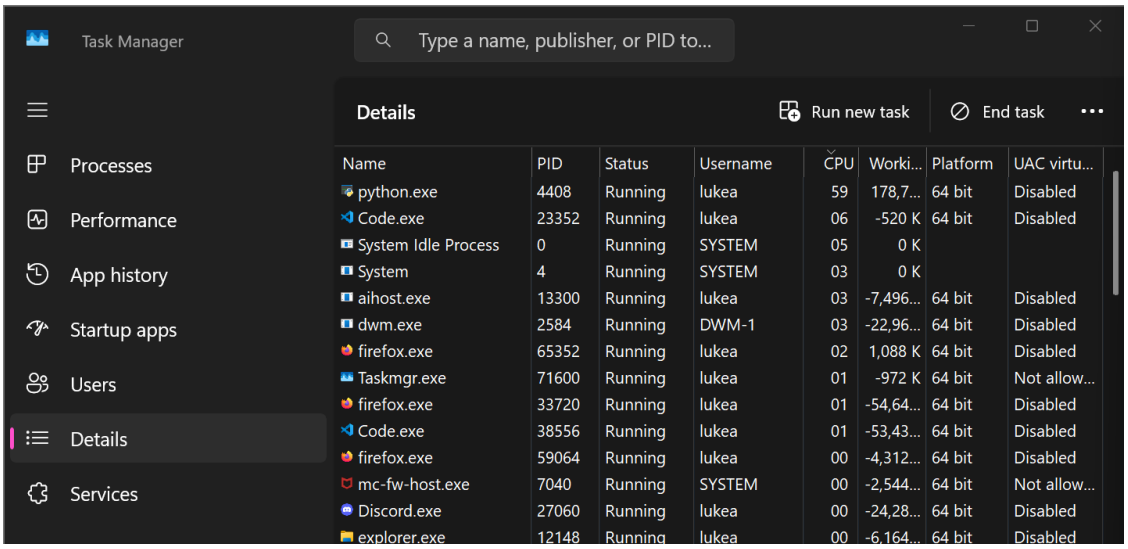


Figure 13 proof of native ARM CPU inference

The laptop used has 16GB of ram and no dedicated GPU, the snapdragon processor has an integrated GPU and NPU, however support for niche and new processor architectures was not a priority, all Fire vs Fire systems fall back to CPU on the qualcomm chip, including for

inference, testing was conducted on this platform because it less powerful than the x86 desktop used for training, when running small loading test on the desktop, processes loaded correctly and without issue, the GPU was found by classifiers with GPU inference functionality and ran on GPU without issue. All testing was conducted while the laptop was plugged in and power settings were set to “best performance” in windows.

Demonstration

This video was recorded on the testing laptop at the aforementioned settings and goes through the software testing demonstration in the same order:

https://drive.google.com/file/d/1KiV8BoDedtRiSCtTxeWaV_HGD5Qx4AB8/view?usp=sharing identical video also submitted with report and artifact.

It is encouraged that you watch this video as you read this demonstration section as it will give you a clearer understanding of the processes described here.

See appendix A for instillation instructions.

The browser extension is now fully operational and will work without any human input, however this demonstration will change various settings throughout in order to demonstrate functions of the project.

the Fire vs Fire logo appears in the top right corner of the screen when the browser extension is loaded, for this to happen the user must pin the browser extension to the task bar in the Firefox browser extension manager, this taskbar icon is the most convenient way to access settings, the settings can also be accessed in full screen mode by right clicking on the extension, pressing manage, then selecting options from the drop down menu, this was not demonstrated in the video.

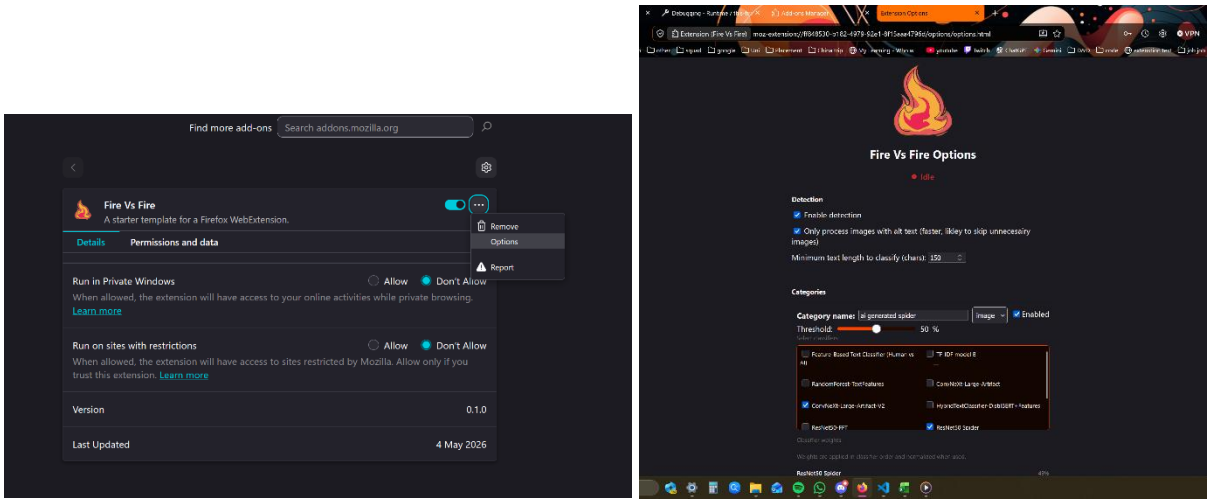


Figure 14 full screen options page instructions

For the demonstration, we first remove all the existing categories using the remove category button beneath each category, the category is removed instantly. A new category is now created using the create category button and the CovaNext large artifact classifier is selected and the threshold is moved to 60%, this means the classifier has to be more confident that a image is generated by AI in order to be flagged, 60% is known to be a good setting for this classifier to avoid false positives. When the user opens the options menu, the browser sends a query the available classifiers in the classifier folder, each classifier is called and returns its name, a classifiers name is not the same as the name of the classifier file.

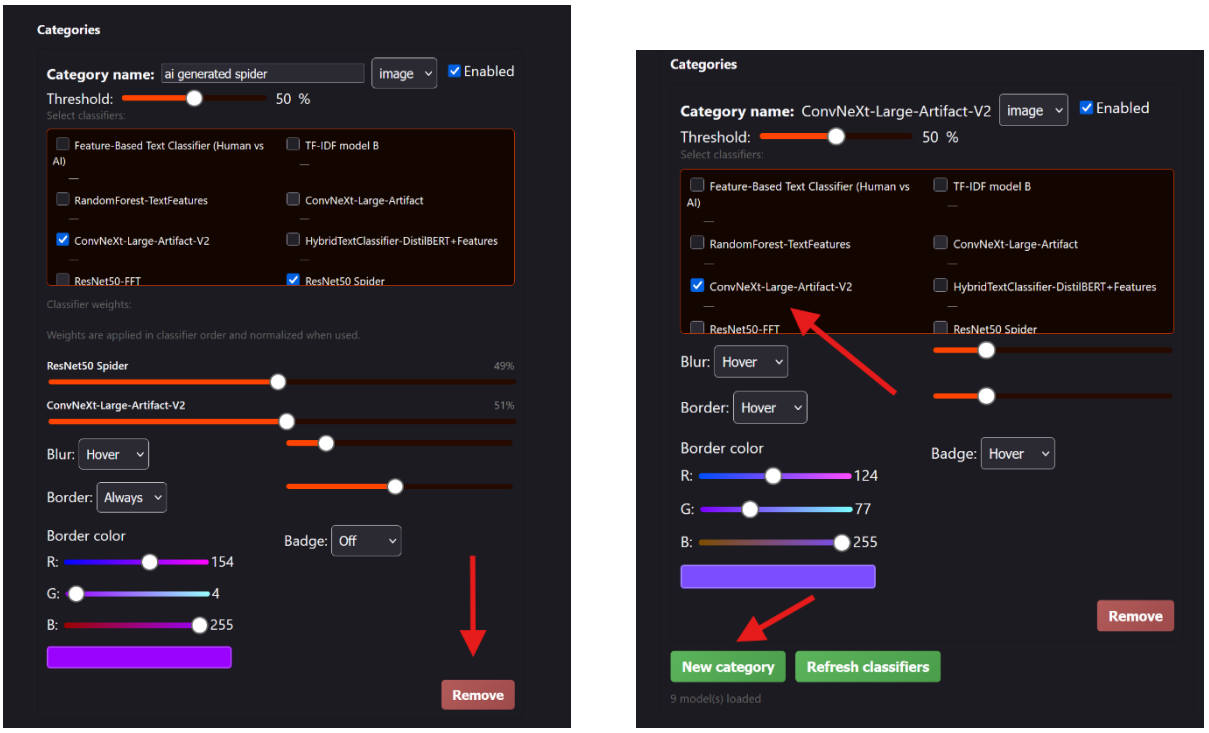


Figure 15 classifier creation and removal

Next, a new google search is initiated for cats, immediately, the browser extension detects that there are images on this page, the browser extension collects the image sources from the DOM and uses them to extract the image data from the Firefox's memory, this avoids downloading images twice, the images are batched and sent to the local server as JSON, including the image data, the local server receives this message, it reconstructs the batches into one list, then separates the list by modality, all content is sent to all relevant classifiers, at this point there is a long pause where the ConvNext classifier is loaded into memory, this can take up to 30 seconds sometimes and is a known flaw of the current process, however this only has to happen once and then the model stays in memory, unless the user has selevted the instantly unload models button in the dev section at the bottom of the options. During this time the logo of Fire vs Fire changes to a green flame to indicate that it is working, in the options GUI the large logo also changes, the number of outstanding classifications is shown in the options GUI and on the extension logo.

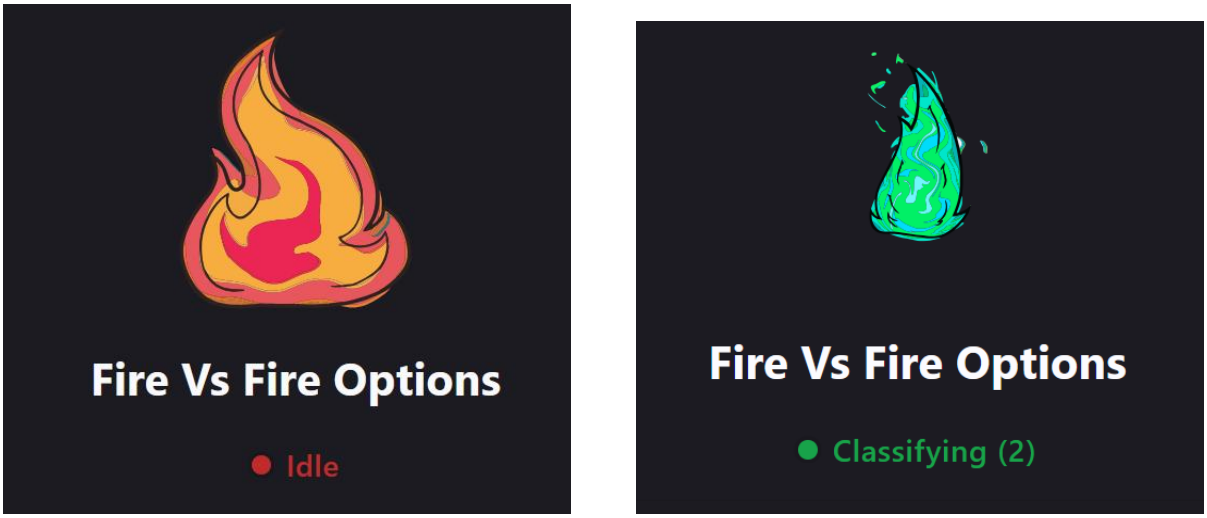


Figure 16 status display system

Once the model finishes loading, it classifies the list of images, it finds no AI images on this page, so nothing changes and the system goes back to idle mode.

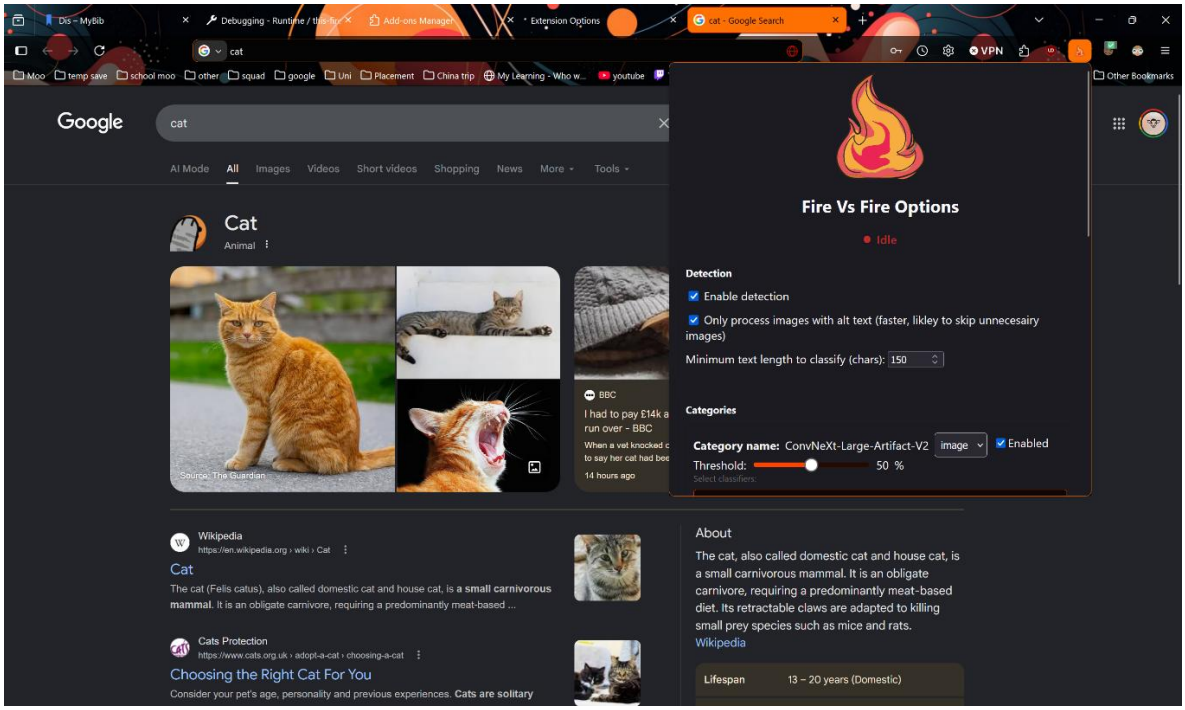


Figure 17 google search classification complete

To demonstrate a worst case scenario with many images, the google search is switched to image mode, upon loading the new webpage, the browser again immediately sends images to the local server, this time the model is already loaded and processing finishes much faster, the model identifies four images on this page as AI generated (different images in the video), images appear in batches because the local server is sending classification results back to the web browse as they finish inference, this means the user does not have to wait for all content

to finish classification before they receive visual changes on their end, to demonstrate the ability of the classifier to identify synthetic and authentic images, we now search for AI generated cat in google imges, the system responds quickly again, the classifier has identified the majority of images on this new web page as AI, each image receives a badge with the classifier's confidence that the image is AI as a percentage, the badge also contains a short human readable label for what the content is, this label is set in the classifier, however, while the current weights in the ConvNext classifier are relatively new after being trained on the bespoke dataset, the classifier harness is older and predates the label text system, the base classifier class is backwards compatible with these older models and defaults to show the text "AI generated" if no label is given, the label also shares the colour of the category.

During the demonstration, images without alt text are ignored by the system because the option is toggled on in the options GUI, this is a elegant distinction that usually prevents 99% of UI and navigation controls from being sent to the local server, slowing down the system and impeding the ability to navigate, this filter also usually catches smaller and less relevant content as well as backgrounds, such as the suggested search refinements bar between the main images section of google search and the search bar, none of these images are considered for classification.

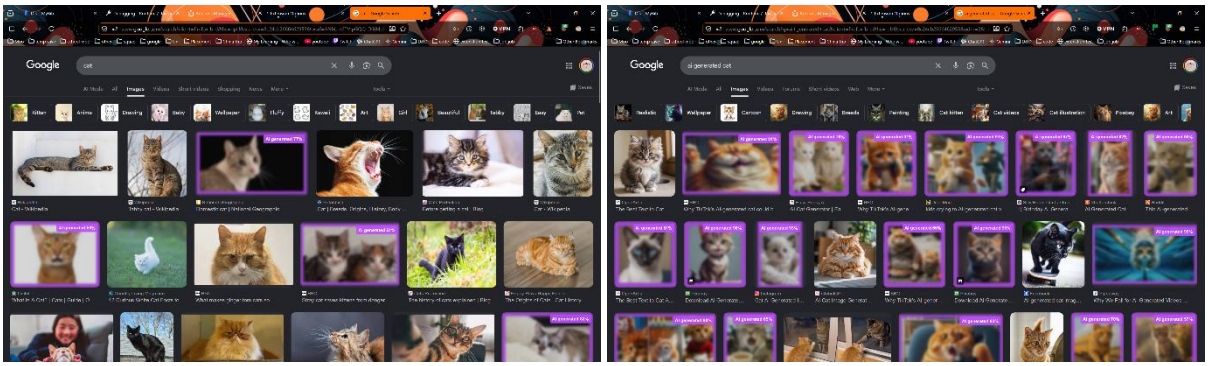


Figure 18 google image search classifications and flagging

To demonstrate the image classification system in a more generic website, we now navigate to Wikipedia and view the page for cattle, after waiting for all classifications for this page to finish, we scroll down and inspect the page for false positives of classified images, finding none, we then search Wikipedia for the generative AI page, upon opening this page, the example AI generated image at the top is immediately flagged as AI, classification on lighter web pages such as Wikipedia respond quicker than google searches because there is less overhead retreating the image data from RAM, meaning classification can start sooner, after waiting for the rest of the images on this page to finish classifying, we scroll down to inspect other images on the page for flagging, we scroll past numerous human generated diagrams and graphs that are not flagged and find one other example AI generated image that has been correctly flagged by the system, with no false positives on this page either.

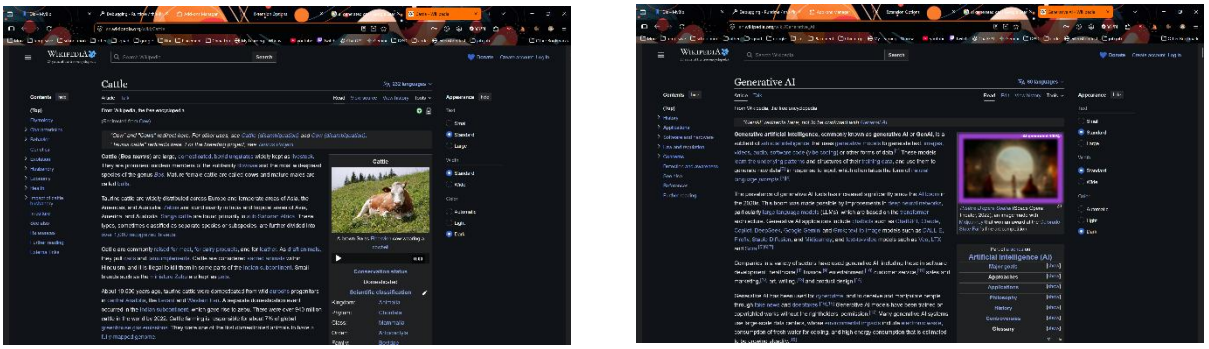


Figure 19 Wikipedia page classifications and flagging

Next, to demonstrate the modularity of the system, we will create a second category and select the ResNet50 spider classifier, trained to identify images with spiders in against all other images, for use by arachnophobic people, shit model must again load slowly as it is the first time it is being used this session, while it is loading we assign a purple colour to the spider category and a orange filter to the AI category, once loaded the spider category rapidly labels the majority of images on the page, this model is smaller and faster compared to the large AI classifier, object detection im images is far simpler and more reliable compared to identifying complex invisible artifacts needed for AI classification, this model is also significantly more accurate and has never been seen to make a false positive during any testing. We will now search google images for ai generated spiders. The AI classifier responds first, then after all the loaded images, including ones beneath the viewport are categorised by the AI classifier, the spider classifier does the same, it was initially intended for this to happen in parallel however testing found that this actually slowed down classification, at least with this configuration and hardware. The majority of the images are classified as both spiders and AI generated, the spider classifier is less successful in this example, likely because it was trained using scientific images of spiders, not fluffy rainbow unicorn ones, you can now see that when multiple categories classify the same image, the category colours and formatting are merged, images that have been classified as both AI and spiders have a pink border, yet retain their original coloured badges, that stack vertically and show the confidence for both classifiers individually, the spider classifier is also more modern, using the label system to label images as “spider”.

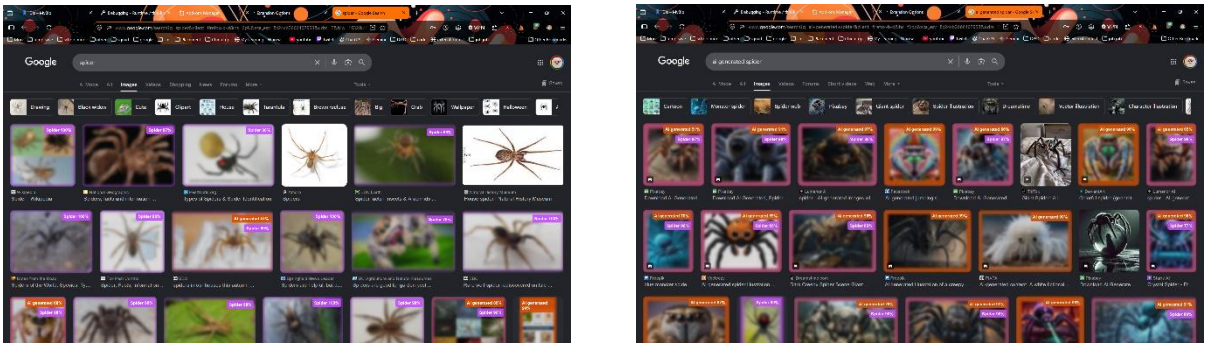


Figure 20 dual category classification example

Now, to demonstrate the category system’s ability to process images using more than one classifier per category, we delete the AI category, and select the AI classifier alongside the

spider classifier in the spider category, when we removed the AI category, all imaged belonging to that category on the page loose the AI tags and formatting, since both of these models have already been used this session, we don't have to wait again for the models to load into memory, the GUI of the category has not changed to include additional elements relevant to multi classifier categories, we can now change the title of the category, this becomes the new label text, instead of randomly picking one of the labels from the classifiers, we can also modify the weight system, to decide which classifiers have more of an effect in this category, as we change these weights, we can see the webpage flagging system automatically updating according to the changes live, without having to reclassify since the browser extension stores the classification results form the individual classifiers, even if the classification results are combined and never seen on their own, this is also true for single classifier categories and changing the category threshold, but this is not demonstrated in the video.

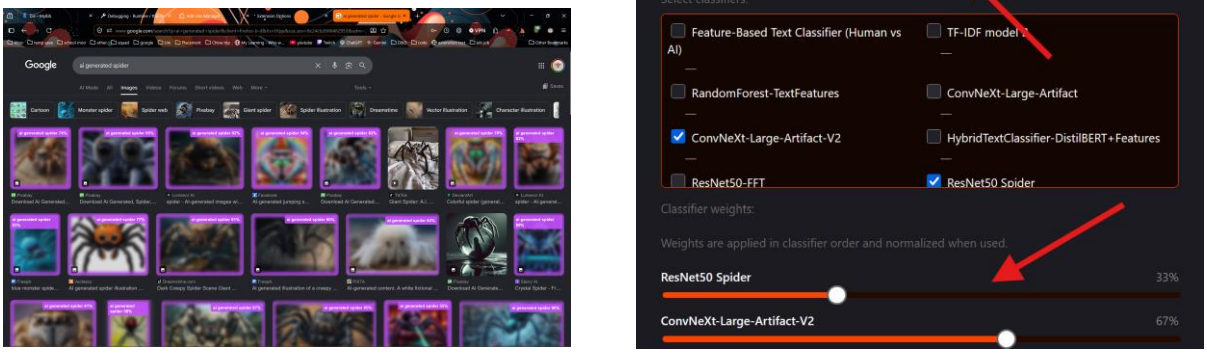


Figure 21 multi classifier category example

There is now a short demonstration showing some of the flagging options for images, and using the 3 options for flagging option behaviours, which are explained elsewhere in this document

We will now move on to the text classification demonstration, we create a new category, above the classifier section of the category GUI, we can change the category modality to text, the user is prevented from choosing a classifier of an incompatible modality if they try, even though all currently available classifiers are visible, we select the 3rd generation hybrid classifier, this is currently the most accurate classifier for Fire vs Fire, first we go to reddit stories, the browser extension detects text and sends it to the local server, once the hybrid classifier model loads, it immediately classifies the whole page, this classifier is incredibly fast, achieving speeds in excess of 1000 text segments/s, we start scrolling down until we find a reddit story that has been identified as potentially AI generated, when viewing the full story, it seems that most of it is AI generated or AI assisted writing, when hovering the cursor over flagged text segments, the AI confidence score is displayed, we inspect other reddit stories and find that the vast majority are not suspected of being AI generated. Next we navigate to the google Gemeni LLM website and give Gemini the prompt to write 10 paragraphs about generative AI, the majority of text segments are classified as AI generated, now we repeat this

test with chat GPT, where it successfully classifies all text segments that were long enough to be considered, it is expected that this model is more successful at identifying chat GPT generation because it uses a locally run GPT 2 small model for perplexity calculation, and later GPT models are more likely to have similar writing patterns to previous GPT models compared to unaffiliated LLMs like Gemeni. We now demonstrate some flagging options similar to the image demonstration.

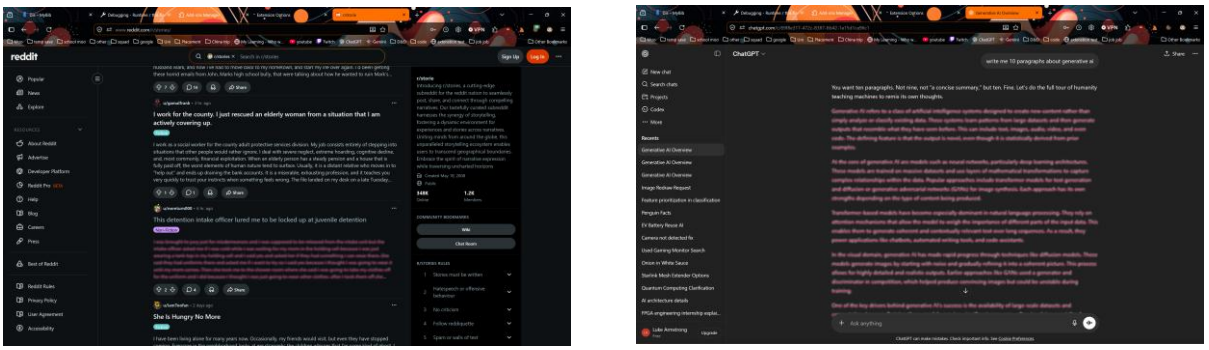


Figure 22 reddit stories and GPT classification and flagging

Finaly, we will demonstrate the manual context flagging system using the right click menu, when right clicking on any content using the Fire vs Fire extension, you will find the logo of the extension at the bottom of the context menu, depending on the status of the content you interact with, the prompt will either be “remove x tag”, which will remove all classification results and formatting from a image or text segment, or the prompt will say “blur this image/text” which will assign the manual blur category to that content, this is a static category with a heavy blur and grey colours

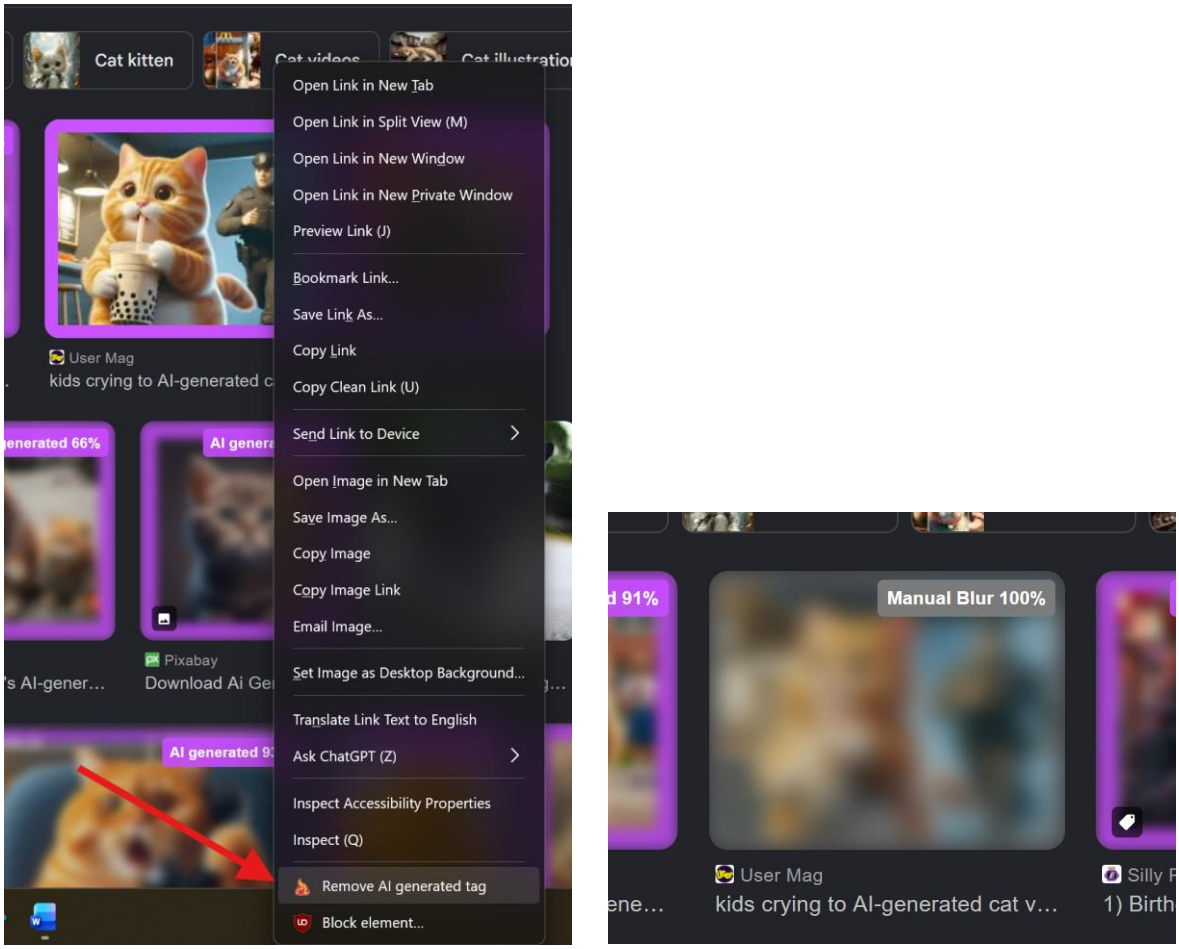


Figure 23 right click manual flagging system

Conclusions

This project was ambitious, and since planning, feature creep has taken this project far past its initial concept in terms of features and scope. The title of this project is Development of Fire vs Fire - an Automated Modular Browser Extension for Identifying and Filtering of AI Generated, and Other Content in User Interfaces, other undergraduate solo projects may focus on developing a specific ML solution to a problem, or development of a functional website, this project's scope exploded out of control and necessitated not only the complexity of developing the browser extension from the title, but also of a server infrastructure, bespoke communication standards, a complex modular system designed for open source and repairability, an information dense GUI, generation of a bespoke and purpose built dataset, and extensive R&D of multiple highly unique and nuanced ML models, for multiple separate modalities, that must each operate with ambitious accuracy goals and balance against size and speed constraints for unknown hardware, to detect invisible signatures that have already been compressed by web processes. If average classification tasks required 17 hand crafted

image channels, there would be pretrained backbones that exist for this task and meet the size and speed constraints of this project.

Despite this, this project has been a success, when evaluating the browser extension, it is a highly feature rich and adaptable solution, many changes were overcome in its development, and its current success only inspires further development and improvement. It remains the plan to continue working on this design past the deadline for this prototype, with the end goal of releasing this project sometime in 2026, many individuals have expressed interest in using this project after learning of it's goals, further showing that there is an appetite for a system like this among the population, not only for individuals who are aware of their lack of ability to distinguish AI from reality, but also for others who can, but are sick of AI content filling their vision when navigating their digital spaces.

Development of the AI classifiers for this project are a secondary objective, it is a feature of this project that the AI models are easily interchangeable, and users can make their own classifiers to suit their needs, the decision was made early in this project to aim for the least capable hardware that hat this project is possible to run on, to this end, the project was developed for a thin and lite laptop, not equipped with a dedicated GPU and using Qualcomm's first generation of processors intended for non mobile devices, the project aims to be accessible to as many people as possible, so the decision to only use the more powerful desktop with a modern flagship GPU was made, in future, more powerful models will be designed by users to run on their high end machines, these have the potential to be orders of magnitude faster and significantly larger, allowing for higher accuracy. The models that are included in this prototype meet the requirements and perform well by all metrics, they are considered a lesser success compared to t browser and infrastructure however because they generalise badly beyond their strengths in classifying content in a similar format to their training data, further improvements could be made by simply expanding training data, now with the knowledge of what the current models succeed in doing and where they struggle, the iterative design process of AI models for this project will continue.

Critical Evaluation

Browser extension evaluation

The browser extension has developed to become a feature Ritch and effective tool, it effectively accomplishes it's objectives, and overcame unforeseen obstacles

GUI

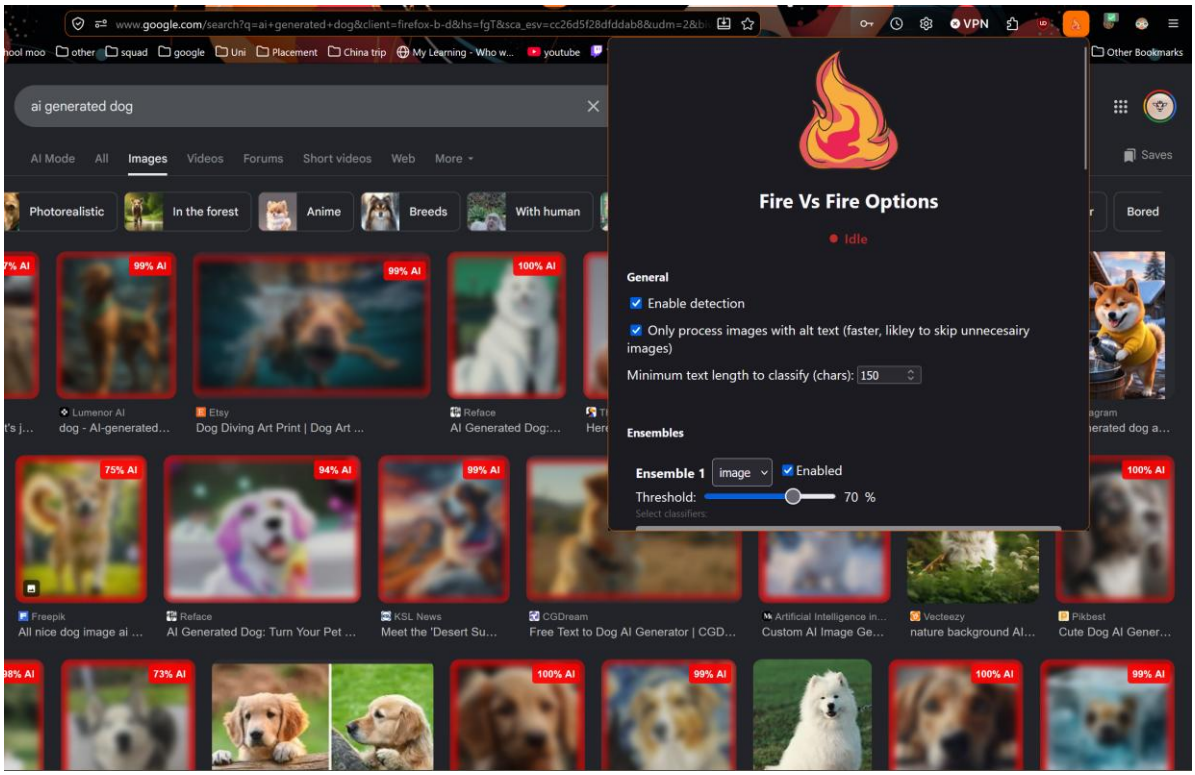


Figure 24 GUI example

GUI implementation is extremely functional, it is intuitive, responsive and interfaces with the external local server as needed for settings changes. It is accessible in a full screen options menu or as a popup from the pinned extension icon, it is split into 4 sections: the first section contains the Fire vs Fire logo, title and status of the system, while the system is classifying, the logo flame changes to a blue “powerful” flame similar to a Bunsen burner, and the amount of classification batches is shown below. This is a uesfull diagnosis and feedback tool because classifying large batches of immages or rapidly opening new tabs with content can backlog the system and make it apear like nothing is happening for tens of seconds. The classification number and status is also represented by the pinned extension icon in the firefox toolbar. Thease elements provide clear and fast feedback for the status of the system.

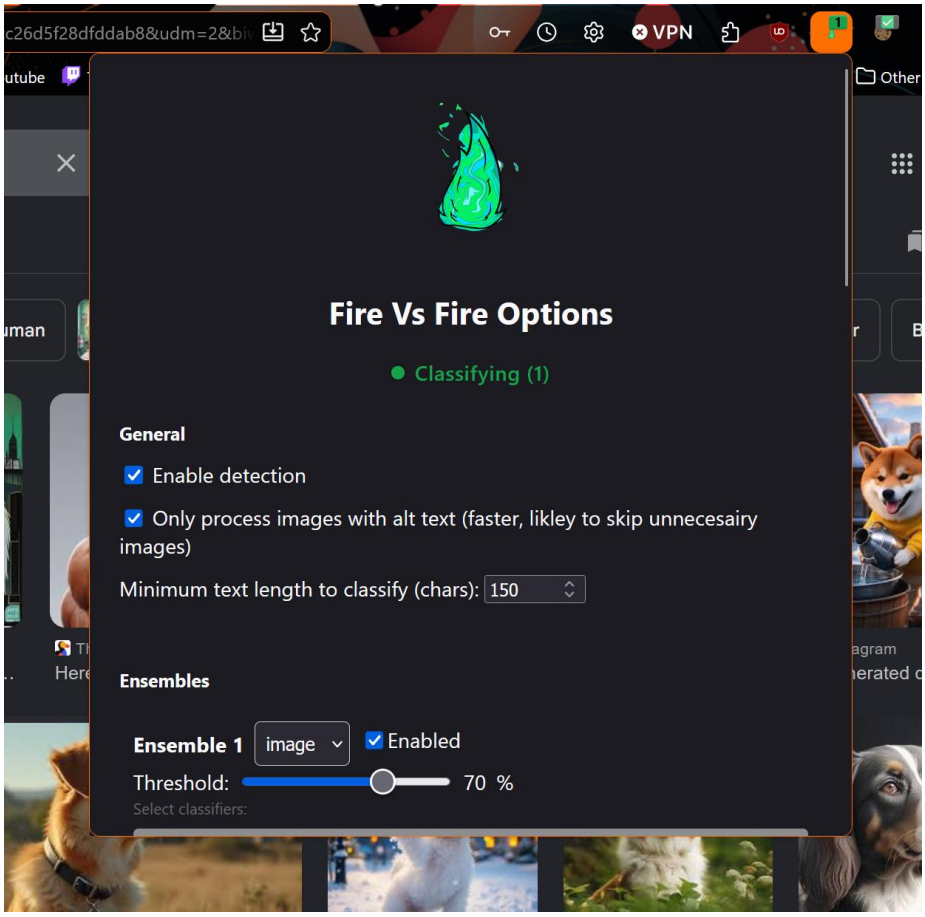


Figure 25 classification status display

Both categories share similar elements of their interface, the first is the threshhold, this defines the confidance needed for content to be flagged, below are the classifiers, classifiers are automaticaly detected from the classifier folder upon startup, or when the “refresh classifiers” button is pressed, all classifiers are shown in both cattegories, however if the user tries to pick a classifier that does not support this modality, the UI does not let them. Below are the flagging options that will be aplied to any content that passes the threshold. Most flagging options have 3 modes: off (effect is not aplied), hover (effect is aplied unless hovering), and always (effect is always on). Images have the option of aplying a gausian blur to the image with customizable intensity, giving the image a boarder, the border width scales automaticaly with the size of the image, with a custamizeable width multiplier, and a badge that displays the confidance that an image is AI. The badge colour and border colour are controlled with one interface of 3 sliders (RGB) and a preview, firefox has a built in colour picker however the Fire vs Fire options are displayed in a popup that closes if you try to interact with the separate firefox colour picker popup. Text has the same blur controll and can also be highlighted, underlined or struck through, each element can be coloured, however underline and strikethough to the same text must be the same colour, the underline is set to take priority. If the user hovers the mouse over any content, it will show the AI confidance.

Below the category section there are some debugging tools for development including the inactivity timer before the browser extention send a batch, controll over the quality of images sent to the local server.

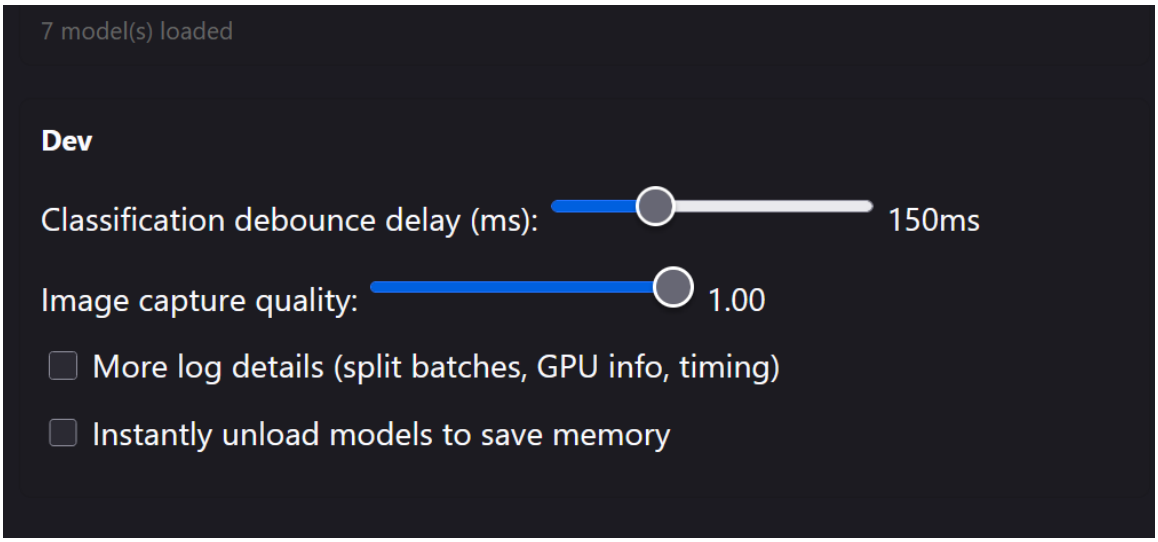


Figure 26 developer controls GUI

Content capture/flagging

Flagging is well implemented, flagging techniques adapt to all different webpages and formats without obstructing web navigation or information. The flagging system is highly customizable in an intuitive interface, classified images and text have unique and varied flagging options to support multiple categories and preferences, below are some screen captures of flagged content.

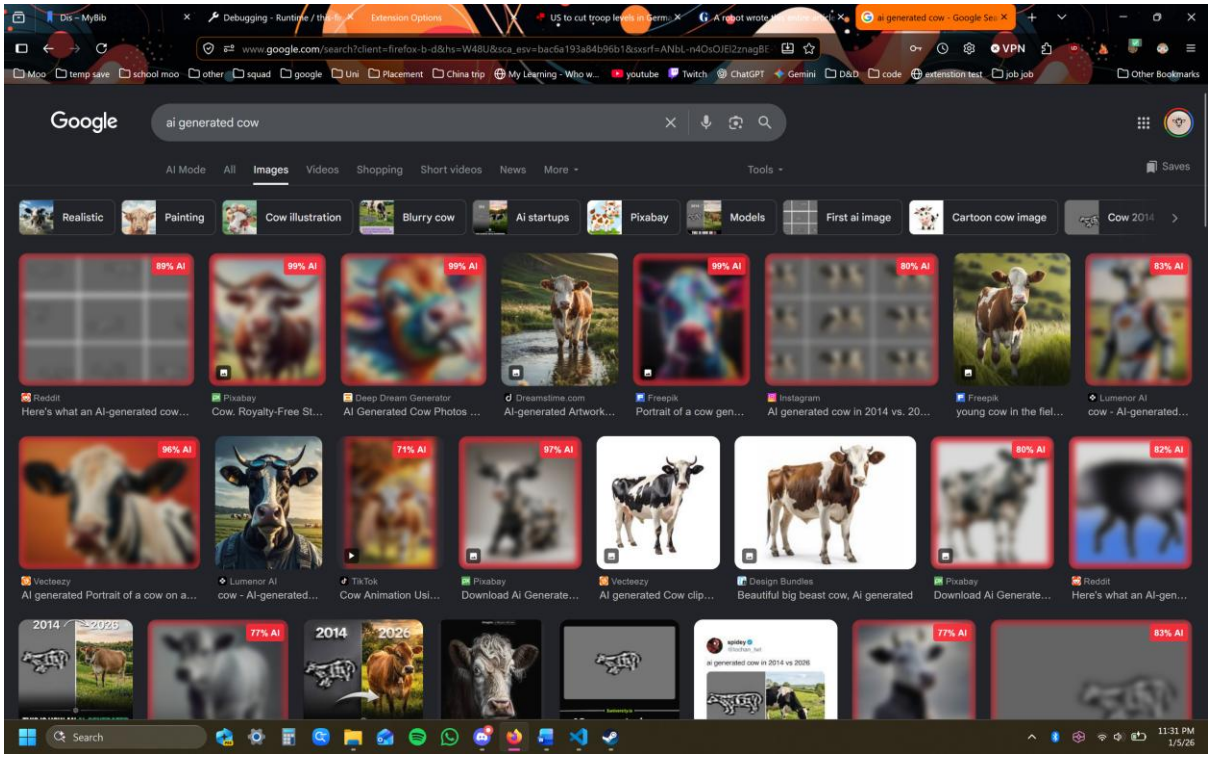


Figure 27 flagging example

When a single item of content is flagged by two or more separate categories, formatting is merged successfully, Tags for all categories are separately added to show the confidence of each category, example below of AI generated spider google search.

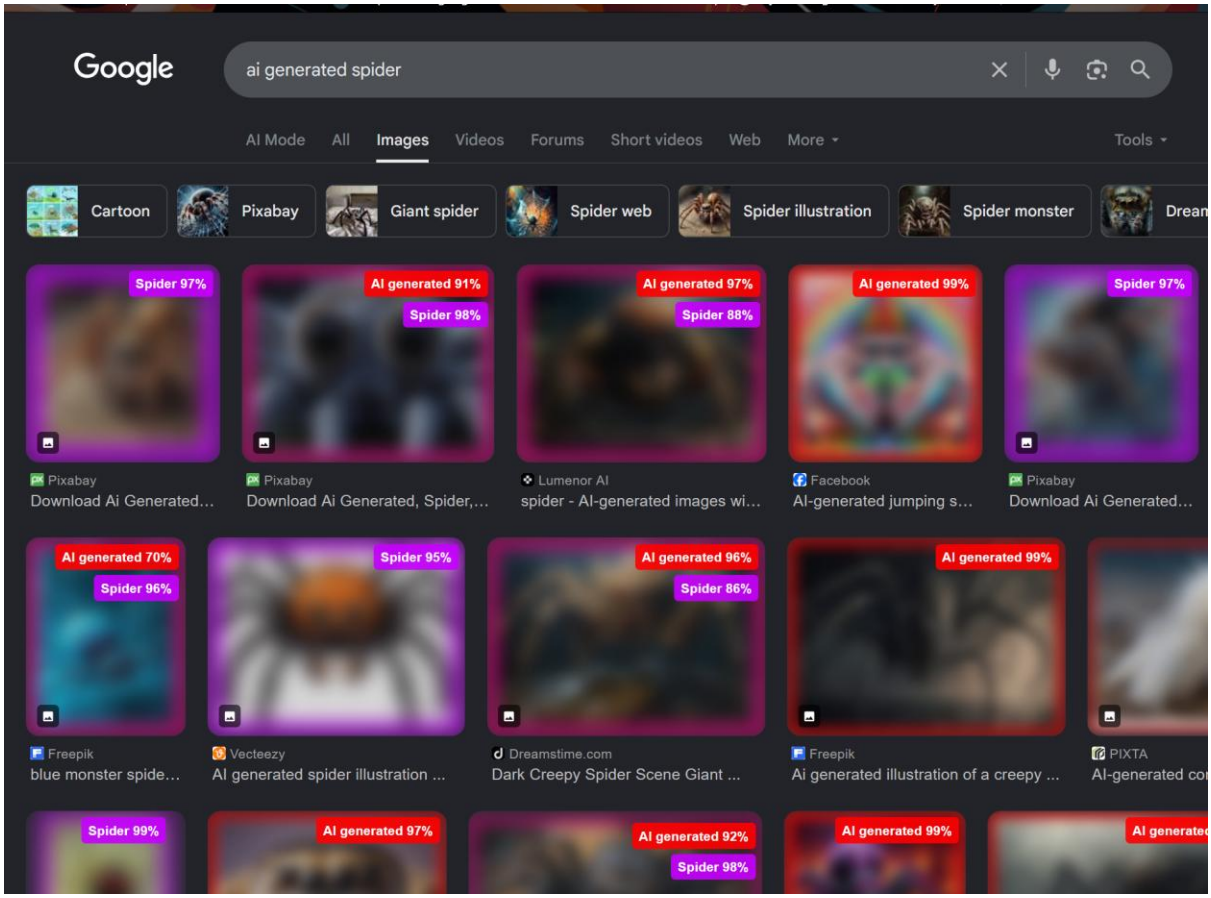


Figure 28 dual flagging example

One obstacle from the development of the flagging system is that flagged content that is layered beneath other elements in the DOM does not detect the generic mouse hover, usually a web developer would notice this and fix it easily, however it is common that the content getting flagged was never intended to use this feature, therefore a custom mouse pointer listener was implemented to detect when it overlaps any flagged content.

the browser is adaptable and manages to see most types of data, only sites that aggressively prevent data scraping are not detected by the script. Many sites such as social media platforms employ JavaScript content delivery, so that they can choose what content a user sees with an algorithm, this content is not included in the DOM and requires nuanced but successful algorithms for detection. The system is shown working for AI generated thumbnails in YouTube here:

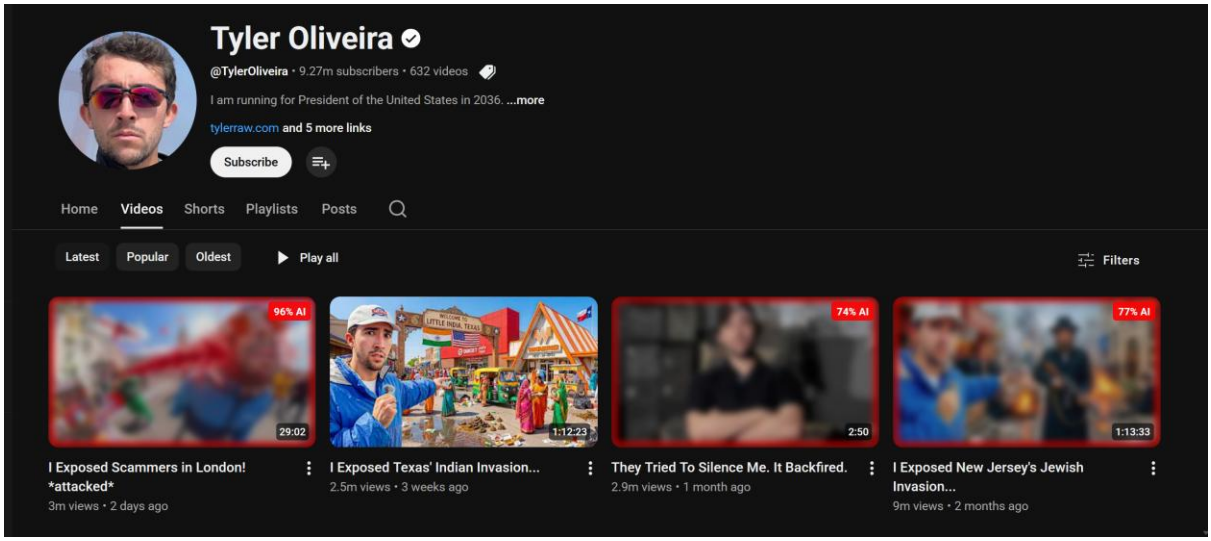


Figure 29 content detection and flagging of JavaScript provided content

Peroformance and usability

The browser extension does not impede the browsing experience of the user in the vast majority of situations, however some edge cases remain, UI elements can still sometimes be captured by the content detecting script, this can lead to some UI elements being blurred unnecessarily, additionally a some websites have reduced performance when using Fire vs Fire, for these websites the browser becomes sluggish and operates at a reduced FPS, it is assumed that this affects all websites however to differing degrees, on most websites it is not noticeable, on some such as YouTube it is slightly noticeable when compared to normal browsing on a 120hz screen, the worst known case of performance loss is store pages for individual games on the steam web store, performance is significantly reduced but only while the game screenshots and images are in the browser view box, scrolling down the page increases performance back to normal, this does not affect any other parts of the steam website. The performance loss does not seem to corelate with any type of content or extreme quantities of content being displayed. This bug can be observed at https://store.steampowered.com/app/1422450/Deadlock/

Image classifier evaluation

Performance

The primairy image classification model for Fire vs Fire is currently the image classification model generation 3: ConvNeXt-Large Artifact, it achieves an average of 73% accuracy, almost meeting the initial goal of 75% from the original project proposal. Test Ing methodology was: searching for “AI generated x” vs searching for “x” using google search, repeated 10 times, the number of AI labelled images vs total images was counted for the first 3 rows of results:

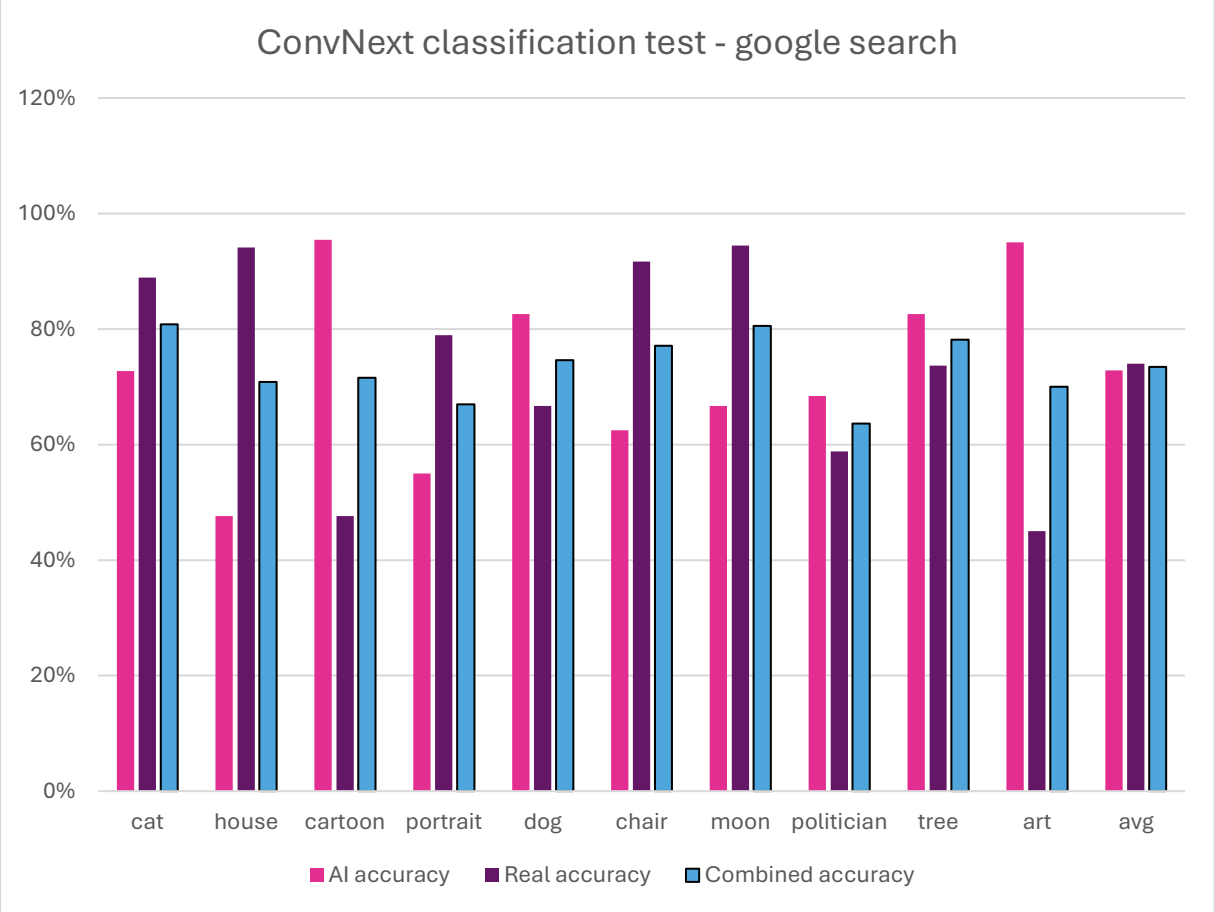


Figure 30 ConvNext classification test results graph

AI Y	AI N	RL Y	RL N	search term	AI accuracy	Real accuracy	Combined accuracy
16	6	2	16	cat	73%	89%	81%
10	11	1	16	house	48%	94%	71%
21	1	11	10	cartoon	95%	48%	72%
11	9	4	15	portrait	55%	79%	67%
19	4	6	12	dog	83%	67%	75%
15	9	2	22	chair	63%	92%	77%
14	7	1	17	moon	67%	94%	81%
13	6	7	10	politician	68%	59%	64%
19	4	5	14	tree	83%	74%	78%
19	1	11	9	art	95%	45%	70%
				avg	73%	74%	73%

Figure 31 ConvNext classification test results table

This test is deterministic and reproducible, this test was conducted in a live environment inside the web browser using the full Fire vs Fire pipeline, screenshots of every test result are located in the appendix.

During this test the model recorded an average classification time of 3 images/s, this is also slightly short of the initial goal of 5 images/s, however this test was performed on a laptop CPU and compared to other systems measuring seconds/image this speed is acceptable.

Limitations

This test is meant to be representative of modern browsing images from the web, which can have different artifact signatures to the dataset the model was trained on (2010 to 2020), in order to test the model properly in real world conditions, no date filtering was applied to the searches as it was in dataset generation, this inevitably introduces AI images into the search results of the real image category, real accuracy is likely higher but it is impossible to prove to what extent. These model weights achieved a 88% accuracy score during the validation of the final epoch during training.

These results are a promising indicator that a local classification model is viable, however accuracy could always be improved. The test environment was fundamentally similar to the dataset creation method even when a different search engine is used and no date window is enforced, however this model is still a result, and component of, ongoing research and development, this prototype has various limitations, mainly related to its ability to generalise to different formats of web images, however the goal of this model was not to be the final perfect solution, but instead to assess the viability of small, fast classification models to exploit generative AI artifacts from generic images that have already undergone whatever standard compression and processes happen to images that are available on the internet, for this goal, the model has proven to be an outstanding success.

From general use of the model over the period of research and development, these key patterns have been identified as weaknesses of the model, these weaknesses have been investigated further for presence in the dataset and shown to be categories that were overlooked or underrepresented in the dataset.

The model demonstrates a capability to generalization to generative models not explicitly included in training data such as Flux, Ideogram, Imagen 3, Nano banana, suggesting it has learned fundamental artifact patterns rather specific signatures related to the 3 explicitly sought after models in the dataset.

Heavy Post-Processing, professional photographs with extensive Photoshop editing (frequency-domain sharpening, synthetic bokeh, content-aware fill) trigger false positives due to artifact similarity with generative models, it is likely that the model has correctly identified artifact signatures present for AI and human images within the dataset, however the behaviour would suggest that the model has not learnt to identify AI images from the dataset, but has instead learnt to identify what it believes is not human, this problem is exacerbated by the amount of image processing modern images in 2026 undergo without user input, smartphones have widley adopted extreme processing pipelines to attempt to capture more

detail and improve images beyond what the initial small sensor captures on the phone, trying to mimic professional cameras that do not need as much processing due to the physical size of the sensor capturing more light and producing a cleaner image, some of the leading smartphone manufacturers are now also beginning to introduce automatic AI enhancement features in their camera apps that are used without user input. This is not a killing blow to the project however, the artifact signatures of these processes are still distinct from full AI generation of images, the system only struggles because these processes are different to the dataset witch captured human images uploaded between 2010 to 2020, the (V3) model process is competent and capable, further training from current weights with a larger dataset that now includes the weakness of the training data would likely allow the model to learn additional patterns for a more varied and generalised model.

Text heavy images, Memes, infographics, screenshots, and thumbnails containing rendered text are disproportionately classified as AI-generated. This likely stems from underrepresentation of text-heavy content in the training dataset, as most web scraping queries targeted photographic content.

technical Diagrams, Scientific plots, architectural blueprints, and vector graphics exhibit false positive rates above baseline. These images contain sharp edges and uniform colour regions that resemble GAN artifacts despite being human-created.

Partitioned images, many AI images in the dataset contain 4 versions of an image generated from a single prompt, see examples below, this format is widely used by image generation models so the user can pick which option they like the most, on real world data, the model classifies similar comparison images as AI more often than other formats.



Figure 32 partitioned images examples from the dataset

Future Improvements

The current dataset underrepresents several important categories that could be amended, these include:

More high resolution content

More professional photography, professional computer aided art

More text heavy content

More technical diagrams

Removal of comparison images from the dataset

Potential architecture refinements

Attention mechanisms, Adding cross-attention between RGB and artifact branches could potentially improve feature fusion

Multi-scale processing, Processing images at multiple resolutions or other resolutions such as 384×384 could capture artifacts better or capture different frequency bands, only 224x224 has been used so far

Convert to 3D, this model has proven to be capable of identifying complex artifact signitures in imagery, it is now a candidate for video classification as well, passing multiple subsequent frames into a 3D version of this model may prove an effective classifier for the video modality, this could potentially perform with higher accuracy due to video generation methods generally being less refined and powerful compared to image generation due to the higher emphases placed on efficiency when generating hundreds of frames, and also because subsequent frames in provide more data compared to a single image whitch is essentially just one frame.

Architecture rework, pre trained backbones do exist for high channel image classification, they were not used here because they are built for astronomy and medicine, and do not prioritise speed of size, however the possibility of using one should not be overlooked, perhaps a model would meet requirements if the large ConvNext 3 channel section were replaced with a high channel backbone and the architecture were unified into a single 17 channel branch.

Potential training improvements

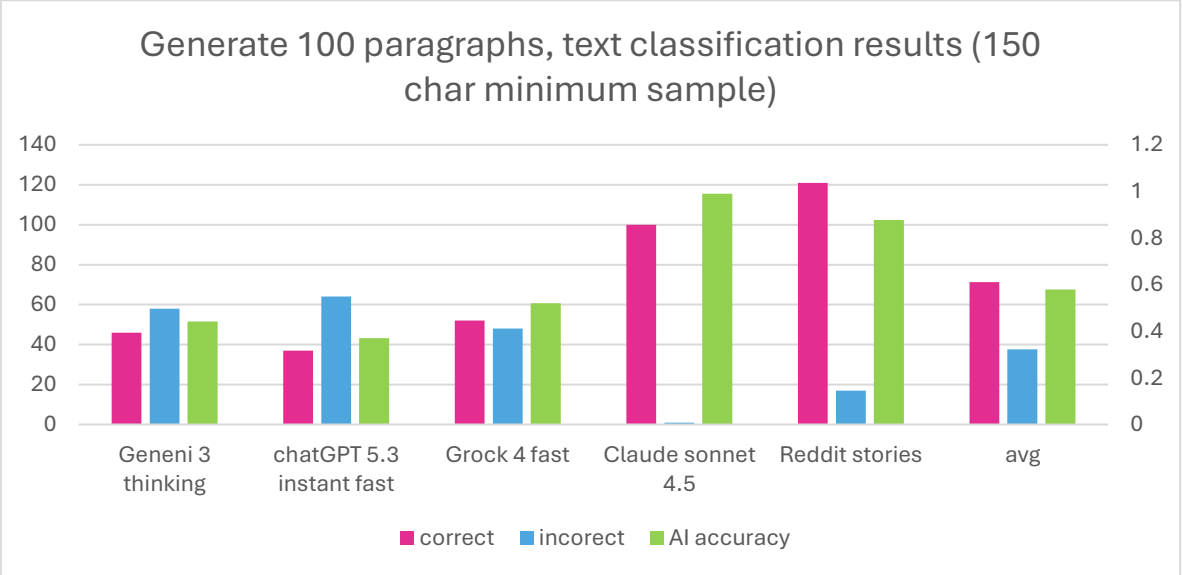
Unfreezing backbone, Fine-tuning the final ConvNeXt stage after initial convergence could improve accuracy at the cost of longer training time

Curriculum learning, Starting with easy training data from older ai models or time periods, and progressively introducing harder data with improved artifact suppression could improve convergence

Text classifier evaluation

Performance

In order to test accuracy of text classification, all other categories were disabled, 5 leading AI chatbots were given the prompt “Write 10 paragraphs about an animal of your choosing”, subsequent prompts were adapted to “Write 10 paragraphs about another animal of your choosing” until 10 prompts had been given, text classification accuracy was calculated based on the percentage of the generated paragraphs generated that were labelled as AI by the text classifier, the hybrid text classifier produces extreamley polarised results, most AI text is classified as either 99% or 100%, most human text is classified as 0%, this test was performed with a threshold of only 15% but changing it does not significantly affect results, the same threshold is also used on the controll group of human text, the main r/stories page was visited, texts were marked as correct if any text within the visible first segment of the story was classified as AI, the whole first page of reedit stories was classified totalling 138 samples reddit stories was seleced as the controll group after conducting the LLM generation because it has a similar casual writing tone to how the LLMs responded. Most models did not produce exactly 100 paragraphs. Model speed during this test was exceptional, the UI element in the options popout read the maximum displayable rate of 1000 texts/s.



Model	correct	incorrect	AI accuracy
Geneni 3 thinking	46	58	44%
chatGPT 5.3 instant fast	37	64	37%
Grock 4 fast	52	48	52%
Claude sonnet 4.5	100	1	99%

Reddit stories	121	17	88%
avg	71.2	37.6	58%

Figure 33 text classification results

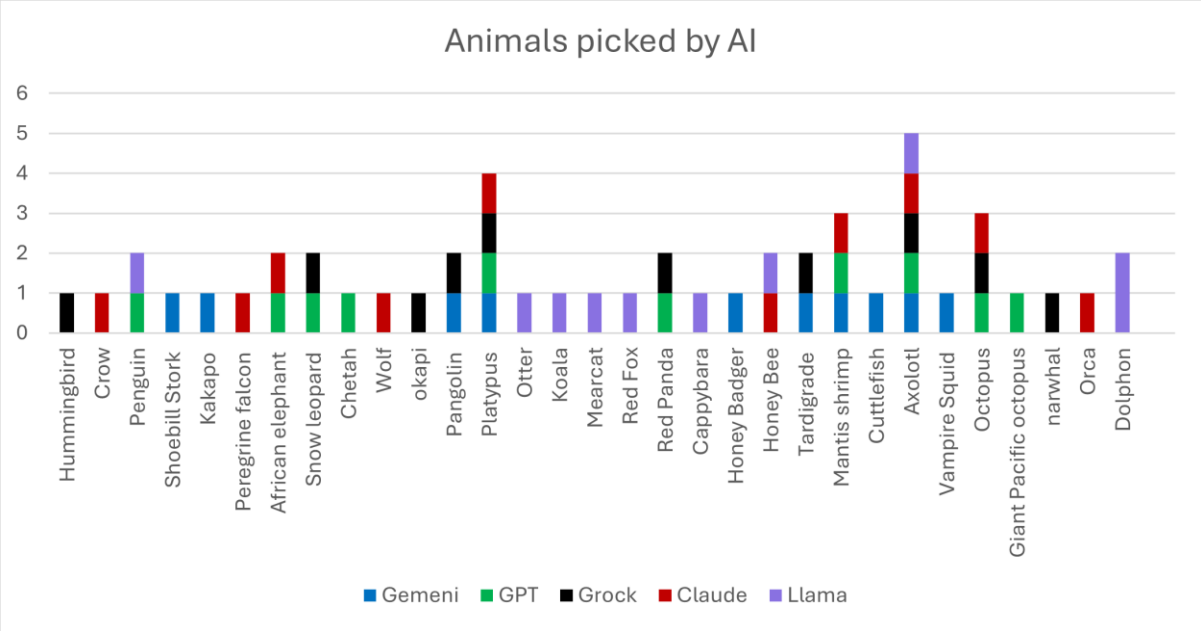


Figure 34 Generate 100 paragraphs, text classification results (150 char minimum sample)

Limitations

Gemeni, ChatGPT and Grock were prompted in their respective websites, however Claude by anthropic does not provide enough usage tokens to perform this test using their web browser, a VS code AI agentic coding extension with unlimited Claude tokens was used (Amazon Q) to generate Claude responses, generation happened in a new and empty workspace with no chat history, Claude sonnet produced the longest and most detailed paragraphs of all 4 LLMs, this same observation was true of the web interface, however amazon may use a modified version of Claude sonnet 4.5 or have additional prompt instructions to make Claude more detailed for improved coding performance, which may make it easier to detect for this classifier and explain the higher mark. Meta Llama 4 was also tested using the same methodology, it similarly does not have a web interface, instead the interface for WhatsApp on windows was used to generate paragraphs, however Llama 4 generated sentences too short for classification, the browser extension does not try to classify text shorter than 150 characters and therefore little text was flagged, performance does not meaningfully improve even after reducing the threshold to 50 characters.

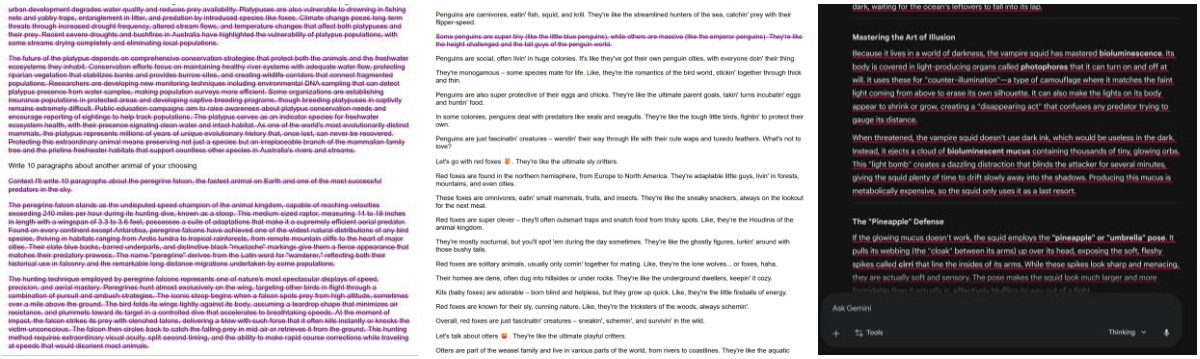


Figure 35 text classification test length variation

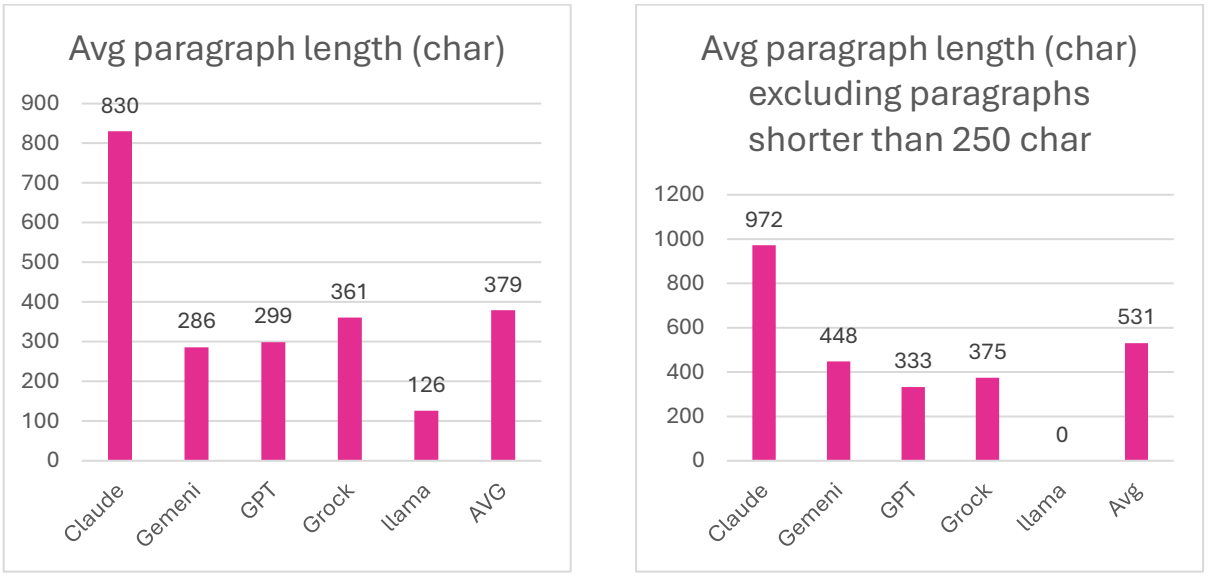


Figure 36 text classification test length variation graphs

The text classifier performed with poor performance, this model was overtrained and returned results in overly polarised confidence of either 0 or 1 instead of a gradient between, this lead to being unable to tune the classifier flagging using the threshold function in the options GUI, this is disappointing since the model clearly understanding the difference between AI and human text patterns, a significantly smaller percentage of text was classified as AI when it was written by humans, however the AI text classification had a success rate of ~50% or lower for most models, if we were able to lower the flagging threshold we could improve this score.

Furthermore, when comparing against a technical writing style, the model is significantly more likely to classify the text as AI, this is likely why the Claude model was also successful, it wrote in the most technical style of all 4 that were LLMs tested, when using this model on a Wikipedia article about cows, almost all paragraphs were classified as AI generated, this is likely due to signals like word diversity, readability, punctuation density and diversity, these signals are meant to capture the predictable and grammatically correct patterns of AI text generation, however they also correspond to more technical writing styles, perplexity should

be the most definitive signal, however the perplexity window was expanded for the V3 model because it was theorised that it would become more reliable if it could see the whole text, however the window was expanded to large and not enough perplexity measurements are taken, leading to this signal mostly being reduced to noise for V3

Future Improvements

The current iteration of the text classifier demonstrates an understanding of semantic and linguistic differences but carries the struggles mentioned above. To improve this there are multiple strategies available:

Upon initial testing, the bias against technical writing was believed to be solved with the introduction of the additional, majority human technical writing dataset and other supplementary variety texts, however even with a dataset of 329,926 samples, the low data density of text allowed the model to simply memorise the dataset and score 99% accuracy in training. A larger dataset is needed to alleviate this issue, and the model should be trained from scratch to forget the polarised confidence levels.

Redesigning the perplexity feature to use a smaller sliding window, or moving the window forward less after each calculation to better fit the small single paragraph text that Fire vs Fire produces, alternatively, the classifier could be redesigned to classify multiple text segments together as one, and return a unified score, this is possible using Fire vs Fire because chunks are sent to the local server in the order they appear in the DOM, and usually all text is loaded onto a page immediately, with no lazy loading, this strategy would not work for genuine short responses however, such as Grock responses mixed with human text on X (twitter).

Without a full retrain, the confidence level polarity could be fixed by implementing Platt Scaling or Isotonic Regression on the final output to convert the raw scores into a true representative probability distribution, but this model does not take long to train so this step is likely unnecessary, and a change to the polarity score calculation would necessitate a full retrain anyway.

The current model keeps the DistilBERT backbone relatively static. Future training could employ a less restrictive strategy, using lower learning rates for the earlier layers to retain general linguistic knowledge while allowing the final layers to amplify learning.

Implementing a small ML model or directory into the classifier that identifies the website's genre from the text itself or URL, such as social media, news, or academic, the system could then choose from multiple different pre trained models that specialise in a specific genre of text, this would more closely align with how publicly available datasets are organised into many distinct writing styles, it may be true that it is simply impossible to make a true general single model system when data density is so low compared to the other modalities.

Future Work

Video and audio

While the original project proposal aimed to classify four modalities—Text, Images, Video, and Audio, the Minimum Viable Product (MVP) focuses exclusively on the two largest categories: text and image classification.

The decision to label Audio and Video for future work was a choice based on factors discovered during development:

- Compute Complexity – Real time on device classification of raw video streams requires significantly higher hardware resources which may exceed hardware capabilities of lower end consumer hardware.
- Implementation Timeframes – Developing high-accuracy models for four distinct modalities within the strict academic deadlines proved unrealistic when paired with additional mounting responsibilities.
- Setup Complexity – An effective method for flagging suspected AI audio has not been finalised.
- Feature creep – In place of spending time on audio and video classification support, other more varied priorities were perused such as support for user created classifiers, support for multiple classification categories, increased security and performance using base 64 encoding of images for transmission to the local server.

By prioritizing text and images, the project demonstrates functional proof of concept that addresses the most common forms of the AI threat encountered during general web browsing.

The cache of recently classified content was implemented successfully but begun failing at some point and was not discovered in time to be fixed before the deadline, and so it was disabled.

Additional features

the local app is currently a simple python file, while this functional solution works well within the time constraints of this project, a graphical user interface (GUI) and additional features could be added for creature functionality and capabilities

potential features for the system include:

- GUI Interface – A GUI interface capable of managing user setting, linked to the setting of the browser extension
- Integrated model training interface – Develop an interface for training simple classification models for less technical users, these users would only need to provide a dataset of content they wish to filter, and a model could be trained locally against a generic database of generic web data for that modality, collected for this project.



- Inspired by a video posted by (josh :), 2026), adding the option to replace suspected images with art or other content could be a valuable feature.
- A potential public space for sharing of community developed classifiers,
- Implement memory of recently classified content inside the local classification server, to prevent classifying twice. This would also solve the issue of rapidly reloading the page causing the extension to have to wait a long time for multiple reclassifications.
- Investigate possibility of in serverless computation, a mong other factors, a working prototype was necessary to be developed within the timeframe so the safe option of a backend server was chosen for development over a browser solution with a simpler installation process, however drawbacks of webGL and memory considerations were never properly tested and may have been overzealous.
- Implement a messaging system from the options GUI to classifiers, allowing classifiers to be passed custom settings defined by the classifier from the user without having to go into the classifier file, users could be able to add custom interface settings or other elements after the default classifier/category options.

Additional platforms

This project aims to useful for owner of all devices and users with varied levels of technical knowledge, this project will be considered a greater success with greater adoption, therefore as many platforms as possible should be supported

- Develop support for other web browsers, chrome, edge, opera, others…
- Develop support for other operating systems, MacOS, Linux.
- Most mobile devices are not powerful enough to run the classification models used in this system, however according to (Kemp, 2025), only 46% of uk web traffic was linked to desktop/laptop users, while 51% was found to be mobile and tablet traffic, it could be possible for users to link their own and their families mobile devices to their desktop machines for classification purposes over LAN networks or even WAN in order to allow for use of the system on mobile devices, alternatively mobile devices with high end flagship processors such as Snapdragon 8 Elite Gen 5, A19 pro processors or M series processors in iPad pro may be powerful enough to run models locally.

Legal, Social, Economic, and Ethical Considerations

Legal Considerations

The legal landscape surrounding AI detection is complex, primarily concerning data privacy, the use of AI training data and the intellectual property rights of those who own data to be trained on remains a legal grey zone, many corporations outside the immediate AI bubble are hesitating on adopting new technologies until firm legal clarification is reached (Hampton, 2026). This project is developed using both public datasets and a large bespoke dataset gathered using public images, however no money is made from this project so legal concerns aren't as extreme or probable compared to larger entities.

This system does not collect or transmit any user data, and is fully GDPR compliant by virtue of the privacy by design approach, which is a core feature of Fire vs Fire that sets it apart from other solutions

This tool will make mistakes, if the tool fails to identify or flag harmful content such as the ai generated foraging book (ecoloma, 2024), the developer could potentially face liability claims from victims who thought they were protected against such threats by Fire vs Fire, to mitigate this all users must be aware that this is an AI tool that is expected to make mistakes, and any important classification should be conducted using separate means.

The browser extension was developed using the Firefox manifest V2 API and its native messaging solutions, as Fire vs Fire is intended to be distributed and published on the Firefox extension store, it had to be developed to be compliant with all Firefox policies and legal guidelines.

Social Considerations

The social impact of Fire vs Fire focuses on democratizing digital safety and improving AI literacy.

Research indicates that populations of western countries and elderly individuals statistically possess the lowest levels of AI literacy Allen (2025). Fire vs Fire's detection process automates the burden of suspicion, protecting users who may not realize they are being exposed to synthetic media.

For users who are aware of the problem but cannot reliably identify AI generated misinformation or content, this system aims to reintroduce the public trust that has evaporating due to the difficulty of distinguishing synthetic and authentic media and rebuild confidence in digital interfaces.

This tool empowers users with the ability to create custom classifiers and categories; however, it runs the risk of enabling users to worsen already prevalent echo chambers or social biases online. This could lead to a fragmented reality where users only see content that



manages to enter their personalized and restrictive bubble, it is difficult to include protective measures against misuse of an open source project when users can simply remove them, however certain proposed future features could be regulated to stop misuse from users who do not know how to reprogram Fire vs Fire, for example a inbuilt classifier trainer from a prompt could contain a simple AI classifier for ensuring the prompt is not malicious, a public registry for user made classifiers could also be regulated.

Economic Considerations

The economic analysis compares the Fire vs Fire model against the current market of paid alternatives.

Competitors such as Winston AI and GPTZero utilize subscription models, often charging £8 £13 per month for even the lowest priced plans. Such costs are prohibitive for the public who know of the risks of AI manipulation, however are not willing to pass such a high barrier of entry.

While the cost of using Fire vs Fire is not free, due to the increased power usage on devices, the annual energy cost for a user to run the tool locally is negligible even at daily peak 2026 UK energy rates during a global oil crisis. Alternative services do not compare well as they are performing a different task of selective classification and would stop users would they attempt to classify similar volumes as Fire vs Fire, if these services were to allow such high throughput at current rates, the price annually would easily reach 5 figures or more.

Ethical Considerations

The modular architecture allows users to develop their own classifiers, this feature could be use maliciously to identify and block types of content that could worsen political bubbles and echo chambers, or used against marginalized groups, specific ethnicities, or dissenting political opinions. This problem is emphasized by the commitment to maintain an open and modular architecture. While this transparency allows the community to maintain and adapt the system to their individual needs, it makes fortifying the system against misuse or reprogramming for malicious means.

Since machine learning models are ineffable, it is often impossible for the developer to explain exactly why a specific piece of media was flagged, or for developers to be certain that their models do not contain unethical biases. Users should also be aware of the potential for false positives that could lead to unwanted confrontation of real world consequences.

References

ALEXANDER, A., 2026. From SETI@home to Hominid Fossils: Citizen Cyberscience Reshapes…. *The Planetary Society* [online]. Available from: https://www.planetary.org/articles/setiathome_20080115 [Accessed 29 Apr 2026].

ALLEN, L., 2025. Majority of the UK public lack AI education or training and don't trust the technology. *KPMG* [online]. Available from: https://kpmg.com/uk/en/media/press-releases/2025/04/majority-of-uk-public.html.

AMAZON, 2026. Amazon.co.uk Help: Conditions of Use. *www.amazon.co.uk* [online]. Available from: https://www.amazon.co.uk/gp/help/customer/display.html?nodeId=GLSBYFE9MGKKQXXM.

ARTEM9K, 2023. artem9k/ai-text-detection-pile · Datasets at Hugging Face. *huggingface.co* [online]. Available from: https://huggingface.co/datasets/artem9k/ai-text-detection-pile.

BORJI , A., 2024. Qualitative Failures of Image Generation Models and Their Application in Detecting Deepfakes. *Arxiv.org* [online]. Available from: https://arxiv.org/html/2304.06470v6 [Accessed 28 Apr 2026].

BOUAISHA, M.M. and ELSHEH, M., 2025. Audio Deep Fake Detection Using an Ensemble Learning Approach. (2) 29 .*مجلة البحوث الأكاديمية*, pp. 103–110.

BYNDER, 2024. Bynder's 'Human Touch' survey uncovers consumers opinions on the use of AI in content creation. *Bynder* [online]. Available from: https://www.bynder.com/en/blog/bynders-human-touch-survey-uncovers-consumers-opinions/.

CHROME, 2019. Personalise Chrome performance - Google Chrome Help. *Google.com* [online]. Available from: https://support.google.com/chrome/answer/12929150?hl=en-GB [Accessed 28 Apr 2026].

CROITORU, F.A., IONESCU , R.T., and HONDRU, V., 2020. PRNU-Bench: A Novel Benchmark and Model for PRNU-Based Camera Identification. *Arxiv.org* [online]. Available from: https://arxiv.org/html/2509.17581v1 [Accessed 28 Apr 2026].

CSUN22, 2025. GitHub - csun22/Synthetic-Voice-Detection-Vocoder-Artifacts: This repository is related to our Dataset and Detection code from the paper: AI-Synthesized Voice Detection Using Neural Vocoder Artifacts accepted in CVPR Workshop on Media Forensic 2023. *GitHub* [online]. Available from: https://github.com/csun22/Synthetic-Voice-Detection-Vocoder-Artifacts [Accessed 28 Apr 2026].

CYBERONIX, 2025. Deepfake CEO Fraud: AI-Powered Phishing Scam Costs Tech Firm $2.3M - Cyberonix Blog. *Cyberonix Blog* [online]. Available from: https://blog.cyberonix.net/security/2025/04/deepfake-ceo-fraud-ai-powered-phishing-scam-costs-tech-firm-2-3m/.

DENG, Z., 2026. spiderTraining100-500. *Huggingface.co* [online]. Available from: https://huggingface.co/datasets/zkdeng/spiderTraining100-500 [Accessed 3 May 2026].

DILBAR , S., ALI QURESHI , M., NOON, S.K., and MANNAN, A., 2025. *Spectrogram comparison of original and disguised voice samples.* [online]. https://www.researchgate.net/figure/Spectrogram-comparison-of-original-and-disguised-voice-samples_fig4_397619911. Available from: https://www.researchgate.net/publication/397619911/figure/fig4/AS:11431281745414083@

1764172694063/Spectrogram-comparison-of-original-and-disguised-voice-samples.tif [Accessed 28 Apr 2026].

DUGAN, L., 2026. raid. *Huggingface.co* [online]. Available from: https://huggingface.co/datasets/liamdugan/raid [Accessed 4 May 2026].

DV, 2024. human_ai_generated_text. *Huggingface.co* [online]. Available from: https://huggingface.co/datasets/dmitva/human_ai_generated_text [Accessed 4 May 2026].

ECOLOMA, 2024. Family poisoned after using AI-generated foraging book bought online. *POP!* [online]. Available from: https://pop.inquirer.net/366719/family-poisoned-after-using-ai-generated-foraging-book-bought-online.

EDMO, 2026. AI Slop: How Greed Is Affecting Democracies – EDMO. *Edmo.eu* [online]. Available from: https://edmo.eu/publications/ai-slop-how-greed-is-affecting-democracies/ [Accessed 27 Apr 2026].

ERUKUDE, S.T., MARELLA, V.C., and VELURU, S.R., 2025. Fourier-Based GAN Fingerprint Detection Using ResNet50. *2025 9th International Conference on Inventive Systems and Control (ICISC)* [online]. pp. 971–976. Available from: https://ieeexplore.ieee.org/document/11187724 [Accessed 15 Apr 2026].

EXPLOSIVE MEDIA and IRAN, 2026. Your goverment is run by pedophiles. *Explosive Media* [online]. Available from: https://commons.wikimedia.org/wiki/File:New_Lego_Video_From_Iran_Losers_First!-1920X1080-Avc1-Mp4a.webm [Accessed 26 Apr 2026].

FOLDING@HOME, 2026. Folding@home. *Folding@home* [online]. Available from: https://foldingathome.org/.

GOLD, H. and REILLY , L., 2025. A ‘rogue employee’ was behind Grok’s unprompted ‘white genocide’ mentions | CNN Business. *CNN* [online]. Available from: https://edition.cnn.com/2025/05/16/business/a-rogue-employee-was-behind-groks-unprompted-white-genocide-mentions [Accessed 26 Apr 2026].

GOOGLE, 2019. Verify Google AI-generated images, videos, and audio with SynthID - Computer - Gemini Apps Help. *Google.com* [online]. Available from: https://support.google.com/gemini/answer/16722517?hl=en&co=GENIE.Platform%3DDesktop&oco=1 [Accessed 27 Apr 2026].

GOOGLE, 2025. Performance Max: AI-Enhanced Google Ads That Convert - Google Ads. *Google Business* [online]. Available from: https://business.google.com/us/ad-solutions/performance-max/.

GPTZERO, 2023. Pricing | GPTZero. *GPTZero* [online]. Available from: https://gptzero.me/pricing.

HADESTTW AI BLOCKER, 2025. AI Blocker. *Mozilla.org* [online]. Available from: https://addons.mozilla.org/en-US/firefox/addon/ai-

blocker/?utm_source=addons.mozilla.org&utm_medium=referral&utm_content=search [Accessed 27 Apr 2026].

HAMPTON, T., 2026. The Future? Artificial Intelligence and Intellectual Property, An Unresolved Question of Training Data - Taylor Hampton. *Taylor Hampton* [online]. Available from: https://taylorhampton.co.uk/the-future-artificial-intelligence-and-intellectual-property-an-unresolved-question-of-training-data/ [Accessed 1 May 2026].

HEMANT APPA TIRMARE, 2026. Analysis on Video Forgery Detection Using Spatial Temporal Feature with 3D CNN. *Journal of Internet Services and Information Security* [online]. 16 (1), pp. 570–594. Available from: https://www.researchgate.net/publication/401730243_Analysis_on_Video_Forgery_Detection_Using_Spatial_Temporal_Feature_with_3D_CNN.

HIVE AI, 2024. Hive. *Hive* [online]. Available from: https://thehive.ai/apis/ai-generated-content-classification.

ICAEW INSIGHTS, 2025. How to guard against voice cloning and deepfake scams. *Icaew.com* [online]. Available from: https://www.icaew.com/insights/viewpoints-on-the-news/2025/jan-2025/how-to-guard-against-voice-cloning-and-deepfake-scams.

IMAM, M.Y., 2026. Why 'AI Watermarks' Are Easy to Remove?. *Medium* [online]. Available from: https://medium.com/activated-thinker/why-ai-watermarks-are-easy-to-remove-042794e908f1 [Accessed 27 Apr 2026].

IMF, 2024. World Economic Outlook. *International Monetary Fund* [online]. Available from: https://www.imf.org/en/publications/weo.

JOSH :), 2026. I Replaced Every Ad on the Internet With Art. *YouTube* [online]. Available from: https://www.youtube.com/watch?v=dDdIGfo7SaE [Accessed 27 Apr 2026].

KEMP, S., 2025. Welcome To Zscaler Directory Authentication. *Datareportal.com* [online]. Available from: https://datareportal.com/reports/digital-2026-united-kingdom.

LANCUCKI, A., 2021. Fastpitch: Parallel Text-to-Speech with Pitch Prediction. Available from: https://www.researchgate.net/publication/352171306_Fastpitch_Parallel_Text-to-Speech_with_Pitch_Prediction [Accessed 28 Jul 2023].

LECUN, Y., BENGIO, Y., and HINTON, G., 2015. Deep Learning. *Nature* [online]. 521 (7553), pp. 436–444. Available from: https://www.nature.com/articles/nature14539.

LIU, Y., ZHAO, X., SONG, D., WORNELL, G., BARBARA , S., BERKELEY, and MASSACHUSETTS INSTITUTE OF TECHNOLOGY , 2026. Position: LLM Watermarking Should Align Stakeholders' Incentives for Practical Adoption. *Arxiv.org* [online]. Available from: https://arxiv.org/html/2510.18333v2 [Accessed 27 Apr 2026].

MA, L., CUI, B., ZHANG, W., ZHANG, J., LIU, Y., LI, Z., LIN, J., and LIANG, H., 2023. Evaluating and Predicting Distorted Human Body Parts for Generated Images. *Arxiv.org* [online]. Available from: https://arxiv.org/html/2503.00811v1 [Accessed 28 Apr 2026].

MACROTRENDS, 2026. NVIDIA Market Cap 2006-2020 | NVDA. *www.macrotrends.net* [online]. Available from: https://www.macrotrends.net/stocks/charts/NVDA/nvidia/market-cap.

MACZAN , J., 2024. Characterizing WebGPU Dispatch Overhead for LLM Inference Across Four GPU Vendors, Three Backends, and Three Browsers. *Arxiv.org* [online]. Available from: https://arxiv.org/html/2604.02344v1 [Accessed 28 Apr 2026].

MACZAN, J., 2026. Characterizing WebGPU Dispatch Overhead for LLM Inference Across Four GPU Vendors, Three Backends, and... *ResearchGate* [online]. Available from: https://www.researchgate.net/publication/403529940_Characterizing_WebGPU_Dispatch_Overhead_for_LLM_Inference_Across_Four_GPU_Vendors_Three_Backends_and_Three_Browsers [Accessed 28 Apr 2026].

MBZUAI-NLP, 2025. GitHub - mbzuai-nlp/M4: M4: Multi-generator, Multi-domain, and Multi-lingual Black-Box Machine-Generated Text Detection. *GitHub* [online]. Available from: https://github.com/mbzuai-nlp/M4 [Accessed 4 May 2026].

MENG, J. and XING, R., 2024. Inside the 'black box': Embedding clinical knowledge in data-driven machine learning for heart disease diagnosis. *Nih.gov* [online]. Available from: https://pmc.ncbi.nlm.nih.gov/articles/PMC9795264/ [Accessed 28 Apr 2026].

META, 2022. Meta Advantage+ Creative: Social Media Ads AI Generator Tool. *Meta for Business* [online]. Available from: https://www.facebook.com/business/ads/meta-advantage-plus/creative.

META SUPERINTELLIGENCE, 2025. Meta Seal. *Meta Seal* [online]. Available from: https://facebookresearch.github.io/meta-seal/ [Accessed 27 Apr 2026].

MINDNER, L., SCHLIPPE, T., and SCHAAFF, K., 2023. *Classification of Human-and AI-Generated Texts: Investigating Features for ChatGPT* [online]. Available from: https://arxiv.org/pdf/2308.05341.

MORLEY, K.R., 2025. National Grid: Live Status. *National Grid: Live* [online]. Available from: https://grid.iamkate.com/.

MOZILLA, 2023. Native messaging - Mozilla | MDN. *developer.mozilla.org* [online]. Available from: https://developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions/Native_messaging.

NOELS , S., ROGIERS , A., and BUYL , M., 2026. Persuasion with Large Language Models: A Survey of Empirical Evidence, Study Methodologies, and Ethical Implications. *Arxiv.org* [online]. Available from: https://arxiv.org/html/2411.06837v2 [Accessed 27 Apr 2026].

O'BRIEN, M. and ASSOCIATED PRESS, 2025. Why was Elon Musk's AI chatbot Grok preoccupied with South Africa's racial politics? *KSAT* [online]. Available from: https://www.ksat.com/business/2025/05/15/why-was-elon-musks-ai-chatbot-grok-preoccupied-with-south-africas-racial-politics/ [Accessed 27 Apr 2026].

ODENA, A., DUMOULIN, V., and OLAH, C., 2016. Deconvolution and Checkerboard Artifacts. *Distill*. 1 (10).

OMIDI, R., DONG, Y., and WANG , B., 2026. On Google's SynthID-Text LLM Watermarking System: Theoretical Analysis and Empirical Validation. *Arxiv.org* [online]. Available from: https://arxiv.org/html/2603.03410v2 [Accessed 27 Apr 2026].

OUR WORLD IN DATA, 2026. Computation used to train notable artificial intelligence systems, by domain. *Our World in Data* [online]. Available from: https://ourworldindata.org/grapher/artificial-intelligence-training-computation?time=1950-07-02..2026-02-17 [Accessed 25 Apr 2026].

RAZA, A., BASIT, A., AMIN, A., ARFEEN, Z.A., MASUD, M.I., FAYYAZ, U., and JUMANI, T.A., 2026. A Comprehensive Review of Deepfake Detection Techniques: From Traditional Machine Learning to Advanced Deep Learning Architectures. *AI*. 7 (2), p. 68.

SEISMOCAT, 2025. *Actual 'difference' between real and ai generated images* [online]. https://www.reddit.com/r/interesting/comments/1jopvdq/actual_difference_between_real_and_ai_generated/. Available from: https://preview.redd.it/actual-difference-between-real-and-ai-generated-images-v0-wy7r66rde6se1.png?width=1080&crop=smart&auto=webp&s=861010f3c3a7a987f7d3669a4b858b593fd0f195 [Accessed 28 Apr 2026].

SHANNON, C., 1950. Maze-Solving Mouse. *cyberneticzoo.com* [online]. Available from: https://cyberneticzoo.com/mazesolvers/1951-maze-solving-mouse-claude-shannon-american/ [Accessed 25 Apr 2026].

SKIPSLOP BY TANATHY, 2026. SkipSlop. *Mozilla.org* [online]. Available from: https://addons.mozilla.org/en-GB/firefox/addon/skipslop/ [Accessed 27 Apr 2026].

SPOTIFY, 2025. Spotify Strengthens AI Protections for Artists, Songwriters, and Producers — Spotify. *Spotify* [online]. Available from: https://newsroom.spotify.com/2025-09-25/spotify-strengthens-ai-protections/.

STORRITO, 2026. TikTok AI Generated Content Policy and Labeling Requirements in 2026. *Storrito.com* [online]. Available from: https://storrito.com/resources/tiktoks-2026-ai-labeling-rules-and-what-they-signal-for-platform-governance/ [Accessed 27 Apr 2026].

SUGAWARA, Y., SHIOTA, S., and KIYA, H., 2019. Checkerboard artifacts free convolutional neural networks. *APSIPA Transactions on Signal and Information Processing*. 8.

THOMAS, H., 2026. AI-generated images behind increase in insurance fraud. *BBC News* [online]. 15 April. Available from: https://www.bbc.co.uk/news/articles/cm2rr9pg4jzo.

WALDEN, V.M., 2024. Revisiting Benford's Law with added AI horsepower. *Acfe.com* [online]. Available from: https://www.acfe.com/fraud-magazine/all-issues/issue/article?s=2024-novdec-revisiting-benfords-law.

WINSTON AI, 2026. Winston AI detector and plagiarism checker. *Winston AI* [online]. Available from: https://gowinston.ai/pricing/ [Accessed 27 Apr 2026].

X, 2024. X Terms of Service. *X* [online]. 15 November. Available from: https://x.com/en/tos.

YADAV, S. and KUMAR, A., 2025. Combining Deep Learning and Forensic Analysis for Robust Fake Image Detection. *International Journal on Science and Technology (IJSAT) IJSAT25026064* [online]. 16 (2). Available from: https://www.ijsat.org/papers/2025/2/6064.pdf.

YOUTUBE, 2019. Disclosing use of altered or synthetic content. *Google.com* [online]. Available from: https://support.google.com/youtube/answer/14328491 [Accessed 27 Apr 2026].

YU, H. and XU, B., 2024. Multi-modal texture fusion network for detecting AI-generated images. *Nih.gov* [online]. Available from: https://pmc.ncbi.nlm.nih.gov/articles/PMC12586049/ [Accessed 28 Apr 2026].

ZEIGERMANN, O., 2022. TensorFlow.js: What is possible with Machine learning in the browser? *devmio - Software Know-How* [online]. Available from: https://devm.io/machine-learning/tensorflow-machine-learning-browser-javascript [Accessed 28 Apr 2026].

ZHANG, D., LI, C., LIN, F., ZENG, D., and GE, S., 2021. *Detecting Deepfake Videos with Temporal Dropout 3DCNN* [online]. Available from: https://www.ijcai.org/proceedings/2021/0178.pdf.

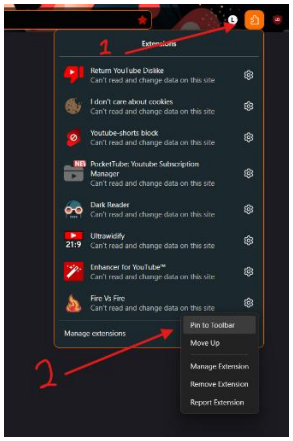
Appendices

Appendix A: instillation instructions:

Open PowerShell in the root Fire vs Fire folder, then run:

powershell -ExecutionPolicy Bypass -File .\native\install-firefox.ps1

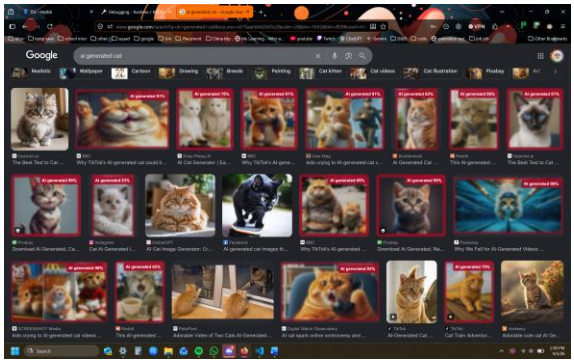
This creates the bridge between the Firefox browser and the native host, it also creates and installs the python virtual environment and all dependencies, be aware that this script edits two registers on your computer, you can revert this if you want to after you are finished testing by running:
powershell -ExecutionPolicy Bypass -File .\native\uninstall-firefox.ps1
Once you have installed the local server, visit this address In the Firefox address bar:
about:debugging#/runtime/this-firefox
Then click the load temporary add on button and select the manifest file that is in the root Fire vs Fire file. Then pin the extension to your taskbar for the best experience in using the extensions drop down in the top right corner:

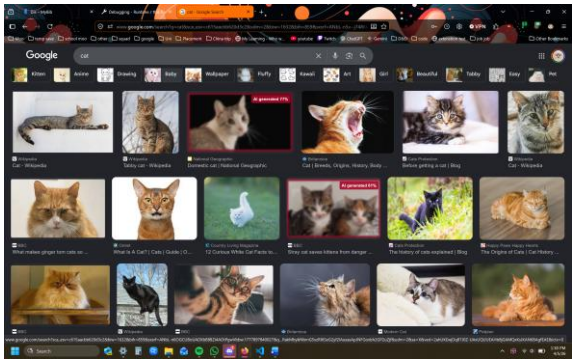


You may need to remove any default categories and make new ones to make it work.
This process worked on a windows laptop that had never had Fire vs Fire installed before.

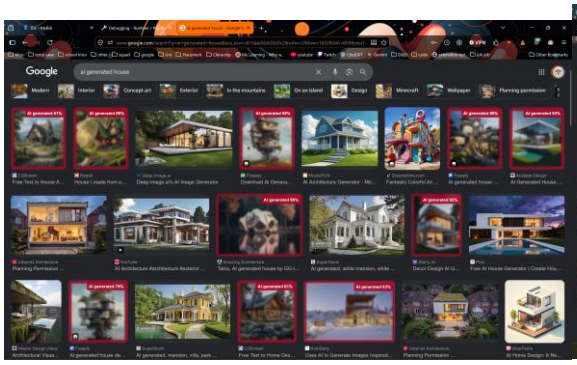
Appendix B: ConvNext evaluation evidence:

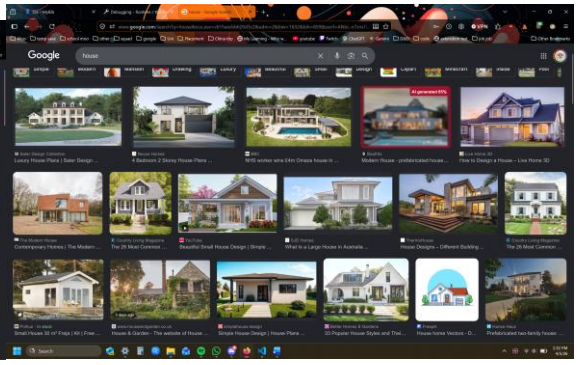
Cat



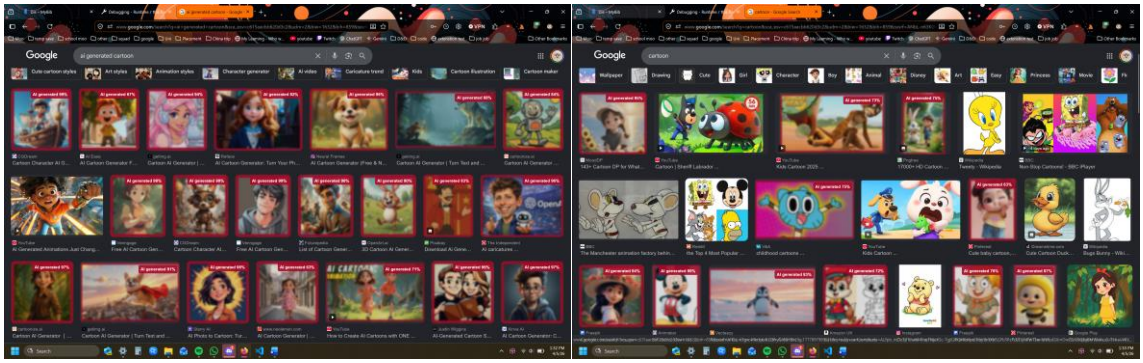


House





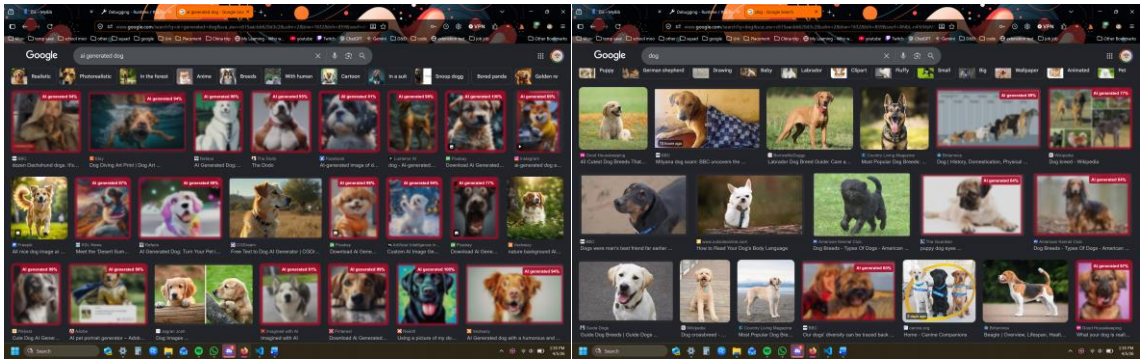
Cartoon



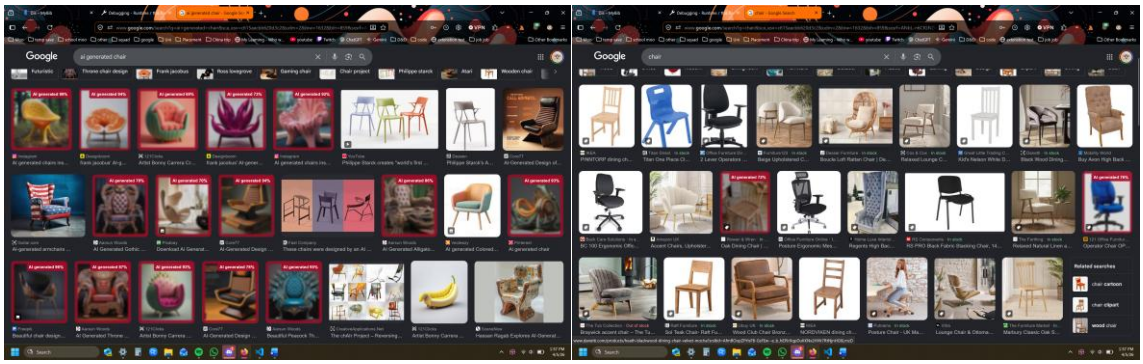
Person



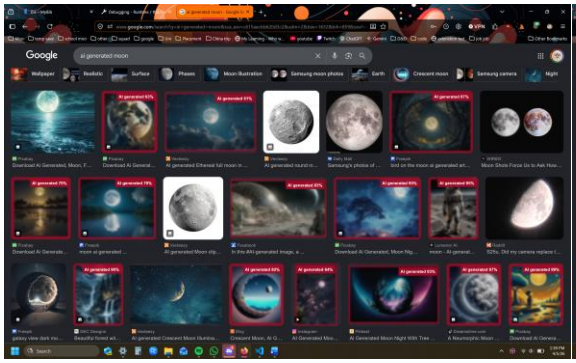
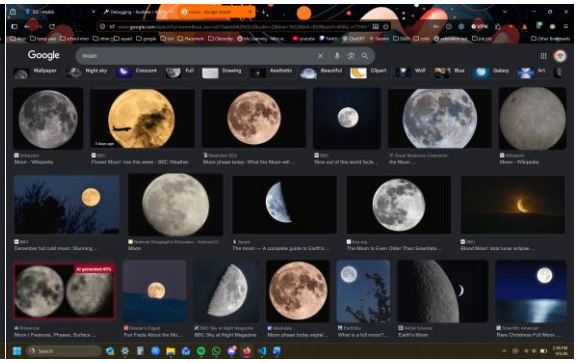
Dog



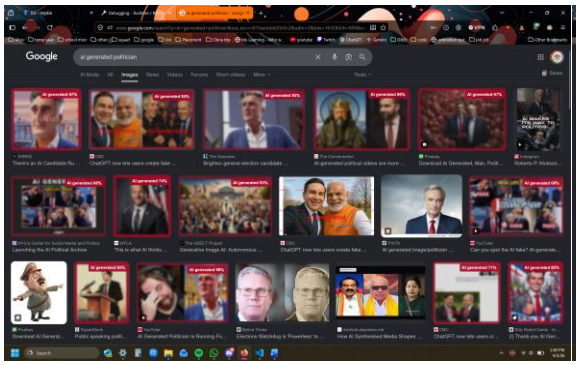
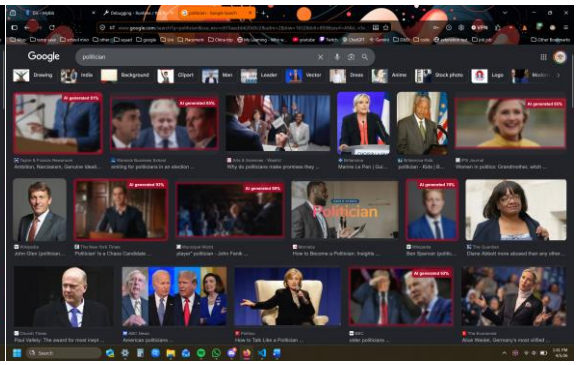
Chair



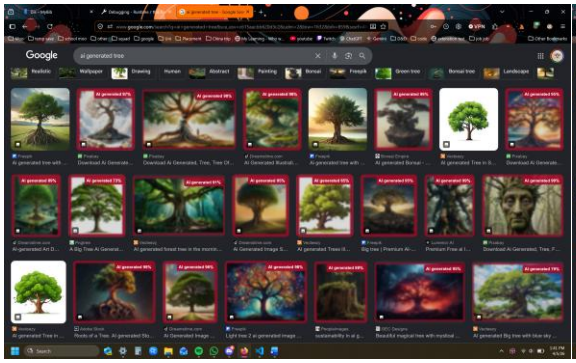
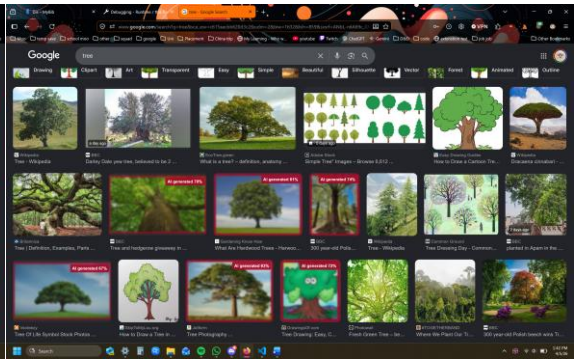
Moon

Politician

Tree

Art

